

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH VIỄN THÔNG



HCMUTE

KHÓA LUẬN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Đề tài

TRIỂN KHAI ADAS TRÊN MÔ HÌNH XE TỰ HÀNH DÙNG

SOC EDGE AI

Sinh viên: **TRẦN GIA HUY**

MSSV: 22119182

LÊ HỒNG PHONG

MSSV: 22119210

Hướng dẫn: **ThS. NGUYỄN NGÔ LÂM**

TP. HỒ CHÍ MINH tháng 6 năm 2026



NHIỆM VỤ KHÓA LUẬN TỐT NGHIỆP

Họ và tên sinh viên: Trần Gia Huy – Lê Hồng Phong

MSSV: 22119182 – 22119210

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 22119B

Giảng viên hướng dẫn: Th.S Nguyễn Ngô Lâm

Ngày nhận đề tài: 1/4/2026

Ngày nộp đề tài: 26/6/2026

1. Tên đề tài: Triển khai ADAS trên mô hình xe tự hành dùng SoC Edge AI.

2. Các số liệu, tài liệu ban đầu

- Kiến thức cơ bản về các môn Mạch điện, Điện tử cơ bản, Vi xử lý, Học máy, Xử lý ảnh và trí tuệ nhân tạo, Học máy ứng dụng.
- Lý thuyết về lập trình phần cứng và các giao thức truyền dữ liệu mạng di động.
- Tài liệu datasheet, thông số kỹ thuật đi kèm của các linh kiện, module sử dụng trong đề tài.
- Lập trình STM32 HAL, C++ OOP, Linux, Visual Studio Code, thiết kế PCB trên EasyEDA.

3. Nội dung thực hiện đề tài:

- Tìm hiểu về các giao thức truyền dẫn dữ liệu.
- Tìm hiểu nguyên lý hoạt động của các vi điều khiển, module sử dụng trong đề tài.
- Xây dựng và lựa chọn mô hình AI phát hiện đèn giao thông.
- Xử lý ảnh từ camera để phát hiện làn đường.
- Phát triển giải thuật để xe giữ được làn đường khi đang chạy.
- Thiết kế sa bàn để xe chạy.
- Thiết kế phần cứng (PCB, khung xe 3D, ...)
- Chỉnh sửa và kiểm tra phần cứng.
- Viết báo cáo.

4. Sản phẩm

- Mô hình xe tự hành (raspberry pi 4, mạch điều khiển xe, pin 3S 21700, khung xe).
- Sa bàn 1 làn đường rộng 2.5m x 2.5m.
- Quyển báo cáo đề tài.

GIẢNG VIÊN HƯỚNG DẪN

BỘ MÔN KT MÁY TÍNH VIỄN THÔNG

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: TRIỂN KHAI ADAS TRÊN MÔ HÌNH XE TỰ HÀNH DÙNG SOC EDGE AI

Sinh viên: + TRẦN GIA HUY

MSSV: 22119182

+ LÊ HỒNG PHONG

MSSV: 22119210

Hướng dẫn: ThS. NGUYỄN NGÔ LÂM

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Tp. HCM, ngày ... tháng năm 20...

Người nhận xét

(Ký và ghi rõ họ tên)

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện khóa luận tốt nghiệp tại trường Đại học Công Nghệ Kỹ Thuật Thành phố Hồ Chí Minh, chúng tôi đã nhận được nhiều sự giúp đỡ, hỗ trợ từ các Thầy, Cô, bạn bè, và gia đình. Nhân dịp hoàn thành đồ án này, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến tất cả mọi người đã giúp đỡ chúng tôi trong suốt thời gian qua.

Chúng tôi xin gửi lời cảm ơn chân thành đến các Thầy, Cô trong khoa Điện –Điện tử. Những kiến thức quý báu và sự hướng dẫn nhiệt tình của quý Thầy, Cô đã giúp chúng tôi có nền tảng vững chắc để hoàn thành đồ án này. Đặc biệt, chúng tôi xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn, ThS Nguyễn Ngô Lâm, người đã tận tâm hướng dẫn, tạo điều kiện, và có những góp ý quý báu giúp chúng tôi hoàn thiện đồ án. Những hướng dẫn, góp ý tận tình và kịp thời của thầy đã giúp chúng tôi định hướng được phương pháp, cách làm, từ đó hoàn thành tốt và đúng hạn những yêu cầu được giao.

Bên cạnh đó, chúng tôi cũng gửi lời cảm ơn tới gia đình, người thân, bạn bè của chúng tôi, đã luôn ủng hộ, động viên chúng tôi trong quá trình thực hiện đề tài. Chúng tôi cũng gửi lời cảm ơn tới các tác giả nghiên cứu về lý thuyết và ứng dụng của định vị nói riêng và các tác giả đã từng nghiên cứu về hệ thống chống trộm nói chung, đã là nguồn tài liệu tham khảo hữu ích giúp chúng tôi đưa ra những ý tưởng và hoàn thành đề tài một cách thuận lợi và nhanh chóng.

Vì thời gian nghiên cứu có hạn, trình độ hiểu biết của bản thân còn nhiều hạn chế nên bài báo cáo của chúng tôi không tránh khỏi những thiếu sót, chúng tôi rất mong nhận được sự góp ý quý báu của tất cả các thầy, cô để bài báo cáo của chúng tôi được hoàn thiện hơn.

Nhóm thực hiện

TÓM TẮT

Trong xu thế phát triển mạnh mẽ của công nghệ ô tô hiện đại, hệ thống hỗ trợ lái xe nâng cao (ADAS) đóng vai trò cốt lõi trong việc đảm bảo an toàn và hướng tới tự động hóa hoàn toàn. Đề tài "Triển khai ADAS trên mô hình xe tự hành dùng SoC Edge AI" được thực hiện nhằm mục đích nghiên cứu, thiết kế và chế tạo một mô hình xe tự hành có khả năng nhận biết môi trường xung quanh và tự động đưa ra các quyết định điều khiển thích hợp thông qua các thuật toán xử lý ảnh và trí tuệ nhân tạo (AI) chạy trực tiếp tại biên (Edge).

Hệ thống được xây dựng trên nền tảng phần cứng tích hợp giữa chip SoC Edge AI đóng vai trò xử lý trung tâm (chạy hệ điều hành Linux) và vi điều khiển STM32 (lập trình qua thư viện HAL) để điều khiển cơ cấu chấp hành trực tiếp. Về mặt phần mềm, hệ thống triển khai hai tính năng ADAS cốt lõi bao gồm:

Xử lý ảnh từ camera để nhận diện vạch kẻ đường, kết hợp phát triển giải thuật điều khiển xe tự động giữ làn đường (Lane Keeping Assist).

Nhúng mô hình AI (Computer Vision) được tối ưu hóa để phát hiện và phân loại trạng thái đèn giao thông theo thời gian thực.

Toàn bộ khung vỏ xe 3D và mạch in (PCB) hệ thống đều được thiết kế, tối ưu hóa và thử nghiệm thực tế trên sa bàn bản đồ mô phỏng tự chế tạo.

Mặc dù quá trình triển khai gặp nhiều thách thức trong việc tối ưu hóa thuật toán điều khiển giữ làn, cấu hình Driver/Kernel trên hệ điều hành Linux, và đồng bộ dữ liệu qua các giao thức truyền dẫn, hệ thống đã hoàn thành các mục tiêu cơ bản đề ra. Xe hoạt động ổn định trên sa bàn, nhận diện chính xác tín hiệu đèn và duy trì quỹ đạo chuyển động đúng làn đường. Trong tương lai, đề tài sẽ tiếp tục nâng cấp khả năng tính toán của SoC, tối ưu độ trễ của mô hình AI và mở rộng thêm các tính năng ADAS nâng cao như bám đuôi xe trước (ACC) hoặc tự động phanh khẩn cấp (AEB) nhằm tiệm cận hơn với các ứng dụng thực tế.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH MỤC TỪ VIẾT TẮT	vii
DANH MỤC BẢNG BIỂU	ix
DANH MỤC HÌNH ẢNH	x
CHƯƠNG 1: TỔNG QUAN.....	1
1.1. GIỚI THIỆU ĐỀ TÀI.....	1
1.2. TÍNH CẤP THIẾT CỦA ĐỀ TÀI.....	1
1.3. MỤC TIÊU CỦA ĐỀ TÀI.....	2
1.4. ĐỐI TƯỢNG NGHIÊN CỨU	2
1.5. PHẠM VI NGHIÊN CỨU	2
1.6. PHƯƠNG PHÁP NGHIÊN CỨU	3
1.7. BỐ CỤC ĐỒ ÁN	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	5
2.1. TỔNG QUAN LÝ THUYẾT	5
2.1.1. Khái niệm về hệ thống ADAS	5
2.1.2. Phân loại cấp độ xe tự hành.....	5
2.1.3. Các tính năng cốt lõi	6
2.2. MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG	6
2.2.1. Mạch điều khiển động cơ DC BTS7960.....	6
2.2.2. Động cơ 385 kèm Encoder 1 kênh	8
2.2.3. Mạch hạ áp SY8205	10
2.2.4. Mạch chuyển tín hiệu 8 kênh TXS0108	11
2.2.5. STM32F103C8T6.....	11
2.2.6. Cảm biến đo khoảng cách TOF VL53L0X	12
2.2.7. Động cơ Digital RC Servo SG92R.....	14
2.2.8. Máy tính nhúng Raspberry Pi 4 4G	15

2.2.9.	Camera Raspberry Pi OV5647	16
2.2.10.	Pin 3S 21700 12.6V 5000mAh	17
2.2.11.	Mạch sạc B3 Pro 800mA	17
2.2.12.	Màn hình LCD 2.4 inch ILI9341	18
2.3.	CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU	19
2.3.1.	Chuẩn giao tiếp UART	19
2.3.2.	Chuẩn giao tiếp I2C	20
2.3.3.	Chuẩn giao tiếp SPI	21
2.4.	TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH CHO HỆ THỐNG	22
2.4.1.	Phần mềm STM32CubeMX	23
2.4.2.	Phần mềm Roboflow	24
2.4.3.	Phần mềm Visual Studio Code	25
2.4.4.	Phần mềm EasyEDA	26
2.5.	TỔNG QUAN VỀ EDGE AI VÀ HỆ THỐNG SOC	27
2.5.1.	Công nghệ điện toán biên (Edge Computing)	27
2.5.2.	Hệ thống System-on-Chip (SoC).....	28
2.6.	CƠ SỞ LÝ THUYẾT VỀ XỬ LÝ ẢNH.....	29
2.6.1.	Không gian màu và các phép biến đổi.....	29
2.6.2.	Lọc nhiễu ảnh và phát hiện biên.....	30
2.6.3.	Biến đổi phối cảnh (Perspective Transformation).....	30
2.7.	TỔNG QUAN VỀ THUẬT TOÁN GIỮ LÀN.....	31
2.7.1.	Phân tích Sai số bám quỹ đạo (Tracking Errors).....	31
2.8.	TỔNG QUAN VỀ AI VÀ MÔ HÌNH YOLO.....	31
2.8.1.	Trí tuệ nhân tạo (AI) trong xe tự hành.....	31
2.8.2.	Mô hình YOLO – You Only Look You	32
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG		34
3.1.	YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG	34
3.1.1.	Yêu cầu của hệ thống.....	34
3.1.2.	Sơ đồ khối hệ thống và chức năng của từng khối.....	36
3.2.	THIẾT KẾ HỆ THỐNG	41
3.2.1.	Thiết kế khối nguồn	41
3.2.2.	Thiết kế khối xử lý trung tâm	44
3.2.3.	Thiết kế khối động cơ	47
3.2.4.	Thiết kế khối cảm biến	49

3.2.5.	Thiết kế khối camera	53
3.2.6.	Thiết kế khối hiển thị.....	56
3.2.7.	Sơ đồ nguyên lý toàn hệ thống	58
3.3.	LỰA CHỌN VÀ HUẤN LUYỆN MODULE AI DỰ ĐOÁN.....	60
3.3.1.	Lựa chọn mô hình.....	60
3.3.2.	Quá trình huấn luyện mô hình YOLO-FastestV2.....	61
3.4.	THIẾT KẾ PCB CHO XE TỰ HÀNH.....	64
3.4.1.	Thiết kế sơ đồ nguyên lý	64
3.4.2.	Thiết kế Layout và Kỹ thuật đi dây (Routing)	65
3.5.	THIẾT KẾ SA BÀN CHO XE TỰ HÀNH.....	67
3.5.1.	Quy mô và cấu trúc của sa bàn	67
3.5.2.	Các phân đoạn quỹ đạo thử nghiệm	67
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG PHẦN MỀM		69
4.1.	TỔNG QUAN KIẾN TRÚC PHẦN MỀM HỆ THỐNG	69
4.1.1.	Phân chia phần mềm trên Raspberry Pi và STM32.....	69
4.1.2.	Sơ đồ kiến trúc phần mềm tổng thể	69
4.2.	LẬP TRÌNH PHẦN MỀM TRÊN RASPBERRY PI.....	70
4.2.1.	Tổng quan chương trình điều khiển trên Raspberry Pi	70
4.2.2.	Xử lý đa luồng trên Raspberry Pi	71
4.2.3.	Luồng thu nhận hình ảnh từ camera	73
4.2.4.	Luồng xử lý chính và đưa ra quyết định.....	74
4.2.5.	Thuật toán phát hiện làn đường	76
4.2.6.	Thuật toán nhận diện đèn giao thông.....	78
4.2.7.	Thuật toán phát hiện khu vực giao lộ	80
4.2.8.	Khối quyết định hành vi điều khiển.....	82
4.2.9.	Thuật toán điều khiển tốc độ và hướng lái	84
4.2.10.	Luồng hiển thị thông tin lên LCD và GSTREAM.....	86
4.3.	LẬP TRÌNH PHẦN MỀM TRÊN STM32	88
4.3.1.	Tổng quan chương trình điều khiển trên STM32	88
4.3.2.	Đọc cảm biến khoảng cách TOF	90
4.3.3.	Điều khiển động cơ DC và Servo lái	91
4.4.	GIAO TIẾP UART GIỮA RASPBERRY PI VÀ STM32.....	93
4.4.1.	Chiều truyền từ Raspberry Pi xuống STM32	93
4.4.2.	Chiều phản hồi từ STM32 lên Raspberry Pi.....	94
4.4.3.	Cơ chế an toàn trong giao tiếp UART	95

CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG	97
5.1. KẾT QUẢ THI CÔNG PHẦN CỨNG VÀ CƠ KHÍ	97
5.1.1. Hoàn thiện cơ khí xe tự hành	97
5.1.2. Kết quả thiết kế và gia công mạch in	98
5.2. ĐÁNH GIÁ ĐỘ ỔN ĐỊNH CỦA GIAO TIẾP VÀ THỜI GIAN THỰC	99
5.2.1. Đánh giá tốc độ đáp ứng của cảm biến	99
5.2.2. Tốc độ phản hồi của cơ cấu chấp hành	102
5.3. ĐÁNH GIÁ HIỆU NĂNG AI VÀ HỆ THỐNG	103
5.3.1. Đánh giá huấn luyện mô hình và thuật toán phân loại	103
5.3.2. Tiêu thụ tài nguyên hệ thống	105
5.4. ĐÁNH GIÁ TỔNG THỂ HỆ THỐNG ADAS TRÊN SA BÀN	108
5.4.1. Kịch bản 1 - Hệ thống tự động giữ làn đường (LKA)	108
5.4.2. Kịch bản 2 - Phản ứng tại giao lộ - đèn giao thông	109
5.4.3. Kịch bản 3 - Phanh khẩn cấp tránh va chạm (AEB)	111
5.5. Kết luận chung	112
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	114
6.1. KẾT LUẬN CHUNG VỀ ĐỀ TÀI	114
6.2. NHỮNG HẠN CHẾ CÒN TỒN TẠI	115
6.3. HƯỚNG PHÁT TRIỂN	115
PHỤ LỤC	117
TÀI LIỆU THAM KHẢO	118

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa đầy đủ	Giải thích
ADAS	Advanced Driver Assistance System	Hệ thống hỗ trợ người lái nâng cao
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự thành số
AEB	Autonomous Emergency Braking	Hệ thống phanh khẩn cấp tự động / Tránh va chạm
AI	Artificial Intelligence	Trí tuệ nhân tạo
CPU	Central Processing Unit	Bộ xử lý trung tâm
DC	Direct Current	Dòng điện một chiều
EKF	Extended Kalman Filter	Bộ lọc Kalman mở rộng
FPS	Frames Per Second	Số khung hình trên giây
GPIO	General Purpose Input Output	Chân vào/ra mục đích chung
HSV	Hue, Saturation, Value	Không gian màu (Sắc độ, Độ bão hòa, Giá trị sáng)
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
I2C	Inter-Integrated Circuit	Giao tiếp hai dây cho các thiết bị ngoại vi
IoT	Internet of Things	Internet vạn vật
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
LED	Light Emitting Diode	Đi-ốt phát sáng
LKA	Lane Keeping Assist	Hệ thống tự động giữ làn đường

MPC	Model Predictive Control	Điều khiển dự báo mô hình
OpenCV	Open Source Computer Vision Library	Thư viện thị giác máy tính mã nguồn mở
OS	Operating System	Hệ điều hành
PID	Proportional Integral Derivative	Bộ điều khiển vi tích phân tỉ lệ
PWM	Pulse Width Modulation	Điều chế độ rộng xung
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
ROI	Region of Interest	Vùng quan tâm (trong xử lý ảnh)
RX	Receive	Thu tín hiệu
SLAM	Simultaneous Localization and Mapping	Định vị và lập bản đồ đồng thời
SoC	System on Chip	Hệ thống trên một vi mạch
SPI	Serial Peripheral Interface	Giao tiếp nối tiếp ngoại vi
ToF	Time-of-Flight	Cảm biến đo khoảng cách bằng thời gian bay của ánh sáng
TX	Transmit	Phát tín hiệu
UART	Universal Asynchronous Receiver-Transmitter	Bộ thu phát bất đồng bộ vạn năng
V2X	Vehicle-to-Everything	Giao tiếp kết nối xe với cơ sở hạ tầng/vạn vật
YOLO	You Only Look Once	Thuật toán phát hiện đối tượng thời gian thực

DANH MỤC BẢNG BIỂU

Bảng 2.1 Thông số của module BTS7960	7
Bảng 2.2 Thông số của động cơ DC 385	9
Bảng 2.3 Thông số mạch hạ áp SY8205	10
Bảng 2.4 Thông số của module TXS0108	11
Bảng 2.5 Thông số của STM32F103C8T6 Bluepill	12
Bảng 2.6 Thông số của cảm biến ToF VL53L0x.....	13
Bảng 2.7 Thông số Digital RC Servo SG92R.....	14
Bảng 2.8 Thông số của Raspberry pi 4	16
Bảng 2.9 Thông số của cell pin 21700.....	17
Bảng 2.10 Thông số của mạch sạc B3 Pro.....	18
Bảng 2.11 Thông số của LCD ILI9341.....	19
Bảng 2.12 So sánh YOLO với RCNN	33
Bảng 3.1 . So sánh 3 mô hình YOLO-FastestV2, YOLOv11n, Haar Cascades	61
Bảng 4.1 Kết quả bảng nhận diện màu đèn giao thông.....	79
Bảng 5.1 Kết quả đo đặc thời gian thu thập khung ảnh camera	100
Bảng 5.2 Kết quả tốc độ cập nhật các giá trị của cảm biến phụ	101
Bảng 5.3 Kết quả tốc độ phản hồi của động cơ	102
Bảng 5.4 Kết quả các thông số đánh giá mô hình.....	104
Bảng 5.5 Kết quả tổng quan về hệ thống	107
Bảng 5.6 Kết quả của Hệ thống giữ làn	109
Bảng 5.7 Kết quả sau khi kiểm thử phản ứng với đèn giao thông.....	110
Bảng 5.8 Kết quả khoảng cách của xe so với vật cản.....	111
Bảng 5.9 Tổng kết kết quả kiểm thử Phanh khẩn cấp.....	112

DANH MỤC HÌNH ẢNH

Hình 2.1 Các cấp độ của chế độ ADAS	5
Hình 2.2 Module BTS7960	6
Hình 2.3 Sơ đồ nguyên lý của module BTS7960.....	7
Hình 2.4 Động cơ DC 385 kèm Encoder	8
Hình 2.5 Mạch hạ áp SY8205	10
Hình 2.6 Module TXS0108.....	11
Hình 2.7 STM32F103C8T6 Bluepill	12
Hình 2.8 Sơ đồ nguyên lý của mạch STM32F103C8T6 Bluepill.....	12
Hình 2.9 Cảm biến ToF VL53L0x.....	13
Hình 2.10 Digital RC Servo SG92R	14
Hình 2.11 Raspberry pi 4	16
Hình 2.12 Camera Raspberry Pi OV5647	17
Hình 2.13 Mạch sạc B3 Pro	18
Hình 2.14 LCD ILI9341.....	19
Hình 2.15 Giao tiếp UART	20
Hình 2.16 Giao tiếp I2C	21
Hình 2.17 Giao tiếp SPI	22
Hình 2.18 Giao diện STM32CUBEMX.....	23
Hình 2.19 Giao diện Roboflow	25
Hình 2.20 VScode	25
Hình 2.21 Giao diện EASYEDA	26
Hình 2.22 AI trong hệ thống xe tự hành	32
Hình 2.23 Ứng dụng mô hình YOLO	32
Hình 3.1 Sơ đồ khối hệ thống	36
Hình 3.2 Sơ đồ nguyên lý của khối nguồn.....	42
Hình 3.3 Sơ đồ nguyên lý của khối xử lý trung tâm	46

Hình 3.4 Sơ đồ nguyên lý của khối động cơ	49
Hình 3.5 Sơ đồ nguyên lý của khối cảm biến	53
Hình 3.6 Sơ đồ nguyên lý của khối camera	55
Hình 3.7 Sơ đồ nguyên lý của khối hiển thị.....	58
Hình 3.8 Sơ đồ nguyên lý hệ thống.....	59
Hình 3.9 Cắt ảnh đúng kích thước 320x64	62
Hình 3.10 Dán nhãn cho ảnh.....	62
Hình 3.11 Sơ đồ nguyên lý mạch điều khiển	65
Hình 3.12 Layout mặt trước, sau của PCB MACH DIEU KHIEN	66
Hình 3.13 Mô phỏng 3D của PCB	66
Hình 3.14 Sa bàn kích thước 2.5m x 2.5m.....	68
Hình 4.1 Sơ đồ kiến trúc tổng thể	69
Hình 4.2 Sơ đồ tổng quan chương trình điều khiển trên Raspberry Pi.....	71
Hình 4.3 Lưu đồ giải thuật xử lý đa luồng.....	72
Hình 4.4 Lưu đồ giải thuật luồng thu nhận hình ảnh từ camera	73
Hình 4.5 Lưu đồ giải thuật luồng xử lý chính và đưa ra quyết định.....	74
Hình 4.6 Lưu đồ giải thuật của thuật toán phát hiện làn đường.....	76
Hình 4.7 Biến đổi phối cảnh Bird's Eye View	77
Hình 4.8 Biến đổi ảnh BEV nhị phân	77
Hình 4.9 Vùng ROI nhận diện đèn.....	78
Hình 4.10 Kết quả nhận diện đèn.....	79
Hình 4.11 Lưu đồ giải thuật cho thuật toán nhận diện đèn giao thông.....	80
Hình 4.12 Khu vực giao lộ.....	80
Hình 4.13 Vùng ROI nhận diện giao lộ	81
Hình 4.14 Kết quả khi nhận diện được vạch giao lộ.....	81
Hình 4.15 Lưu đồ giải thuật cho thuật toán phát hiện giao lộ.....	82
Hình 4.16 Lưu đồ giải thuật cho khối quyết định hành vi điều khiển	84
Hình 4.17 Lưu đồ giải thuật cho thuật toán điều khiển hướng lái	85

Hình 4.18 Lưu đồ giải thuật cho thuật toán điều khiển tốc độ.....	86
Hình 4.19 Lưu đồ giải thuật cho luồng hiển thị LCD và Gstreamer.	87
Hình 4.20 Màn hình HUD.....	87
Hình 4.21 Màn hình BEV	88
Hình 4.22 Màn hình HUD.....	88
Hình 4.23 Sơ đồ khối tổng thể STM32	89
Hình 4.24 Lưu đồ giải thuật trên STM32.....	89
Hình 4.25 Lưu đồ giải thuật của thuật toán đọc cảm biến	91
Hình 4.26 Lưu đồ giải thuật thuật toán điều khiển động cơ DC và Servo.....	92
Hình 4.27 Sơ đồ luồng nhận dữ liệu từ STM32.....	94
Hình 4.28 Sơ đồ luồng nhận dữ liệu từ Pi.....	95
Hình 4.29 Sơ đồ luồng xử lý an toàn trong giao tiếp UART	96
Hình 5.1 Kết cấu của mô hình xe tự hành.....	97
Hình 5.2 Hệ thống dẫn động cầu sau của xe tự hành.....	97
Hình 5.3 Hệ thống đánh lái của xe tự hành.....	98
Hình 5.4 Vị trí đặt camera và cảm biến ToF.....	98
Hình 5.5 Hình ảnh mạch PCB thành phẩm.....	99
Hình 5.6 Log khi đo đạt tốc độ xử lý khung hình.....	99
Hình 5.7 Log kết quả đo thời gian giao tiếp UART với các cảm biến phụ	101
Hình 5.8 Biểu đồ thể hiện đặc tuyến tốc độ theo tín hiệu PWM	102
Hình 5.9 Log khi đang trong quá trình huấn luyện mô hình.....	104
Hình 5.10 Biểu đồ thể hiện biến thiên của %CPU, mức sử dụng RAM và nhiệt độ khi chạy hệ thống	107
Hình 5.11 Các đo đặc độ lệch tâm	109
Hình 5.12 Thiết lập môi trường kiểm thử Hệ thống giữ làn	109
Hình 5.13 Thiết lập môi trường kiểm thử Hệ thống giữ làn	110
Hình 5.14 Hệ thống nhận diện và phản ứng với đèn giao thông	111
Hình 5.15 Thiết lập môi trường kiểm thử Phanh khẩn cấp.....	112

CHƯƠNG 1: TỔNG QUAN

1.1. GIỚI THIỆU ĐỀ TÀI

Trong những năm gần đây, sự phát triển mạnh mẽ của trí tuệ nhân tạo (Artificial Intelligence – AI) và thị giác máy tính (Computer Vision) đã thúc đẩy sự ra đời của các hệ thống hỗ trợ lái xe tiên tiến (Advanced Driver Assistance Systems – ADAS) và xe tự hành. Các hệ thống này giúp nâng cao mức độ an toàn, giảm thiểu tai nạn giao thông và hỗ trợ người lái trong các tình huống phức tạp.

Đề tài “Nghiên cứu và triển khai hệ thống ADAS hỗ trợ giữ làn, tránh vật cản và nhận diện biển báo giao thông trên mô hình xe tự hành sử dụng SoC Edge AI” tập trung xây dựng một mô hình xe tự hành thu nhỏ. Hệ thống ứng dụng nền tảng xử lý biên (Edge AI) để thực hiện các chức năng như phát hiện làn đường, giữ làn, nhận diện đèn giao thông và tránh vật cản trong thời gian thực.

Thông qua đề tài, các kỹ thuật về xử lý ảnh, học máy, hệ thống nhúng và điều khiển thời gian thực được tích hợp nhằm tạo ra một hệ thống hoàn chỉnh, có khả năng mô phỏng các chức năng cơ bản của xe tự hành trong thực tế.

1.2. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Tai nạn giao thông hiện nay vẫn là một trong những vấn đề nghiêm trọng trên toàn cầu, trong đó phần lớn nguyên nhân đến từ yếu tố con người như mất tập trung, phản xạ chậm hoặc xử lý tình huống không kịp thời. Do đó, việc nghiên cứu và phát triển các hệ thống hỗ trợ lái xe thông minh là một hướng đi cần thiết nhằm giảm thiểu rủi ro và nâng cao an toàn giao thông.

Bên cạnh đó, xu hướng phát triển xe tự hành đang được các hãng công nghệ và ô tô lớn trên thế giới đầu tư mạnh mẽ. Các công nghệ như nhận diện làn đường, phát hiện vật cản và nhận diện tín hiệu giao thông là những thành phần cốt lõi của hệ thống ADAS.

Trong bối cảnh đó, việc xây dựng một mô hình thử nghiệm quy mô nhỏ sử dụng Edge AI không chỉ giúp tiếp cận công nghệ mới mà còn tạo nền tảng cho việc nghiên cứu sâu hơn trong tương lai. Đề tài mang tính thực tiễn cao, phù hợp với xu hướng phát triển công nghệ và nhu cầu đào tạo trong lĩnh vực kỹ thuật điều khiển – tự động hóa và trí tuệ nhân tạo.

1.3. MỤC TIÊU CỦA ĐỀ TÀI

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và triển khai một hệ thống xe tự hành thu nhỏ có tích hợp các chức năng cơ bản của ADAS. Cụ thể:

Xây dựng hệ thống nhận diện làn đường và điều khiển xe giữ làn ổn định

Phát triển mô hình nhận diện đèn giao thông và đưa ra phản ứng phù hợp

Thiết kế hệ thống phát hiện và tránh vật cản trong quá trình di chuyển

Triển khai kiến trúc Edge AI sử dụng Raspberry Pi để xử lý hình ảnh thời gian thực

Thiết kế hệ thống điều khiển thời gian thực sử dụng vi điều khiển

Tích hợp và đồng bộ toàn bộ hệ thống phần cứng và phần mềm

Đánh giá hiệu năng hệ thống dựa trên các tiêu chí cụ thể

1.4. ĐỐI TƯỢNG NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài bao gồm:

Các thuật toán thị giác máy tính phục vụ phát hiện làn đường, nhận diện biển báo và phát hiện vật cản

Mô hình học máy ứng dụng trong nhận dạng hình ảnh

Hệ thống nhúng sử dụng Raspberry Pi và vi điều khiển

Giao tiếp giữa các thiết bị (UART, GPIO)

Các phương pháp điều khiển chuyển động của xe (điều khiển tốc độ và góc lái)

Hệ thống cấp nguồn và tích hợp phần cứng cho mô hình xe tự hành

1.5. PHẠM VI NGHIÊN CỨU

Đề tài được thực hiện trong phạm vi sau:

Xây dựng mô hình xe tự hành quy mô nhỏ (prototype)

Sử dụng camera đơn để thu thập dữ liệu hình ảnh

Thực hiện các chức năng cơ bản của ADAS gồm: Giữ làn đường, nhận diện đèn giao thông, phát hiện và tránh vật cản

Xử lý dữ liệu tại thiết bị biên (Edge AI), không sử dụng cloud

Môi trường thử nghiệm trong điều kiện có kiểm soát (indoor hoặc sa hình)

Đề tài chưa tập trung vào:

Các thuật toán tự hành phức tạp như SLAM, lập bản đồ

Hệ thống định vị GPS

Triển khai trên xe thực tế kích thước lớn

1.6. PHƯƠNG PHÁP NGHIÊN CỨU

Để thực hiện đề tài, các phương pháp nghiên cứu sau được áp dụng:

Nghiên cứu lý thuyết: Tìm hiểu các tài liệu, bài báo khoa học liên quan đến ADAS, xử lý ảnh và hệ thống nhúng

Thiết kế hệ thống: Xây dựng kiến trúc tổng thể bao gồm phần cứng và phần mềm

Lập trình và mô phỏng: Phát triển thuật toán xử lý ảnh và điều khiển bằng Python và C/C++

Thực nghiệm: Triển khai hệ thống trên mô hình thực tế, thu thập dữ liệu và đánh giá hiệu năng

Phân tích và đánh giá: So sánh kết quả đạt được với các tiêu chí đề ra, từ đó rút ra nhận xét và cải tiến

1.7. BỐ CỤC ĐỒ ÁN

Chương 1: Tổng Quan: Trong chương 1, sinh viên sẽ trình bày những nội dung về tính cấp thiết của đề tài dựa trên xu hướng và tình hình phát triển hiện nay của xã hội. Sinh viên cũng sẽ đề cập tới những sự phát triển của công nghiệp và những tác động của nó tới đời sống hàng ngày, từ đó đưa ra lý do lựa chọn đề tài, mục đích, phương pháp thực hiện và phạm vi của đề tài.

Chương 2: Cơ sở lý thuyết: Chương này sẽ cung cấp góc nhìn tổng quan về các thành phần và chức năng của từng loại phần cứng có trong hệ thống. Sinh viên sẽ mô tả chi tiết từng thành phần và cách chúng liên kết với nhau để xây dựng mô hình hoàn chỉnh.

Chương 3: Thiết Kế và Xây Dựng Hệ Thống: Từ các yêu cầu của đề tài, chương 3 sẽ trình bày sơ đồ tổng quan của hệ thống. Sinh viên sẽ tìm hiểu về các phương pháp xử lý dữ liệu được sử dụng và từ đó thiết kế mô hình hệ thống bao gồm cả phần cứng và phần mềm.

Chương 4: Lập Trình Hệ Thống: Chương này mô tả quá trình lên ý tưởng, lập trình, biên dịch cho phần cứng lẫn phần mềm.

Chương 5: Kết Quả và Đánh Giá Hệ Thống: Chương này trình bày các kết quả đạt được từ quá trình thi công và thử nghiệm hệ thống, đánh giá hoạt động và các thông số đạt được của hệ thống

Chương 6: Kết Luận và Hướng Phát Triển: Trong chương cuối cùng, sinh viên sẽ tổng kết lại những gì đã đạt được, ưu điểm và hạn chế trong quá trình thực hiện đề tài. Dựa trên các kết quả và nhận xét, sinh viên sẽ đề xuất các hướng phát triển để cải thiện và mở rộng hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN LÝ THUYẾT

2.1.1. Khái niệm về hệ thống ADAS

Hệ thống hỗ trợ người lái nâng cao (Advanced Driver Assistance Systems - ADAS) là một tập hợp các hệ thống và tính năng an toàn chủ động được tích hợp trên xe cơ giới. Mục tiêu cốt lõi của ADAS là giảm thiểu lỗi do con người trong quá trình điều khiển phương tiện, thông qua việc tự động hóa, tăng cường khả năng quan sát và đưa ra các cảnh báo kịp thời. Hệ thống ADAS thu thập dữ liệu từ môi trường xung quanh xe thông qua các cảm biến như Camera, Radar, LiDAR và siêu âm, sau đó xử lý thông tin bằng các thuật toán phức tạp để đưa ra quyết định điều khiển. Các khối kiến thức cơ bản này được tham chiếu trực tiếp từ tài liệu giảng dạy chuyên ngành của Khoa [1].

2.1.2. Phân loại cấp độ xe tự hành

Theo tiêu chuẩn J3016 của Hiệp hội Kỹ sư Ô tô Quốc tế (SAE International), mức độ tự động hóa của phương tiện được chia thành 6 cấp độ, từ Cấp độ 0 (Không tự động hóa) đến Cấp độ 5 (Tự động hóa hoàn toàn). [2]



Hình 2.1 Các cấp độ của chế độ ADAS

Trong phạm vi của nghiên cứu này, hệ thống nhằm tới việc đáp ứng các tính năng của xe tự hành ở Cấp độ 2 (Tự động hóa một phần). Ở cấp độ này, hệ thống có khả năng tự động can thiệp đồng thời vào cả hệ thống lái (Steering) và *hệ thống gia tốc/phanh* ($\frac{Acceleration}{Deceleration}$) thông qua các tính năng như bám quỹ đạo và nhận diện biển báo, tuy nhiên người lái vẫn phải liên tục giám sát môi trường và sẵn sàng tiếp quản hệ thống.

2.1.3. Các tính năng cốt lõi

Hệ thống giữ làn đường (Lane Keeping Assist - LKA): LKA sử dụng camera phía trước để liên tục theo dõi vạch kẻ đường. Khi phát hiện xe có xu hướng đi lệch khỏi tâm làn đường mà người lái không có tín hiệu chuyển làn (bật đèn xi-nhan), hệ thống sẽ tự động tính toán góc đánh lái và can thiệp vào cơ cấu chấp hành (động cơ lái) để điều chỉnh xe quay trở lại đúng quỹ đạo.

Nhận diện biển báo giao thông (Traffic Sign Recognition - TSR): Tính năng này ứng dụng công nghệ thị giác máy tính và học sâu (Deep Learning) để trích xuất đặc trưng của biển báo từ luồng video trực tiếp. Hệ thống có nhiệm vụ phân loại các biển báo (giới hạn tốc độ, cấm rẽ, dừng lại,...) để đưa ra cảnh báo cho người lái hoặc tự động hạ tốc độ của động cơ.

2.2. MỘT SỐ LINH KIỆN VÀ MODULE TRONG HỆ THỐNG

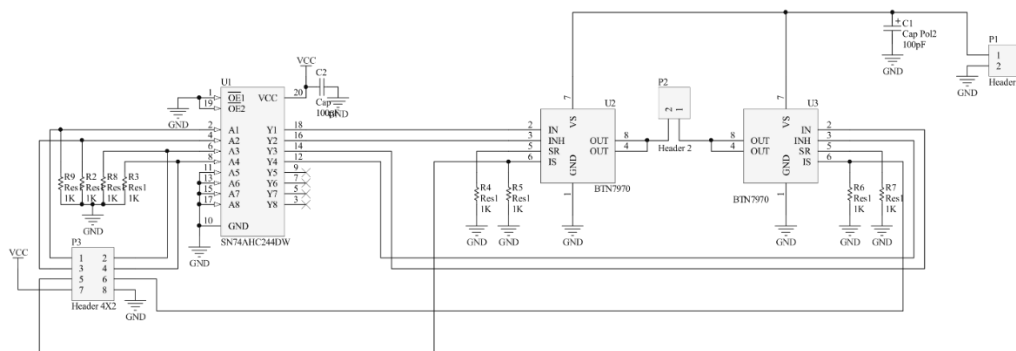
2.2.1. Mạch điều khiển động cơ DC BTS7960



Hình 2.2 Module BTS7960

Mạch điều khiển động cơ DC BTS7960 là module cầu H dòng cao chuyên dụng, sử dụng hai vi mạch BTS7960 tích hợp IC điều khiển và transistor công suất lớn. Module này có khả năng chống nhiễu tốt, bảo vệ quá dòng, quá nhiệt và có thể điều khiển đảo chiều, điều tốc động cơ DC thông qua tín hiệu PWM một cách mượt mà.

Trong dự án này, mạch BTS7960 được sử dụng để điều khiển động cơ DC dòng lớn kích kéo hệ thống bánh xe chủ động di chuyển hoặc cơ cấu chấp hành công suất cao.



Hình 2.3 Sơ đồ nguyên lý của module BTS7960

Bảng 2.1 Thông số của module BTS7960

STT	Thông số	Mô tả
1	Điện áp đầu vào	6 ~ 27V
2	Dòng điện tải mạch	43A
3	Tín hiệu logic điều khiển	3.3 ~ 5V
4	Tần số điều khiển tối đa	~ 25KHz
5	Nhiệt độ hoạt động	0 ~ 50°C

Code và thư viện sử dụng (STM32 HAL - Cấu hình TIM PWM):

```
/* Cấu hình trong STM32CubeIDE:
- TIM1_CH1 (PA8) làm chân RPWM (Quay thuận)
```

- TIM1_CH2 (PA9) làm chân LPWM (Quay nghịch)
- Chân GPIO Output (PA6, PA7) làm chân R_EN, L_EN */

// Bật Driver cầu H và khởi động các kênh PWM

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // R_EN = 1
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET); // L_EN = 1
```

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
```

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
```

```
void Control_Motor_Forward(uint16_t speed) {
```

```
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, speed); // Thiết lập
```

Duty Cycle quay thuận

```
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 0); // Khóa quay
```

nghịch

```
}
```

2.2.2. Động cơ 385 kèm Encoder 1 kênh

Động cơ DC dòng 385 là loại động cơ một chiều nam châm vĩnh cửu có tốc độ cao, moment xoắn vừa phải, hoạt động ổn định ở dải điện áp rộng. Động cơ được tích hợp thêm đĩa mã hóa xung quang học hoặc từ tính (Encoder 1 kênh) gắn ở trục sau, phát ra các xung vuông khi trục động cơ quay.



Hình 2.4 Động cơ DC 385 kèm Encoder

Ứng dụng: Sử dụng làm động cơ truyền động chính hoặc cơ cấu dịch chuyển, cung cấp phản hồi xung vận tốc (xung/vòng) giúp vi điều khiển tính toán tốc độ thực tế của robot.

Bảng 2.2 Thông số của động cơ DC 385

STT	Thông số	Mô tả
1	Điện áp hoạt động	6 ~ 12VDC
2	Tốc độ không tải (tại 12V)	~ 10000 - 15000 RPM (Tùy hộp số)
3	Dòng không tải	~ 0.2A
4	Loại Encoder	1 Kênh (Ngõ ra Hall hoặc Quang)
5	Điện áp cấp cho Encoder	3.3V hoặc 5VDC

Code và thư viện sử dụng:

```

/* Cấu hình trong STM32CubeIDE:
- Chọn 1 chân GPIO làm ngắt ngoài (Ví dụ: PB0), cấu hình chế độ External Interrupt
Rising Edge */

volatile uint32_t pulse_count = 0;

// Hàm Callback xử lý ngắt ngoài khi có xung cạnh lên từ Encoder
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) {
        pulse_count++; // Tăng bộ đếm xung
    }
}

// Đọc tốc độ định kỳ trong luồng lặp chính
uint32_t last_tick = 0;
uint32_t rpm = 0;

```

```

void Check_Speed_Loop(void) {
    if (HAL_GetTick() - last_tick >= 1000) { // Mỗi 1 giây
        rpm = pulse_count; // Xử lý toán học tính toán RPM tùy theo tỷ số truyền
        pulse_count = 0; // Reset bộ đếm xung
        last_tick = HAL_GetTick();
    }
}

```

2.2.3. Mạch hạ áp SY8205

Mạch hạ áp (Buck Converter) SY8205 sử dụng IC hạ áp đồng bộ hiệu suất cao, có khả năng chuyển đổi dải điện áp DC cao từ nguồn pin hoặc adapter thành điện áp DC thấp hơn một cách ổn định, ít tỏa nhiệt, đáp ứng dòng tải lớn liên tục mà không gây sụt áp hệ thống.

Ứng dụng: Hạ áp từ nguồn Pin chính (11.1V - 12.6V) xuống mức cố định 5VDC để cấp nguồn nuôi công suất ổn định cho máy tính nhúng Raspberry Pi 4, STM32 và các cảm biến ngoại vi.



Hình 2.5 Mạch hạ áp SY8205

Bảng 2.3 Thông số mạch hạ áp SY8205

STT	Thông số	Mô tả
1	Điện áp đầu vào	7 ~ 30VDC
2	Điện áp đầu ra	Cố định 5VDC
3	Dòng điện đầu ra tối đa	5A (Đỉnh 6A)
4	Hiệu suất chuyển đổi	Lên đến 96%

5	Tần số đáp ứng	500 KHz
---	----------------	---------

2.2.4. Mạch chuyển tín hiệu 8 kênh TXS0108

Mạch TXS0108 là IC dịch cấp điện áp logic (Logic Level Shifter) 8 kênh phản hồi hai chiều tốc độ cao [3]. Module cho phép chuyển đổi qua lại giữa hai mức điện áp logic khác biệt mà không cần cấu hình hướng truyền dữ liệu, bảo vệ an toàn cho các IO của vi điều khiển.

Ứng dụng: Kết nối giao tiếp an toàn (I2C, UART, SPI) giữa các thiết bị logic 3.3V (Raspberry Pi 4, STM32) và các thiết bị sử dụng logic 5V (Mạch công suất, Cảm biến cũ).



Hình 2.6 Module TXS0108

Bảng 2.4 Thông số của module TXS0108

STT	Thông số	Mô tả
1	Số kênh chuyển đổi	8 kênh độc lập (Hai chiều)
2	Điện áp bên A (VCCA)	1.2V ~ 3.6V (Kết nối thiết bị điện áp thấp)
3	Điện áp bên B (VCCB)	1.65V ~ 5.5V (Kết nối thiết bị điện áp cao)
4	Tốc độ truyền tối đa	60 Mbps (Đẩy kéo), 2 Mbps (Hở cực máng)

2.2.5. STM32F103C8T6

STM32F103C8T6 (thường gọi là Blue Pill) là vi điều khiển mạnh mẽ dựa trên lõi ARM Cortex-M3 32-bit hiệu năng cao [4]. Nó sở hữu đầy đủ các bộ ngoại vi mạnh mẽ: Bộ định thời nâng cao (Hardware PWM, Encoder Interface), ADC tốc độ cao, và các chuẩn truyền thông công nghiệp đa dạng.

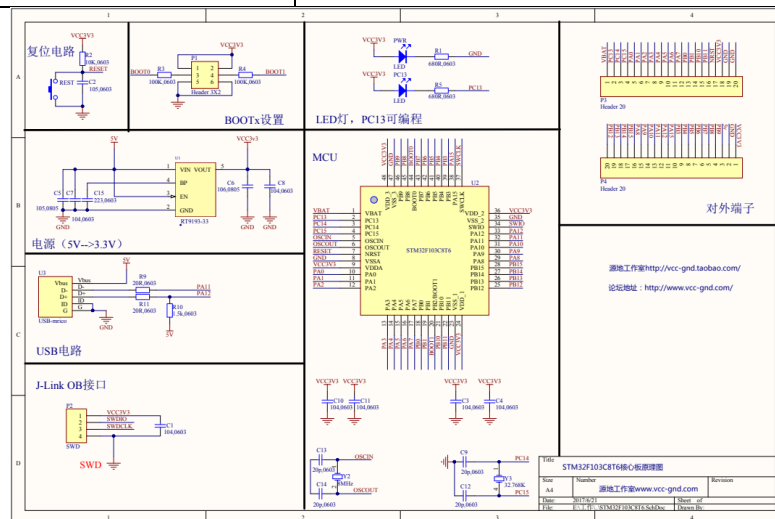
Đóng vai trò là mạch điều khiển chính ở tầng phần cứng (Low-level controller), chịu trách nhiệm đọc xung encoder tốc độ cao, điều khiển động cơ qua BTS7960, Servo SG92R và giao tiếp UART với Raspberry Pi.



Hình 2.7 STM32F103C8T6 Bluepill

Bảng 2.5 Thông số của STM32F103C8T6 Bluepill

STT	Thông số	Mô tả
1	Lõi xử lý	ARM Cortex-M3 32-bit
2	Tốc độ CPU	Tối đa 72 MHz
3	Bộ nhớ Flash	64 KB
4	Bộ nhớ SRAM	20 KB
5	Điện áp hoạt động	2.0V ~ 3.6V (I/O chịu được mức 5V)



Hình 2.8 Sơ đồ nguyên lý của mạch STM32F103C8T6 Bluepill

2.2.6. Cảm biến đo khoảng cách TOF VL53L0X

VL53L0X là cảm biến đo khoảng cách thể hệ mới dựa trên công nghệ Flight-of-Time (TOF) tiên tiến từ hãng STMicroelectronics [5]. Cảm biến sử dụng một nguồn phát laser hồng ngoại vô hại phát ra các xung photon và đo thời gian photon phản hồi ngược lại, từ đó tính chính xác khoảng cách bất kể độ phản xạ bề mặt vật thể.

Ứng dụng: Gắn phía trước hoặc các góc của robot trong Khối Cảm Biến giúp phát hiện vật cản một cách chính xác trong phạm vi gần, phục vụ tính năng phanh khẩn cấp hoặc giữ khoảng cách an toàn (ADAS).



Hình 2.9 Cảm biến ToF VL53L0x

Bảng 2.6 Thông số của cảm biến ToF VL53L0x

STT	Thông số	Mô tả
1	Công nghệ đo	Time-of-Flight (TOF)
2	Khoảng cách đo	Tối đa 2 mét (Hiệu quả cao trong phạm vi 1.2 mét)
3	Giao tiếp	I2C (Mặc định địa chỉ 0x29)
4	Điện áp hoạt động	2.8V ~ 5VDC (Tích hợp IC ổn áp trên module)
5	Thời gian phản hồi	< 30ms

Code và thư viện sử dụng (STM32 HAL - Giao tiếp I2C ở mức Driver cơ bản):

```
/* Thư viện Driver thanh ghi VL53L0X được triển khai thông qua I2C của STM32
HAL */
#define VL53L0X_ADDR 0x52 // Địa chỉ I2C dạng 8-bit (0x29 << 1)
uint16_t VL53L0X_Read_Distance(void) {
    uint8_t reg = 0x14; // Thanh ghi kết quả đo khoảng cách
    uint8_t data[2];
    uint16_t distance = 0;
```

```

// Gửi yêu cầu đọc thanh ghi cảm biến qua HAL I2C
if (HAL_I2C_Mem_Read(&hi2c1, VL53L0X_ADDR, reg,
I2C_MEMADD_SIZE_8BIT, data, 2, HAL_MAX_DELAY) == HAL_OK) {
    distance = (data[0] << 8) | data[1]; // Gộp 2 byte cao thấp
}
return distance; // Đơn vị: mm
}

```

2.2.7. Động cơ Digital RC Servo SG92R

Servo SG92R là phiên bản nâng cấp của dòng servo phổ biến SG90, được trang bị hệ thống bánh răng bằng sợi carbon cường độ cao gia tăng độ bền chắc và mạch kỹ thuật số (Digital) cho phản ứng điều khiển nhanh, độ chính xác góc quay cao, giảm thiểu sai số trôi góc.

Điều khiển cơ cấu góc lái (Steering) cho trục bánh xe phía trước của mô hình xe tự hành ADAS theo thuật toán bám làn hoặc tránh vật cản.



Hình 2.10 Digital RC Servo SG92R

Bảng 2.7 Thông số Digital RC Servo SG92R

STT	Thông số	Mô tả
1	Góc quay tối đa	180°
2	Lực kéo (Torque)	2.5 kg-cm (tại điện áp 4.8V)
3	Tốc độ đáp ứng	0.1 giây / 60 độ (tại 4.8V)

4	Chất liệu bánh răng	Sợi Carbon (Carbon Fiber)
5	Điện áp hoạt động	4.8V ~ 6VDC

Code và thư viện sử dụng (STM32 HAL - Cấu hình TIM PWM tần số 50Hz, Chu kỳ 20ms):

```

/* Cấu hình TIM PWM: Prescaler = 72-1, Counter Period = 20000-1 (Cho clock hệ
thống 72MHz)

Chu kỳ ngõ ra sẽ đạt chuẩn 20ms (50Hz) của Servo.
- 0 độ tương ứng góc xung 1ms (Compare = 1000)
- 90 độ tương ứng góc xung 1.5ms (Compare = 1500)
- 180 độ tương ứng góc xung 2ms (Compare = 2000) */

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1); // Khởi động kênh PWM
Servo

void Set_Steering_Angle(uint8_t angle) {
    if(angle > 180) angle = 180;
    // Ánh xạ góc từ 0-180 độ sang giá trị Compare từ 1000-2000
    uint16_t compare_val = 1000 + ((uint32_t)angle * 1000 / 180);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, compare_val);
}

```

2.2.8. Máy tính nhúng Raspberry Pi 4 4G

Raspberry Pi 4 Model B là một máy tính nhúng kích thước nhỏ nhưng sở hữu hiệu năng mạnh mẽ nhờ vi xử lý ARM lõi tứ 64-bit, dung lượng RAM lớn và khả năng xuất hình ảnh chất lượng cao [6]. Nó chạy trên các hệ điều hành Linux (Raspberry Pi OS, Ubuntu), hỗ trợ đầy đủ các thư viện xử lý ảnh và trí tuệ nhân tạo nâng cao.

Đóng vai trò là bộ não xử lý trung tâm (High-level Brain) của hệ thống ADAS. Raspberry Pi 4 thu nhận luồng video từ camera, chạy trực tiếp các mô hình

AI/Computer Vision để phát hiện làn đường, nhận diện biển báo, chướng ngại vật rồi gửi lệnh điều hướng xuống vi điều khiển STM32 thông qua giao thức UART.



Hình 2.11 Raspberry pi 4

Bảng 2.8 Thông số của Raspberry pi 4

STT	Thông số	Mô tả
1	Bộ vi xử lý	Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
2	Bộ nhớ RAM	4GB LPDDR4-3200 SDRAM
3	Kết nối mạng	Dual-band Wi-Fi, Bluetooth 5.0, Gigabit Ethernet
4	Cổng kết nối ngoại vi	2x USB 3.0, 2x USB 2.0, 40-pin GPIO
5	Kết nối Camera	Cổng MIPI CSI (2-lane) tích hợp sẵn trên bo mạch
6	Nguồn cấp yêu cầu	5VDC qua cổng USB-C (Tối thiểu 3A)

2.2.9. Camera Raspberry Pi OV5647

Camera OV5647 là module camera chuyên dụng thiết kế riêng cho bo mạch Raspberry Pi, kết nối trực tiếp thông qua giao tiếp cáp phẳng CSI dải băng thông cao, giúp giảm thiểu độ trễ hình ảnh và không chiếm dụng tài nguyên xử lý của các cổng USB.

Thu thập dữ liệu hình ảnh video thời gian thực từ phía trước lộ trình của robot để cung cấp đầu vào dữ liệu cho thuật toán Computer Vision chạy trên Raspberry Pi 4.



Hình 2.12 Camera Raspberry Pi OV5647

2.2.10.Pin 3S 21700 12.6V 5000mAh

Khối pin lithium-ion dung lượng cao cấu hình 3S (3 cell nối tiếp) sử dụng cell pin chuẩn kích thước 21700 hiện đại cho mật độ năng lượng vượt trội, dòng xả lớn và tuổi thọ pin cao, đáp ứng tốt cho hệ thống tải nặng và thời gian hoạt động kéo dài.

Đóng vai trò làm nguồn năng lượng tổng lưu trữ cấp cho toàn bộ hệ thống lưu động. Điện áp 12.6V được cấp thẳng cho mạch cầu H điều khiển động cơ DC và đồng thời cấp qua mạch hạ áp để nuôi vi điều khiển, máy tính nhúng.

Bảng 2.9 Thông số của cell pin 21700

STT	Thông số	Mô tả
1	Cấu hình cell	3S1P (3 cell nối tiếp ghép lại)
2	Điện áp danh định	11.1V
3	Điện áp nạp đầy	12.6V
4	Dung lượng khối pin	5000 mAh (5Ah)
5	Dòng xả liên tục	Tối đa 15A - 20A (Đảm bảo dòng khởi động động cơ)

2.2.11.Mạch sạc B3 Pro 800mA

Mạch sạc cân bằng B3 Pro là bộ sạc tự động thông minh nhỏ gọn chuyên dùng cho các khối pin Lithium LiPo/Li-ion cấu hình từ 2S đến 3S. Bộ sạc nạp năng lượng trực tiếp vào từng cell pin độc lập thông qua cổng sạc cân bằng (Balance Port) giúp bảo vệ pin không bị quá áp lệch cell.



Hình 2.13 Mạch sạc B3 Pro

Sử dụng trong khối quản lý nguồn ngoài, dùng để sạc an toàn và kéo dài tuổi thọ cho khối Pin 3S 21700 của hệ thống robot sau mỗi chu kỳ hoạt động.

Bảng 2.10 Thông số của mạch sạc B3 Pro

STT	Thông số	Mô tả
1	Điện áp ngõ vào	110V ~ 240VAC (Lưới điện xoay chiều trực tiếp)
2	Loại pin hỗ trợ	Pin Li-Po / Li-ion cấu hình 2S (7.4V) hoặc 3S (11.1V)
3	Dòng sạc định mức	Tối đa 800mA trên mỗi cell
4	Chế độ hiển thị	Đèn LED trạng thái độc lập (Đỏ: Đang sạc / Xanh: Pin đầy)

2.2.12. Màn hình LCD 2.4 inch ILI9341

Màn hình LCD màu TFT sử dụng IC điều khiển (Driver) ILI9341 là loại màn hình hiển thị đồ họa kỹ thuật số sắc nét, hỗ trợ dải màu rộng lên đến 65.000 màu (16-bit màu RGB565) với góc nhìn tốt. Khác với các dòng màn hình ký tự đơn giản, ILI9341 cho phép hiển thị linh hoạt các phần tử đồ họa như giao diện người dùng (GUI), biểu đồ dữ liệu, hình ảnh và văn bản với nhiều font chữ tùy biến thông qua chuẩn giao tiếp nối tiếp SPI băng thông cao.

Trong dự án này, màn hình LCD ILI9341 đóng vai trò là giao diện hiển thị chính trực quan (HMI) của hệ thống ADAS, đảm nhận nhiệm vụ hiển thị trạng thái hoạt động của xe, giá trị khoảng cách từ cảm biến TOF, tốc độ thực tế từ Encoder và các cảnh báo khẩn cấp từ bộ xử lý trung tâm.



Hình 2.14 LCD ILI9341

Bảng 2.11 Thông số của LCD ILI9341

STT	Thông số	Mô tả
1	Kích thước đường chéo	2.2 inch, 2.4 inch hoặc 2.8 inch (Tùy phiên bản)
2	IC Driver điều khiển	ILI9341
3	Độ phân giải	240 x 320 Pixels
4	Màu sắc hiển thị	RGB 65K màu (Định dạng màu 16-bit)
5	Điện áp hoạt động	Điện áp nuôi mạch: 3.3V / 5VDC; Điện áp tín hiệu: 3.3VDC
6	Giao tiếp	SPI (4-Wire Serial Peripheral Interface)
7	Đèn nền (Backlight)	Đèn LED trắng, hỗ trợ điều khiển bật/tắt hoặc điều độ sáng qua PWM

2.3. CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU

2.3.1. Chuẩn giao tiếp UART

UART (Universal Asynchronous Receiver – Transmitter) là một chuẩn giao tiếp truyền dữ liệu nối tiếp không đồng bộ được tích hợp phổ biến trên các bộ vi điều khiển, vi xử lý và nhiều thiết bị điện tử nhúng hiện nay.

Giao tiếp UART sử dụng hai đường tín hiệu chính gồm TX (Transmit) và RX (Receive), trong đó chân TX có chức năng truyền dữ liệu và chân RX dùng để nhận dữ liệu. Khi hai thiết bị giao tiếp với nhau, chân TX của thiết bị này sẽ được kết nối với chân RX của thiết bị còn lại và ngược lại để tạo thành đường truyền hai chiều.

Dữ liệu trong UART được truyền dưới dạng khung dữ liệu (Data Frame) với cấu trúc tiêu chuẩn bao gồm:

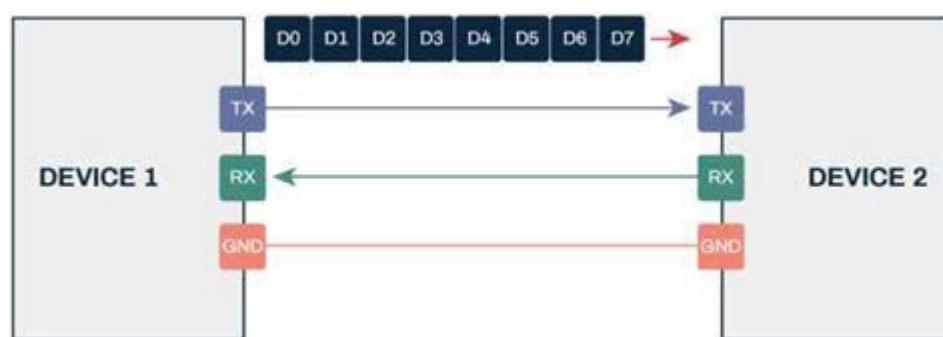
Bit bắt đầu (Start Bit): dùng để báo hiệu bắt đầu quá trình truyền dữ liệu, thường ở mức logic thấp (Low).

Bit dữ liệu (Data Bits): chứa nội dung thông tin cần truyền, phổ biến từ 5 đến 9 bit, trong đó cấu hình 8 bit được sử dụng rộng rãi nhất.

Bit kiểm tra lỗi (Parity Bit): là thành phần tùy chọn dùng để phát hiện lỗi trong quá trình truyền dữ liệu.

Bit kết thúc (Stop Bit): dùng để đánh dấu kết thúc khung dữ liệu và đưa đường truyền trở về trạng thái nghỉ.

Hiện nay, chuẩn UART được ứng dụng rộng rãi trong việc giao tiếp giữa vi điều khiển với máy tính thông qua bộ chuyển đổi USB-to-UART, kết nối với các module ngoại vi như GPS, GSM, Bluetooth, LoRa, Zigbee cũng như phục vụ mục đích debug và giám sát hệ thống trong quá trình phát triển phần mềm nhúng.



Hình 2.15 Giao tiếp UART

2.3.2. Chuẩn giao tiếp I2C

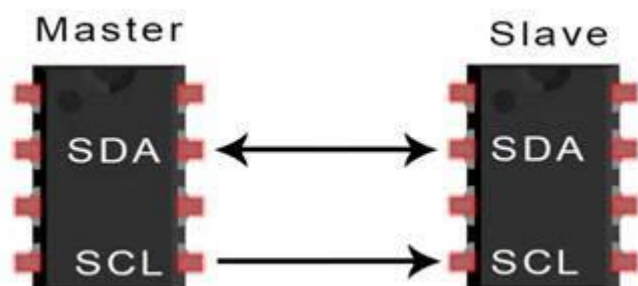
I2C (Inter-Integrated Circuit) là một chuẩn giao tiếp nối tiếp đồng bộ được phát triển bởi hãng Philips Semiconductors (nay là NXP Semiconductors) vào đầu những năm 1980 nhằm phục vụ việc trao đổi dữ liệu giữa các vi mạch tích hợp trên cùng một bo mạch điện tử.

Chuẩn giao tiếp I2C hoạt động dựa trên mô hình Controller – Target (trước đây thường được gọi là Master – Slave). Trong đó, thiết bị Controller giữ vai trò điều phối toàn bộ quá trình truyền nhận dữ liệu, bao gồm việc phát tín hiệu clock, gửi địa chỉ thiết bị và kiểm soát phiên giao tiếp. Các thiết bị Target sẽ phản hồi tương ứng khi nhận đúng địa chỉ được yêu cầu.

I2C chỉ sử dụng hai đường tín hiệu chính:

SDA (Serial Data Line): đường truyền dữ liệu nối tiếp giữa các thiết bị.

SCL (Serial Clock Line): đường truyền tín hiệu xung nhịp dùng để đồng bộ quá trình truyền dữ liệu.



Hình 2.16 Giao tiếp I2C

Giao tiếp I2C có ưu điểm là chỉ cần hai dây (SDA và SCL), giúp tiết kiệm chân kết nối và hỗ trợ nhiều thiết bị trên cùng một bus nhờ hệ thống địa chỉ riêng biệt cho từng thiết bị. Ngoài ra, Giao tiếp I2C có tốc độ truyền ổn định, khả năng đồng bộ dữ liệu tốt và dễ triển khai nhờ thư viện hỗ trợ sẵn trên các nền tảng phổ biến như Arduino, ESP32, STM32 và Raspberry Pi.

2.3.3. Chuẩn giao tiếp SPI

Giao thức SPI (Serial Peripheral Interface) là một chuẩn truyền thông nối tiếp đồng bộ, tốc độ cao, được phát triển bởi hãng Motorola vào những năm 1980. Khác với I2C, SPI hoạt động theo cơ chế toàn song công (Full-duplex), cho phép thiết bị có thể truyền và nhận dữ liệu cùng một thời điểm. Giao thức này được ứng dụng rộng rãi để kết nối giữa bộ vi điều khiển với các thiết bị ngoại vi yêu cầu tốc độ truyền tải dữ liệu

nhANH như thẻ nhớ SD, màn hình LCD/OLED, chip Flash hoặc các bộ chuyển đổi ADC/DAC hiệu năng cao.

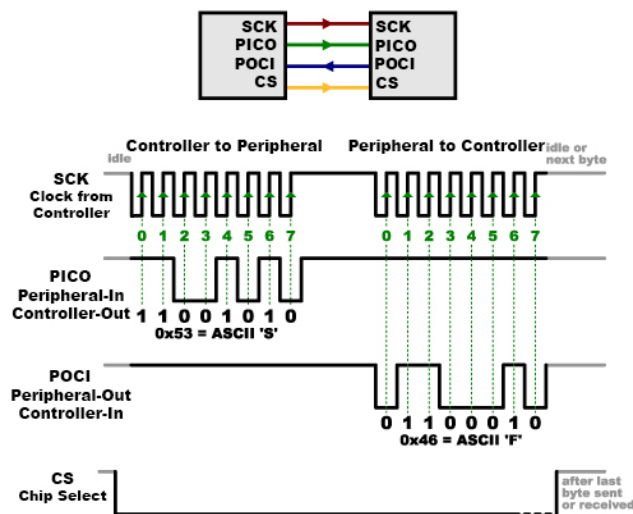
Chuẩn giao tiếp SPI cũng hoạt động dựa trên mô hình Controller - Target (trước đây gọi là Master - Slave). Tuy nhiên, thay vì sử dụng hệ thống địa chỉ như I2C, SPI quản lý các thiết bị ngoại vi bằng các đường vật lý độc lập. Một cấu hình SPI cơ bản yêu cầu 4 đường tín hiệu chính:

SCLK (Serial Clock): Đường truyền xung nhịp đồng bộ do thiết bị Master phát ra để điều khiển tốc độ truyền dữ liệu.

MOSI (Master Out Slave In): Đường truyền dữ liệu theo hướng từ thiết bị Master đến các thiết bị Slave.

MISO (Master In Slave Out): Đường truyền dữ liệu theo hướng ngược lại, từ Slave về Master.

SS / CS (Slave Select / Chip Select): Đường tín hiệu chọn chip độc lập cho từng Slave. Khi Master muốn giao tiếp với Slave nào, nó sẽ kéo đường SS/CS của Slave đó xuống mức thấp (Low).

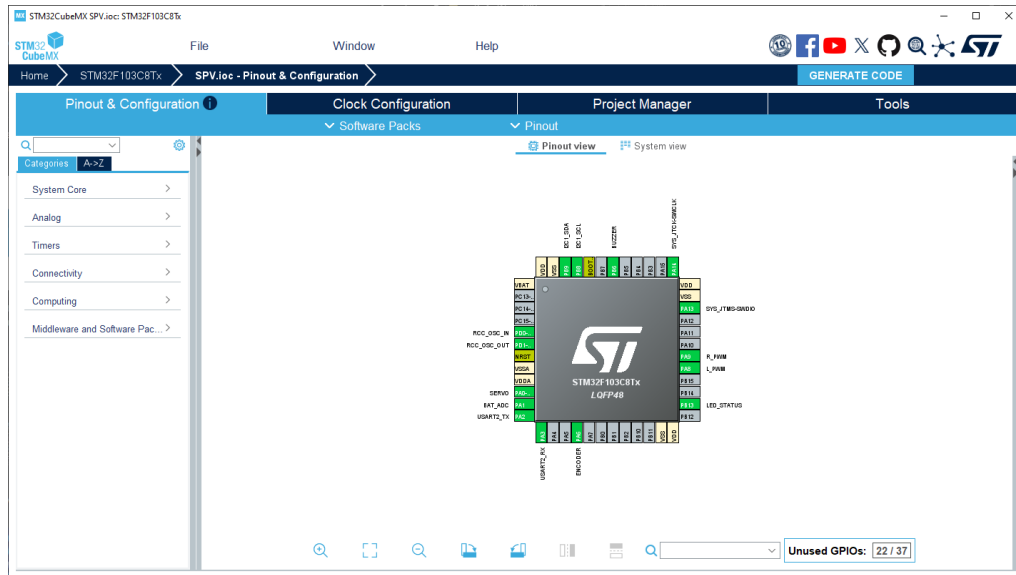


Hình 2.17 Giao tiếp SPI

2.4. TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH CHO HỆ THỐNG

2.4.1. Phần mềm STM32CubeMX

STM32CubeMX là một công cụ đồ họa cấu hình trực quan do hãng STMicroelectronics phát triển, nhằm hỗ trợ tối ưu cho các kỹ sư trong việc thiết kế và khởi tạo dự án trên các dòng vi điều khiển STM32. Phần mềm này giúp đơn giản hóa đáng kể quá trình thiết lập ban đầu, giảm thiểu thời gian phát triển và hạn chế sai sót khi cấu hình phần cứng.



Hình 2.18 Giao diện STM32CUBEMX

Các tính năng nổi bật của STM32CubeMX bao gồm:

Cấu hình sơ đồ chân (Pinout Wizard): Giao diện đồ họa trực quan mô phỏng chip thực tế cho phép người dùng lựa chọn và cấu hình chức năng cho từng chân vi điều khiển (GPIO, ADC, UART, SPI, I2C,...) một cách dễ dàng và tránh xung đột chân.

Thiết lập hệ thống xung nhịp (Clock Tree Configuration): Cung cấp một sơ đồ cây xung nhịp chi tiết, giúp người dùng trực quan hóa và tự động tính toán, cấu hình tần số hoạt động cho lõi và các ngoại vi mà không cần tra cứu thanh ghi phức tạp.

Cấu hình ngoại vi và Middleware: Hỗ trợ thiết lập thông số cho các ngoại vi và tích hợp sẵn các gói phần mềm trung gian (Middleware) phổ biến như FreeRTOS, FATFS, LwIP, USB Library.

Tự động sinh mã nguồn (Code Generation): Sau khi hoàn tất cấu hình phần cứng trên giao diện đồ họa, STM32CubeMX có khả năng tự động khởi tạo mã nguồn C chuẩn. Dự án sinh ra có thể tương thích ngay với các môi trường lập trình (IDE) phổ biến như Keil C ARM (MDK-ARM), IAR Embedded Workbench, hoặc STM32CubeIDE.

2.4.2. Phần mềm Roboflow

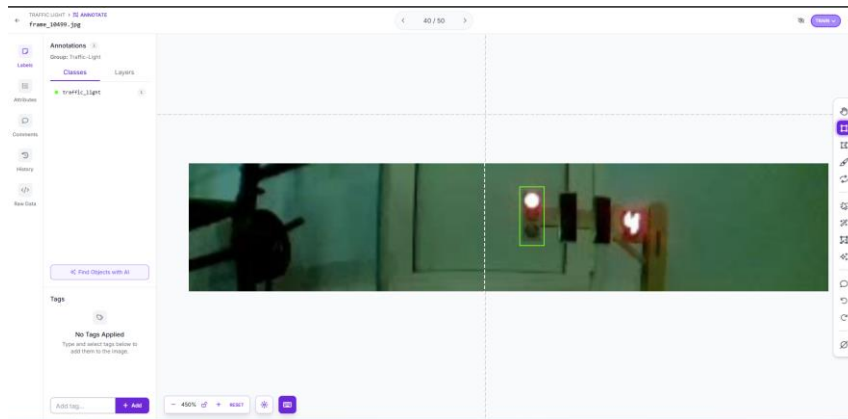
Roboflow là một nền tảng quản lý và xây dựng thị giác máy tính (Computer Vision) toàn diện dựa trên điện toán đám mây [7]. Công cụ này được thiết kế nhằm tối giản hóa quy trình chuẩn bị dữ liệu hình ảnh – một trong những giai đoạn tốn nhiều thời gian nhất khi triển khai các mô hình học sâu (Deep Learning) và trí tuệ nhân tạo (AI). Các chức năng cốt lõi của phần mềm Roboflow bao gồm:

Quản lý và gán nhãn dữ liệu (Labeling/Annotation): Cung cấp các công cụ dán nhãn thông minh, trực quan trên trình duyệt cho các bài toán phân loại ảnh (Classification), phát hiện vật thể (Object Detection) hay phân vùng ngữ nghĩa (Segmentation).

Tiền xử lý hình ảnh (Preprocessing): Cho phép chuẩn hóa tập dữ liệu tự động như thay đổi kích thước ảnh (Resize), chuyển đổi hệ màu (Grayscale), căn chỉnh độ tương phản, giúp dữ liệu đầu vào đồng nhất trước khi huấn luyện.

Tăng cường dữ liệu (Data Augmentation): Tích hợp các thuật toán tạo thêm dữ liệu nhân tạo từ tập dữ liệu gốc (lật ảnh, xoay, thay đổi độ sáng, thêm nhiễu, cắt góc,...). Tính năng này giúp mở rộng quy mô tập dữ liệu, từ đó nâng cao độ chính xác và giảm hiện tượng quá khớp (Overfitting) cho mô hình AI.

Xuất bản và chuyển đổi định dạng (Format Export): Hỗ trợ chuyển đổi linh hoạt tập dữ liệu sang hơn 30 định dạng mã nguồn mở phổ biến hiện nay như YOLO (v5, v8, v11), COCO JSON, Pascal VOC, TensorFlow TFRecord, giúp dễ dàng tích hợp vào các framework huấn luyện khác nhau.



Hình 2.19 Giao diện Roboflow

2.4.3. Phần mềm Visual Studio Code

Visual Studio Code (VS Code) là một trình soạn thảo mã nguồn tự do, mã nguồn mở và hỗ trợ đa nền tảng (Windows, macOS, Linux) do Microsoft phát triển. Nhờ sở hữu giao diện trực quan, hiệu năng cao cùng hệ sinh thái tiện ích mở rộng phong phú, VS Code hiện là một trong những công cụ lập trình phổ biến và được ưa chuộng nhất trên thế giới.



Hình 2.20 VScode

Các tính năng cốt lõi và thế mạnh của Visual Studio Code bao gồm:

Tính năng IntelliSense: Hỗ trợ tự động gợi ý mã lệnh, hoàn thiện tên biến, hàm và hiển thị tài liệu API nhanh chóng. Tiện ích này giúp lập trình viên tối ưu hóa thời gian viết code và hạn chế tối đa các lỗi cú pháp.

Trình quản lý Git tích hợp: Cho phép người dùng thực hiện trực tiếp các thao tác quản lý phiên bản như *commit*, *push*, *pull*, *merge* ngay trên giao diện làm việc mà không cần sử dụng terminal độc lập hay phần mềm bên thứ ba.

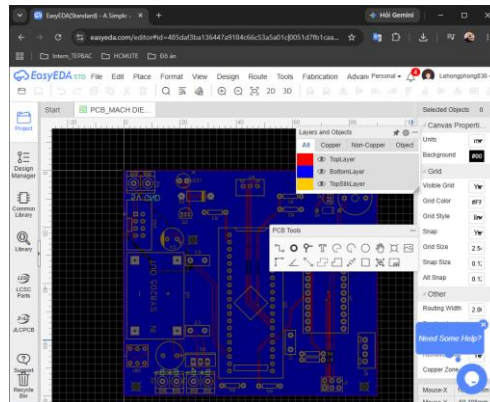
Hệ thống Debugging mạnh mẽ: Hỗ trợ thiết lập breakpoint (điểm dừng), theo dõi giá trị biến và kiểm soát luồng thực thi trực quan cho nhiều ngôn ngữ lập trình khác nhau như C++, Python, JavaScript,...

Terminal tích hợp: Cung cấp cửa sổ dòng lệnh ngay trong môi trường phát triển, giúp lập trình viên chạy các lệnh hệ thống, biên dịch hoặc thực thi chương trình một cách nhanh chóng.

Kho tiện ích mở rộng khổng lồ (Extension Marketplace): Người dùng có thể linh hoạt cài đặt thêm các công cụ định dạng mã (formatter), kiểm tra lỗi (linting), container hóa với Docker, hoặc các thư viện hỗ trợ chuyên sâu cho Machine Learning và các Web framework hiện đại (React, Angular, Vue).

2.4.4. Phần mềm EasyEDA

EasyEDA là một bộ công cụ thiết kế mạch điện tử (EDA - Electronic Design Automation) hoạt động hoàn toàn trên nền tảng điện toán đám mây. Do không yêu cầu cài đặt phức tạp và hoàn toàn miễn phí, EasyEDA là giải pháp tối ưu được ứng dụng rộng rãi bởi các kỹ sư điện, giảng viên, sinh viên ngành kỹ thuật và những người đam mê điện tử.



Hình 2.21 Giao diện EASYEDA

Phần mềm sở hữu nhiều ưu điểm và tính năng nổi bật:

Thiết kế mạch nguyên lý (Schematic) nhanh chóng: Tích hợp kho thư viện linh kiện khổng lồ, liên tục được cập nhật trực tuyến, giúp người dùng dễ dàng tìm kiếm và thiết kế sơ đồ nguyên lý trên trình duyệt.

Mô phỏng mạch linh hoạt: Hỗ trợ kiểm tra, mô phỏng các mạch tín hiệu tương tự (analog), tín hiệu số (digital) và mạch hỗn hợp thông qua các mô hình mạch phụ tích hợp sẵn.

Thiết kế mạch in (PCB Layout) đa lớp: Cung cấp công cụ mạnh mẽ để bố trí linh kiện hợp lý và đi dây tối ưu cho cả các bo mạch nhiều lớp (Multi-layer PCB) có độ phức tạp cao.

Tính tiện dụng và khả năng truy cập: Giao diện thân thiện, dễ tiếp cận, hỗ trợ làm việc trực tuyến mọi lúc mọi nơi mà không phụ thuộc vào cấu hình phần cứng của máy tính cá nhân.

2.5. TỔNG QUAN VỀ EDGE AI VÀ HỆ THỐNG SOC

2.5.1. Công nghệ điện toán biên (Edge Computing)

Trong các kiến trúc Internet of Things (IoT) truyền thống, mô hình xử lý dữ liệu thường được tổ chức theo dạng tập trung, trong đó dữ liệu từ các cảm biến được truyền tải lên các máy chủ đám mây (Cloud Server) để tiến hành phân tích, xử lý và sau đó gửi kết quả trở lại thiết bị đầu cuối. Mô hình này mang lại lợi thế về khả năng mở rộng và sức mạnh tính toán gần như vô hạn, tuy nhiên lại tồn tại những hạn chế đáng kể khi áp dụng vào các hệ thống yêu cầu phản hồi theo thời gian thực, điển hình như xe tự hành.

Một trong những vấn đề nghiêm trọng nhất là độ trễ mạng (Latency). Khi dữ liệu phải di chuyển qua nhiều lớp mạng trước khi đến được trung tâm xử lý, thời gian trễ có thể dao động từ hàng chục đến hàng trăm mili-giây, thậm chí cao hơn trong điều kiện mạng không ổn định. Đối với hệ thống xe tự hành, nơi các quyết định như đánh lái, phanh hay tránh vật cản cần được đưa ra gần như tức thời, độ trễ này có thể dẫn đến hậu quả nghiêm trọng. Ngoài ra, sự phụ thuộc vào kết nối internet liên tục cũng làm giảm độ tin cậy của hệ thống, đặc biệt trong các môi trường không có hạ tầng mạng ổn định.

Để khắc phục các hạn chế trên, mô hình điện toán biên (Edge Computing) đã được đề xuất và ngày càng được ứng dụng rộng rãi. Trong mô hình này, tài nguyên tính toán được phân tán và đưa xuống gần nguồn phát sinh dữ liệu hơn, tức là tại “rìa” của mạng lưới. Cụ thể, thay vì gửi toàn bộ dữ liệu lên đám mây, thiết bị đầu cuối (ví dụ như xe tự hành) sẽ tự thực hiện các tác vụ xử lý chính ngay trên phần cứng của mình.

Một khái niệm quan trọng liên quan là Edge AI, tức là việc triển khai và thực thi các mô hình trí tuệ nhân tạo trực tiếp trên thiết bị biên. Điều này cho phép hệ thống thực hiện các tác vụ như nhận diện làn đường, phát hiện vật cản, phân loại đối tượng hay dự đoán hành vi trong thời gian thực mà không cần phụ thuộc vào máy chủ từ xa. Nhờ đó, độ trễ được giảm xuống mức mili-giây (ms), đáp ứng yêu cầu thời gian thực (Real-time) của các hệ thống điều khiển.

Bên cạnh việc giảm độ trễ, Edge Computing còn mang lại nhiều lợi ích khác. Thứ nhất là giảm tải cho hệ thống mạng và máy chủ đám mây, do chỉ những dữ liệu quan trọng hoặc đã được xử lý sơ bộ mới cần truyền đi. Thứ hai là tăng cường bảo mật và quyền riêng tư, khi dữ liệu nhạy cảm không cần phải rời khỏi thiết bị. Thứ ba là nâng cao tính ổn định và khả năng hoạt động độc lập của hệ thống trong các tình huống mất kết nối mạng.

Trong bối cảnh phát triển xe tự hành cỡ nhỏ hoặc mô hình nghiên cứu, Edge Computing đóng vai trò then chốt, cho phép hệ thống hoạt động linh hoạt, phản ứng nhanh và không phụ thuộc vào hạ tầng bên ngoài. Đây là nền tảng quan trọng để xây dựng các hệ thống điều khiển thông minh, an toàn và hiệu quả.

2.5.2. Hệ thống System-on-Chip (SoC)

System-on-Chip (SoC) là một giải pháp tích hợp cao trong thiết kế phần cứng, trong đó toàn bộ các thành phần chính của một hệ thống máy tính được tích hợp trên một vi mạch duy nhất. Thay vì sử dụng nhiều chip riêng lẻ như trong các kiến trúc truyền thống, SoC cho phép giảm kích thước, tiêu thụ điện năng và độ phức tạp của hệ thống, đồng thời tăng hiệu suất tổng thể.

Một SoC điển hình thường bao gồm nhiều thành phần quan trọng. Trước hết là Bộ vi xử lý trung tâm (CPU), thường dựa trên kiến trúc ARM, đảm nhiệm việc điều khiển và xử lý các tác vụ logic chung của hệ thống. Bên cạnh đó là Bộ xử lý đồ họa (GPU), hỗ trợ các phép toán song song, đặc biệt hữu ích trong xử lý hình ảnh và video. Ngoài ra, SoC còn tích hợp bộ nhớ RAM, các bộ điều khiển ngoại vi (I/O controllers), và

trong nhiều trường hợp hiện đại, còn có cả Bộ xử lý thần kinh (NPU – Neural Processing Unit) chuyên dụng để tăng tốc các phép toán liên quan đến mạng nơ-ron. Sự xuất hiện của NPU là một bước tiến quan trọng trong bối cảnh phát triển trí tuệ nhân tạo tại biên. NPU được thiết kế để thực hiện các phép nhân ma trận và tích chập với hiệu suất cao và tiêu thụ điện năng thấp hơn so với CPU hoặc GPU. Điều này đặc biệt phù hợp với các ứng dụng như nhận diện hình ảnh, xử lý tín hiệu và học máy, vốn là những thành phần cốt lõi trong hệ thống xe tự hành.

Trong thực tế, các SoC thường được tích hợp sẵn trong các máy tính bo mạch đơn (Single-board Computer – SBC) như Raspberry Pi, Orange Pi hoặc NVIDIA Jetson. Những nền tảng này cung cấp một môi trường phát triển hoàn chỉnh, bao gồm phần cứng, hệ điều hành (thường là các bản phân phối Linux như Ubuntu) và hệ sinh thái phần mềm phong phú. Nhờ đó, các nhà phát triển có thể dễ dàng triển khai các ứng dụng từ mức thấp (driver, giao tiếp phần cứng) đến mức cao (xử lý ảnh, AI).

Đối với các hệ thống xe tự hành cỡ nhỏ, việc sử dụng SBC dựa trên SoC mang lại nhiều lợi ích. Thứ nhất là kích thước nhỏ gọn, phù hợp với không gian hạn chế trên xe. Thứ hai là khả năng tiêu thụ điện năng thấp, giúp kéo dài thời gian hoạt động khi sử dụng pin. Thứ ba là hiệu năng đủ mạnh để xử lý các tác vụ phức tạp như thị giác máy tính (Computer Vision) và điều khiển thời gian thực.

Tóm lại, SoC là nền tảng phần cứng cốt lõi cho các hệ thống nhúng hiện đại, đặc biệt trong lĩnh vực xe tự hành. Khi kết hợp với mô hình Edge Computing, SoC cho phép triển khai các thuật toán phức tạp ngay trên thiết bị, đảm bảo hiệu suất cao, độ trễ thấp và khả năng hoạt động độc lập. Đây là yếu tố then chốt trong việc xây dựng các hệ thống thông minh thế hệ mới.

2.6. CƠ SỞ LÝ THUYẾT VỀ XỬ LÝ ẢNH

2.6.1. Không gian màu và các phép biến đổi

Ảnh thô thu được từ camera thường ở định dạng RGB (Red, Green, Blue). Tuy nhiên, không gian màu RGB rất nhạy cảm với sự thay đổi của cường độ ánh sáng môi trường (bóng râm, lóa sáng). Do đó, để phân tách vạch kẻ đường, hệ thống thường chuyển

đổi ảnh từ không gian RGB sang không gian HSV (Hue, Saturation, Value) hoặc Grayscale.

2.6.2. Lọc nhiễu ảnh và phát hiện biên

Trước khi trích xuất đặc trưng hình học, ảnh cần được làm mịn để loại bỏ nhiễu hạt (noise). Bộ lọc Gauss (Gaussian Blur) thực hiện phép tích chập (Convolution) giữa ảnh và một ma trận hạt nhân (Kernel) phân bố chuẩn [8]. Công thức của hàm Gauss 2 chiều được biểu diễn như sau:

$$G(x, y) = \left(\frac{1}{2\pi\sigma^2} \right) e^{\left\{ -\left(\frac{x^2+y^2}{2\sigma^2} \right) \right\}} \quad (2.1)$$

Trong đó, x, y là khoảng cách từ gốc tọa độ đến điểm đang xét trên trục hoành và trục tung, σ là độ lệch chuẩn của phân bố.

Sau khi làm mịn, thuật toán Canny Edge Detection được áp dụng. Thuật toán này sử dụng đạo hàm không gian để tìm ra các điểm có sự thay đổi đột ngột về cường độ sáng (gradient pixel), từ đó phác họa lên các đường viền của vạch kẻ đường [8].

2.6.3. Biến đổi phối cảnh (Perspective Transformation)

Trong các hệ thống xe tự hành, hình ảnh thô từ camera hành trình luôn chịu hiệu ứng phối cảnh vật lý, khiến hai vạch kẻ làn đường song song có xu hướng hội tụ tại đường chân trời. Phép biến đổi phối cảnh sang góc nhìn chim bay (Bird's-Eye View) đóng vai trò loại bỏ hoàn toàn sự méo dạng hình học này, ánh xạ các pixel từ mặt phẳng thấu kính góc nghiêng về một mặt phẳng lưới 2D tuyến tính nhìn thẳng từ trên xuống [9]. Điều này cho phép hệ thống tính toán và quy đổi các khoảng cách hình học trực tiếp bằng đơn vị vật lý thực tế (như cm hoặc mét) thay vì đo đạc mơ hồ bằng số lượng pixel đại diện vốn bị biến thiên theo chiều sâu của tầm nhìn.

Ảnh chụp từ camera trên xe có hiệu ứng phối cảnh, khiến các vạch kẻ đường song song có xu hướng hội tụ tại một điểm ở đường chân trời. Để tính toán góc lệch và độ cong của làn đường một cách chính xác, hệ thống cần thực hiện biến đổi phối cảnh sang góc nhìn từ trên xuống (Bird's-eye view). Phép biến đổi ma trận để ánh xạ tọa

độ pixel (x, y) trên ảnh gốc sang tọa độ (x', y') trên ảnh góc nhìn chim bay được thực hiện thông qua ma trận biến đổi 3×3 :

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.2)$$

Tọa độ thực tế của pixel sau phép chiếu được tính bằng $X = \frac{x'}{w'}$ và $Y = \frac{y'}{w'}$.

2.7. TỔNG QUAN VỀ THUẬT TOÁN GIỮ LÀN

Sau khi module xử lý ảnh đã tính toán ra phương trình quỹ đạo của làn đường và sai số lệch tâm, nhiệm vụ tiếp theo của hệ thống ADAS là điều khiển vô lăng (góc lái) để xe bám sát quỹ đạo mong muốn. Để thiết kế luật điều khiển chính xác, cần xây dựng nền tảng toán học cho mô hình động học của phương tiện.

2.7.1. Phân tích Sai số bám quỹ đạo (Tracking Errors)

Hệ thống sử dụng hai loại sai số chính làm đầu vào cho bộ điều khiển:

Sai số ngang (Cross-Track Error - CTE): Khoảng cách vuông góc ngắn nhất từ tâm đầu xe đến tâm làn đường (được xác định từ phương trình đa thức nội suy qua các điểm ảnh vạch kẻ đường).

Sai số góc hướng (Heading Error - θ_e): Độ chênh lệch giữa góc định hướng hiện tại của thân xe và góc tiếp tuyến của quỹ đạo chuẩn tại điểm đang xét.

$$\theta_e = \theta_{xe} - \theta_{quỹđạo} \quad (2.3)$$

2.8. TỔNG QUAN VỀ AI VÀ MÔ HÌNH YOLO

2.8.1. Trí tuệ nhân tạo (AI) trong xe tự hành

Trí tuệ nhân tạo (Artificial Intelligence – AI) là một nhánh của khoa học máy tính tập trung nghiên cứu và phát triển các hệ thống có khả năng tự học hỏi từ dữ liệu, suy luận và đưa ra quyết định giống hoặc vượt trội con người ở một số nhiệm vụ cụ thể. Trong lĩnh vực xe tự hành (Autonomous Vehicles), AI đóng vai trò là "bộ não" cốt lõi, đặc biệt là trong phân hệ Nhận thức (Perception) sử dụng thị giác máy tính (Computer Vision) để hiểu môi trường xung quanh.



Hình 2.22 AI trong hệ thống xe tự hành

Nhờ các thuật toán học sâu (Deep Learning), AI có thể phân tích luồng video camera theo thời gian thực để nhận diện các thực thể trên đường phố với độ chính xác cao, giúp xe đưa ra các quyết định điều khiển an toàn.

2.8.2. Mô hình YOLO – You Only Look You

YOLO (You Only Look Once) là một trong những thuật toán phát hiện đối tượng thời gian thực (real-time object detection) nổi bật nhất hiện nay. Được Joseph Redmon và các cộng sự giới thiệu lần đầu vào năm 2016 [10], YOLO đã thay đổi hoàn toàn cách tiếp cận trong lĩnh vực thị giác máy tính nhờ triết lý đơn giản nhưng cực kỳ hiệu quả: “chỉ cần nhìn ảnh một lần” là đủ để xác định đồng thời vị trí (bounding box) và lớp (class) của tất cả các đối tượng trong ảnh.



Hình 2.23 Ứng dụng mô hình YOLO

Khác với các mô hình truyền thống hai giai đoạn (ví dụ: R-CNN, Fast R-CNN, Faster R-CNN) phải sinh vùng đề xuất (region proposals) rồi mới phân loại từng vùng, YOLO sử dụng kiến trúc một giai đoạn (single-stage detector): Chia ảnh đầu vào từ camera hành trình thành lưới $S \times S$ ô (grid cells).

Mỗi ô chịu trách nhiệm dự đoán một số khung bao (bounding boxes) cố định cùng xác suất lớp (class probabilities) của đèn giao thông hoặc vật thể.

Kết quả cuối cùng được tinh chỉnh bằng thuật toán Non-Maximum Suppression (NMS) để loại bỏ các khung trùng lặp, giữ lại vị trí đèn giao thông chính xác nhất.

Bảng 2.12 So sánh YOLO với RCNN

STT	Tiêu chí	YOLO (Kiến trúc một giai đoạn)	Mô hình hai giai đoạn (R-CNN series)
1	Tốc độ xử lý	Cực nhanh (Đạt từ 45 FPS đến hơn 100 FPS trên GPU)	Chậm (Vài khung hình trên giây, gây trễ lớn)
2	Độ chính xác	Cân bằng tốt giữa tốc độ và độ chính xác	Cao hơn một chút ở các phiên bản đầu
3	Kiến trúc	Đơn giản, mạng nơ-ron liên mạch (End-to-end)	Phức tạp, tách biệt nhiều giai đoạn xử lý
4	Khả năng thời gian thực	Tốt nhất, đáp ứng hoàn hảo cho xe tự hành	Không phù hợp cho ứng dụng di động thực tế
5	Bối cảnh hóa (Context)	Ít sai sót nhận diện nền nhờ thấy toàn bộ ảnh	Dễ nhầm nền do chỉ nhìn cục bộ các vùng đề xuất

YOLO đã trải qua nhiều phiên bản cải tiến mạnh mẽ (YOLOv5, YOLOv8, YOLOv11...) với các nâng cấp lớn về cấu trúc mạng (backbone chuyển sang CSPDarknet, C2f), đầu phát hiện chuyển từ dạng dựa trên khung neo (anchor-based) sang không dựa trên khung neo (anchor-free), và tối ưu hóa tối đa cho các chip nhúng phân cứng.

Tuy nhiên, ý tưởng cốt lõi “nhìn một lần – dự đoán tất cả” vẫn là nền tảng làm nên sức mạnh và sự phổ biến của dòng mô hình này trong các ứng dụng yêu cầu phản hồi tức thì và nghiêm ngặt về độ an toàn như bài toán Phân loại đèn giao thông cho xe tự hành.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1. YÊU CẦU VÀ SƠ ĐỘ KHỐI HỆ THỐNG

3.1.1. Yêu cầu của hệ thống

Hệ thống xe tự hành được thiết kế theo kiến trúc Edge AI nhằm thực hiện các chức năng hỗ trợ lái xe nâng cao (ADAS) trên mô hình thu nhỏ. Hệ thống bao gồm hai khối chính: khối xử lý trí tuệ nhân tạo và khối điều khiển thời gian thực, phối hợp với nhau để đảm bảo khả năng nhận thức môi trường và điều khiển chuyển động của xe một cách chính xác và ổn định.

Khối xử lý AI sử dụng nền tảng Raspberry Pi kết hợp với camera để thu nhận dữ liệu hình ảnh môi trường phía trước. Dữ liệu này được xử lý thông qua các thuật toán thị giác máy tính và mô hình học máy nhằm thực hiện các chức năng như phát hiện làn đường, giữ làn, nhận diện đèn giao thông và phát hiện vật cản. Kết quả xử lý được hiển thị trực tiếp trên màn hình LCD để phục vụ giám sát hệ thống theo thời gian thực. Khối điều khiển sử dụng vi điều khiển STM32 để đảm bảo khả năng điều khiển thời gian thực. Các lệnh điều khiển (tốc độ và góc lái) được truyền từ Raspberry Pi đến STM32 thông qua giao tiếp UART. STM32 xử lý các lệnh này và tạo tín hiệu PWM để điều khiển động cơ thông qua driver công suất, cho phép xe thực hiện các hành vi như di chuyển thẳng, rẽ trái, rẽ phải hoặc dừng lại.

Hệ thống nguồn sử dụng pin Li-ion, kết hợp với bộ chuyển đổi DC-DC để cung cấp điện áp ổn định cho toàn bộ hệ thống.

Để đảm bảo hệ thống hoạt động đúng mục tiêu, các yêu cầu chức năng và tiêu chí đánh giá được xác định như sau:

- + **Giữ làn đường (Lane Keeping):** Hệ thống phải phát hiện được biên làn đường và duy trì xe di chuyển trong làn.
- + **Phát hiện đèn giao thông (Traffic Light Detection):** Hệ thống phải nhận diện và phân loại trạng thái đèn giao thông (đỏ, vàng, xanh).

- + **Phản ứng với đèn giao thông (Traffic Light Response):** Xe phải dừng lại khi gặp đèn đỏ và tiếp tục di chuyển khi đèn chuyển sang xanh.
- + **Phát hiện vật cản (Obstacle Detection):** Hệ thống phải phát hiện được vật cản phía trước trong phạm vi hoạt động.

Hiển thị thông tin (LCD Display): Hệ thống sử dụng màn hình LCD để giám sát vận hành và hỗ trợ gỡ lỗi với tốc độ cập nhật tối thiểu 30 FPS. Giao diện được phân thành 3 chế độ linh hoạt:

- **Giao diện điều khiển chính (Main HUD):** Tập trung vào nhận thức môi trường và trạng thái xe:

Cảnh báo: Cảnh báo vật cản và hiển thị khoảng cách từ cảm biến TOF.

Điều khiển: Độ lệch làn (ERROR), góc lái (STEER), tốc độ (SPEED).

Trạng thái: Chế độ chạy (MODE), điện áp (PIN).

Hạ tầng: Trạng thái giao lộ (INTERSEC) và đèn giao thông (TL: Red, Green, Yellow...).

- **Bảng điều khiển hệ thống (System Dashboard):** Giám sát thông số kỹ thuật phần cứng:

BATTERY: Dung lượng Pin.

MOTOR RPM: Tốc độ vòng quay động cơ.

TOF DISTANCE: Dữ liệu khoảng cách vật cản thời gian thực.

- **Chế độ gỡ lỗi (BEV Debug Mode):** Hiển thị dữ liệu chuyên sâu để hiệu chỉnh thuật toán giữ làn:

BEV Binary: Hình ảnh nhị phân từ góc nhìn chim bay (Bird's Eye View).

Lane Metrics: Độ rộng làn đường (Lane Width).

Geometry: Sai số (Err) và góc lệch (Angle) so với tâm đường.

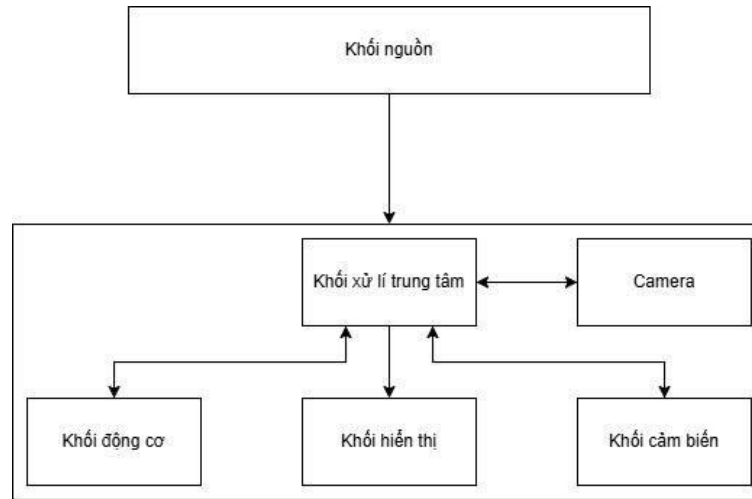
Điều khiển thời gian thực (Real-time Control): Việc truyền lệnh giữa khối xử lý và khối điều khiển phải đảm bảo tính liên tục và độ trễ thấp.

Tiêu chí: Tần suất gửi lệnh tối thiểu 10 Hz.

Điều khiển lái vi sai (Differential Steering): Xe phải điều hướng bằng cách điều chỉnh góc lái và tốc độ động cơ.

Những yêu cầu trên là cơ sở để thiết kế, triển khai và đánh giá hiệu năng của hệ thống xe tự hành trong phạm vi nghiên cứu.

3.1.2. Sơ đồ khối hệ thống và chức năng của từng khối



Hình 3.1 Sơ đồ khối hệ thống

Chức năng cơ bản của các khối trong hệ thống:

- Khối Nguồn

Khối nguồn cung cấp năng lượng cho toàn bộ hệ thống. Nguồn này cấp điện cho:

- + Driver động cơ
- + Bộ hạ áp
- + Vi điều khiển STM32
- + Vi xử lý Raspberry Pi

- Khối xử lý trung tâm

Khối xử lý trung tâm đóng vai trò là “bộ não” của toàn bộ hệ thống xe tự hành, thực hiện chức năng tiếp nhận dữ liệu, xử lý thông tin và đưa ra quyết định điều khiển. Khối này bao gồm hai thành phần chính: vi xử lý Raspberry Pi và vi điều khiển

STM32, phối hợp với nhau để đảm bảo khả năng xử lý thông minh và điều khiển thời gian thực.

Vi xử lý Raspberry Pi đảm nhiệm vai trò xử lý cấp cao, đặc biệt là các tác vụ liên quan đến thị giác máy tính và trí tuệ nhân tạo. Dữ liệu hình ảnh từ camera được xử lý để trích xuất thông tin môi trường, bao gồm ba chức năng chính:

Phát hiện và giữ làn đường (Lane Detection & Lane Keeping): Xử lý hình ảnh nhằm phát hiện vạch kẻ đường, xác định vị trí tương đối của xe so với tâm làn, từ đó tính toán sai lệch quỹ đạo và cung cấp dữ liệu phục vụ điều khiển giữ làn.

Nhận diện biển báo giao thông (Traffic Sign/Light Detection): Sử dụng các thuật toán xử lý ảnh hoặc mô hình học máy để phát hiện và phân loại các loại biển báo giao thông, đặc biệt là trạng thái đèn giao thông.

Phát hiện vật cản (Obstacle Detection): Xác định sự xuất hiện của các vật thể nằm trên quỹ đạo di chuyển của xe, cung cấp thông tin về vị trí và mức độ nguy hiểm của vật cản.

Dựa trên các thông tin đầu vào này, Raspberry Pi thực hiện tổng hợp và phân tích dữ liệu để đưa ra quyết định điều khiển ở mức cao như: duy trì hướng đi, rẽ trái/phải, giảm tốc hoặc dừng xe. Đồng thời, các thông tin trạng thái hệ thống cũng được cập nhật và hiển thị trên màn hình.

Vi điều khiển STM32 đảm nhiệm vai trò điều khiển thời gian thực. STM32 nhận các lệnh điều khiển (góc lái và tốc độ) từ Raspberry Pi thông qua giao tiếp UART, sau đó chuyển đổi thành tín hiệu điều khiển phù hợp để điều khiển cơ cấu chấp hành (servo và động cơ). Ngoài ra, STM32 còn trực tiếp thu thập dữ liệu từ các cảm biến để phát hiện vật cản trong thời gian thực, từ đó có thể chủ động thực hiện các hành động an toàn như dừng khẩn cấp hoặc hỗ trợ tránh vật cản ngay cả khi chưa có phản hồi từ khối xử lý AI.

Sự kết hợp giữa Raspberry Pi (xử lý cấp cao) và STM32 (điều khiển thời gian thực) giúp hệ thống đạt được sự cân bằng giữa khả năng tính toán phức tạp và độ phản hồi nhanh, đảm bảo xe vận hành ổn định và an toàn trong các tình huống thực tế.

- Khối động cơ

Khối động cơ có nhiệm vụ thực thi các lệnh điều khiển về hướng lái và tốc độ được gửi từ khối xử lý trung tâm. Khối này bao gồm hai thành phần chính: servo điều khiển góc lái và bộ điều khiển động cơ (motor driver).

Servo được sử dụng để điều khiển hướng lái của xe thông qua việc thay đổi góc quay của bánh trước. Góc lái được thiết lập trong khoảng từ 45° đến 135° , trong đó 90° tương ứng với trạng thái xe đi thẳng, 45° là góc rẽ tối đa sang trái và 135° là góc rẽ tối đa sang phải.

Bộ điều khiển động cơ có nhiệm vụ nhận tín hiệu điều khiển tốc độ dưới dạng giá trị chuẩn hóa trong khoảng $[0, 1]$ từ khối xử lý trung tâm. Tín hiệu này được chuyển đổi thành xung điều chế độ rộng xung (PWM) để điều khiển tốc độ quay của động cơ DC. Thông qua việc thay đổi độ rộng xung PWM, hệ thống có thể điều chỉnh tốc độ xe một cách linh hoạt và liên tục.

Khối động cơ đóng vai trò quan trọng trong việc đảm bảo xe di chuyển chính xác theo quỹ đạo mong muốn, đáp ứng kịp thời các yêu cầu điều khiển từ hệ thống xử lý AI.

- Khối Camera

Khối Camera có nhiệm vụ thu thập dữ liệu hình ảnh từ môi trường phía trước xe. Camera liên tục ghi lại video hoặc khung hình của mặt đường, biển báo và các vật thể xung quanh để cung cấp dữ liệu đầu vào cho hệ thống thị giác máy tính. Dữ liệu hình ảnh này sẽ được gửi đến khối thị giác máy tính và AI để phân tích.

- Khối hiển thị

Khối HMI đóng vai trò là giao diện giám sát và hiển thị trạng thái hoạt động của hệ thống xe tự hành theo thời gian thực. Hệ thống sử dụng màn hình TFT LCD với tốc độ cập nhật tối thiểu 30 FPS nhằm hỗ trợ vận hành và debug thuật toán.

Màn hình được thiết kế gồm 3 giao diện chính:

Main HUD: Hiển thị các thông tin quan trọng trong quá trình xe hoạt động như: cảnh báo vật cản, độ lệch tâm làn (ERROR), hướng lái (STEER), tốc độ xe (SPEED), điện

áp pin (PIN), trạng thái giao lộ (INTERSEC), chế độ hoạt động của xe (MODE), trạng thái đèn giao thông (TL) và khoảng cách phát hiện vật cản (TOF).

System Dashboard: Hiển thị các thông số phần cứng và trạng thái hệ thống gồm dung lượng pin (BATTERY), tốc độ động cơ (MOTOR RPM) và khoảng cách cảm biến TOF.

BEV Debug Mode: Hiển thị ảnh Bird-Eye View (BEV) dạng nhị phân phục vụ kiểm tra thuật toán giữ làn, bao gồm các thông tin như độ rộng làn đường (Lane Width), sai lệch tâm làn (ERR) và góc lái điều khiển (ANGLE).

Khối HMI giúp người dùng dễ dàng theo dõi trạng thái hoạt động của xe, đồng thời hỗ trợ quá trình đánh giá và tối ưu hệ thống trong quá trình nghiên cứu và phát triển.

- Khối cảm biến

Đây là khối cảm biến bổ sung (cảm biến TOF) đo khoảng cách và giúp phát hiện vật cản ở khoảng cách gần. Thông tin từ cảm biến này được gửi trực tiếp đến khối điều khiển xe để hỗ trợ việc dừng xe khẩn cấp hoặc tránh va chạm.

*** Nguyên lý hoạt động của hệ thống**

Hệ thống xe tự hành hoạt động dựa trên sự phối hợp giữa các khối chức năng, trong đó khối xử lý trung tâm đóng vai trò tiếp nhận dữ liệu từ camera và cảm biến, xử lý thông tin và đưa ra quyết định điều khiển. Tùy theo điều kiện môi trường và kết quả nhận thức, hệ thống sẽ chuyển đổi giữa các trạng thái hoạt động khác nhau nhằm đảm bảo khả năng vận hành ổn định và an toàn.

Trạng thái giữ làn ổn định (Normal Lane Keeping Mode)

Trong điều kiện lý tưởng, camera liên tục thu nhận hình ảnh môi trường phía trước và truyền về vi xử lý Raspberry Pi để xử lý. Hệ thống thực hiện phát hiện làn đường và xác định vị trí tương đối của xe so với tâm làn. Dựa trên sai lệch này, thuật toán điều khiển sẽ tính toán góc lái và tốc độ phù hợp.

Các lệnh điều khiển sau đó được truyền qua giao tiếp UART đến vi điều khiển STM32 để điều khiển servo và động cơ. Đồng thời, các thông tin như hình ảnh đã xử lý, vị trí làn đường và trạng thái xe được hiển thị trên màn hình LCD nhằm phục vụ

giám sát và debug. Quá trình này diễn ra liên tục theo thời gian thực, giúp xe duy trì quỹ đạo ổn định trong làn đường.

Trạng thái nhận diện một bên làn (Single Lane Detection Mode)

Trong trường hợp hệ thống chỉ phát hiện được một biên của làn đường (trái hoặc phải), khối xử lý trung tâm sẽ ước lượng biên còn lại dựa trên mô hình hình học hoặc thông tin lịch sử. Từ đó, hệ thống suy ra vị trí tâm làn giả định và tiếp tục điều khiển xe di chuyển theo hướng phù hợp.

Cơ chế này giúp xe duy trì khả năng vận hành trong điều kiện môi trường không hoàn hảo (ví dụ: vạch đường bị mờ hoặc bị che khuất). Khi cả hai biên làn được nhận diện đầy đủ trở lại, hệ thống sẽ tự động chuyển về trạng thái giữ làn ổn định.

Trạng thái mất làn (Lane Loss Mode)

Khi hệ thống không thể phát hiện được bất kỳ biên làn nào trong một khoảng thời gian xác định, trạng thái mất làn được kích hoạt. Để đảm bảo an toàn, hệ thống sẽ đưa ra lệnh dừng xe khẩn cấp.

Việc dừng xe trong tình huống này giúp tránh các rủi ro do thiếu thông tin định hướng, đặc biệt trong môi trường có nhiều nhiễu hoặc điều kiện ánh sáng không thuận lợi.

Trạng thái phát hiện và xử lý vật cản (Obstacle Handling Mode)

Khi phát hiện vật cản phía trước thông qua camera hoặc cảm biến, hệ thống sẽ đánh giá khoảng cách và đưa ra hành vi phù hợp:

Trường hợp khoảng cách ≤ 15 cm: Hệ thống kích hoạt cơ chế dừng khẩn cấp nhằm tránh va chạm.

Trường hợp khoảng cách > 15 cm: Hệ thống thực hiện dừng xe ngay lập tức để đảm bảo an toàn.

Quá trình này yêu cầu sự phối hợp giữa nhận thức môi trường và điều khiển thời gian thực để đảm bảo phản ứng nhanh và chính xác.

Trạng thái nhận diện và phản ứng với đèn giao thông (Traffic Light Handling Mode)

Khi hệ thống phát hiện đèn giao thông, trạng thái hoạt động của xe sẽ được điều chỉnh dựa trên kết quả phân loại:

Đèn đỏ: Xe dừng hoàn toàn.

Đèn vàng: Xe dừng, không tăng ga nữa.

Đèn xanh: Xe tiếp tục di chuyển theo chế độ giữ làn bình thường.

Cơ chế này giúp mô hình xe tự hành mô phỏng hành vi tham gia giao thông thực tế, đảm bảo tuân thủ các quy tắc cơ bản.

Tổng thể, hệ thống hoạt động theo nguyên tắc vòng lặp khép kín (closed-loop), trong đó dữ liệu môi trường được liên tục thu nhận, xử lý và phản hồi thành các lệnh điều khiển. Sự kết hợp giữa các chế độ hoạt động khác nhau cho phép hệ thống thích nghi với nhiều tình huống thực tế, đồng thời đảm bảo tính ổn định và an toàn trong quá trình vận hành.

3.2. THIẾT KẾ HỆ THỐNG

3.2.1. Thiết kế khối nguồn

* Chức năng của khối

Khối nguồn đóng vai trò là hệ thống tuần hoàn năng lượng, chịu trách nhiệm tiếp nhận, chuyển đổi và phân phối điện áp một cách ổn định, an toàn cho toàn bộ các linh kiện điện tử trên xe tự hành. Hệ thống nguồn phải đảm bảo cung cấp dòng điện đủ lớn, chống nhiễu tốt nhằm duy trì sự hoạt động chính xác của bộ xử lý AI (Raspberry Pi 4), vi điều khiển (STM32), các cảm biến và khối công suất động cơ.

Input: Điện áp DC từ khối Pin Lithium-Ion 3S (điện áp danh định 11.1V, sạc đầy lên đến 12.6V).

Output: Đường nguồn cấp trực tiếp từ Pin cấp cho khối công suất động cơ (MOTOR). Đường nguồn ổn áp +5V công suất cao phục vụ cho Raspberry Pi 4, vi điều khiển, màn hình LCD và các thiết bị ngoại vi khác.

Tín hiệu điện áp phản hồi V_{ref} đưa về chân ADC của vi điều khiển để giám sát dung lượng pin.

* Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng mạch ổn áp tuyến tính (Linear Regulator - họ L78xx)

Sử dụng các IC ổn áp tuyến tính như L7805 để hạ áp từ Pin 3S xuống +5V. Mạch cực kỳ đơn giản, chi phí thấp, tín hiệu đầu ra phẳng và ít nhiễu tần số cao. Tuy nhiên, hiệu suất cực kỳ thấp ($\approx 40\% - 45\%$). Do độ chênh lệch điện áp lớn ($12.6V \rightarrow 5V$) và dòng tiêu thụ của Raspberry Pi 4 rất cao (lên đến 3A), IC sẽ tỏa nhiệt cực kỳ lớn dẫn đến sụt áp, gây nguy hiểm cho hệ thống và làm lãng phí năng lượng pin.

Phương án 2: Sử dụng mạch ổn áp xung giảm áp tiêu chuẩn (Buck Regulator - LM2596)

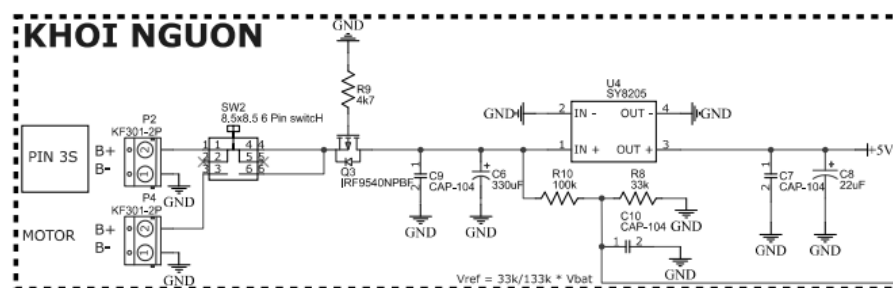
Sử dụng dòng IC nguồn xung phổ thông dòng LM2596-5.0. Hiệu suất năng lượng cải thiện rõ rệt ($\approx 75\% - 80\%$), ít tỏa nhiệt hơn mạch tuyến tính. Tuy nhiên, dòng ra tối đa của LM2596 chỉ đạt tối đa 3A trong điều kiện lý tưởng (thực tế dao động quanh 2A), không đủ độ dự phòng công suất (Headroom) an toàn khi xe tự hành chạy đồng thời cả camera, màn hình LCD và xử lý các mô hình AI nặng.

Phương án 3: Sử dụng IC nguồn xung chuyển mạch hiệu suất cao SY8205

Đặc điểm: Sử dụng IC Buck DC-DC chuyên dụng SY8205 hiệu suất cao, tần số băm xung lớn, hỗ trợ dòng xả liên tục lên đến 5A. Hiệu suất chuyển đổi năng lượng vượt trội (lên tới $> 90\%$), tỏa nhiệt rất thấp, kích thước mạch nhỏ gọn. Dòng đáp ứng lên tới 5A hoàn toàn đáp ứng công suất đỉnh cho Raspberry Pi 4 và các thiết bị ngoại vi mà không lo sụt áp.

Phương án lựa chọn: Nhóm quyết định lựa chọn Phương án 3 (Sử dụng IC nguồn xung SY8205 kết hợp mạch bảo vệ chống ngược nguồn bằng P-Channel MOSFET) để thiết kế khối nguồn.

* Thiết kế mạch và nguyên lý hoạt động



Hình 3.2 Sơ đồ nguyên lý của khối nguồn

Kết nối chính

Mạch ngõ vào và Công tắc: Nguồn Pin 3S kết nối qua Terminal P2. Công tắc gạt 6 chân SW2 (8.5x8.5 6 Pin switchH) đóng vai trò là khóa nguồn tổng. Khi bật công tắc, nguồn từ Pin 3S sẽ được phân nhánh: một nhánh cấp trực tiếp ra Terminal P4 để nuôi khối Động cơ (MOTOR), nhánh còn lại đi qua mạch bảo vệ chống ngược nguồn để cấp cho tầng ổn áp DC-DC.

Mạch chống ngược nguồn bằng MOSFET: Sử dụng P-Channel MOSFET Q3 (RF9540NPBF) kết hợp với điện trở R9 (4.7k) nối đất tại cực cổng (G).

Mạch ổn áp hạ áp DC-DC (Tầng hạ áp chính): Trái tim của khối là IC nguồn xung U4 (SY8205). Chân ngõ vào IN+ (chân 1) nhận điện áp sau MOSFET, chân IN- (chân 2) nối đất. Ngõ ra OUT+ (chân 3) cho ra điện áp +5V chuẩn, chân OUT- (chân 4) nối mass hệ thống.

Mạch phân áp giám sát Pin (Voltage Divider): Gồm hai điện trở chính R10 (100k Ω) và R8 (33k Ω) nối tiếp nhau từ đường nguồn sau công tắc xuống đất. Điểm giữa của cầu phân áp được tụ C10 lọc nhiễu và cho ra đường tín hiệu Vref kết nối trực tiếp về bộ chuyển đổi ADC của MCU STM32.

Các nhánh nguồn/tín hiệu và linh kiện lọc nhiễu

Nhánh nguồn trước ổn áp (V_{bat}): Được lọc bởi cặp tụ song song gồm tụ gốm C9 (CAP-104) dùng để lọc nhiễu cao tần và tụ hóa C6 (330uF) có dung lượng lớn giúp bù dòng tức thời và san phẳng điện áp nhấp nháy từ pin khi động cơ khởi động.

Nhánh nguồn sau ổn áp (+5V): Được san phẳng bằng tụ gốm C7 (0.1uF) chống nhiễu ký sinh và tụ hóa C8 (22uF) để ổn định điện áp ngõ ra cấp cho mạch logic.

Nguyên lý đo lường và giám sát dung lượng Pin (V_{ref})

Do dải điện áp của Pin 3S (9.0V $\rightarrow$ 12.6V) vượt quá dải đo cho phép của chân ADC trên vi điều khiển STM32 (tối đa 3.3V), mạch sử dụng cầu phân áp để hạ tỷ lệ điện áp tương ứng. Theo sơ đồ, điện áp phản hồi được tính toán dựa trên định luật Ohm theo công thức:

$$V_{ref} = \frac{R_8}{R_{10}+R_8} \times V_{bat} = \frac{33k\Omega}{100k\Omega+33k\Omega} \times V_{bat} = \frac{33}{133} \times V_{bat} \quad (3.1)$$

Khi Pin đầy (12.6V): $V_{ref} \approx \frac{33}{133} \times 12.6 \approx 3.12V$ (Nhằm trong dải an toàn < 3.3V của ADC).

Khi Pin cạn (9.6V): $V_{ref} \approx \frac{33}{133} \times 9.6 \approx 2.38V$.

Tín hiệu điện áp sau phân áp được dẫn qua tụ C10 ($0.1\mu F$) để triệt tiêu các nhiễu răng cưa tần số cao sinh ra từ nguồn xung, giúp tín hiệu analog đưa về chân MCU đạt độ mịn và độ chính xác cao nhất. Vi điều khiển sẽ liên tục lấy mẫu giá trị V_{ref} này để tính toán ngược lại dung lượng pin hiện tại của xe và hiển thị phần trăm pin lên màn hình LCD TFT của khối hiển thị. Khi V_{ref} giảm xuống dưới ngưỡng an toàn quy định (2.38V), MCU sẽ phát lệnh hạ chân STATUS báo lỗi đồng thời rú còi báo động yêu cầu sạc pin để bảo vệ xả sâu cho Cell pin Lithium.

3.2.2. Thiết kế khối xử lý trung tâm

* Chức năng của khối

Khối xử lý trung tâm đóng vai trò điều phối chính trong toàn bộ hệ thống xe tự hành. Đây là khối tiếp nhận dữ liệu từ các cảm biến, camera và các module ngoại vi, sau đó xử lý thông tin để đưa ra quyết định điều khiển phù hợp cho xe. Trong đề tài này, khối xử lý trung tâm được thiết kế theo kiến trúc xử lý phân tầng, bao gồm Raspberry Pi và STM32F103C8T6.

Trong kiến trúc này, Raspberry Pi đảm nhiệm các tác vụ xử lý cấp cao như thu nhận hình ảnh từ camera, xử lý ảnh, chạy các thuật toán AI/ADAS, nhận diện làn đường, phát hiện vật cản và đưa ra quyết định điều khiển tổng quát. Trong khi đó, STM32 đảm nhiệm các tác vụ điều khiển mức thấp như nhận lệnh từ Raspberry Pi, phát tín hiệu điều khiển động cơ/servo, đọc cảm biến khoảng cách, điều khiển còi, LED và các tín hiệu ngoại vi khác. Việc phân chia nhiệm vụ này giúp hệ thống vừa có khả năng xử lý thông minh, vừa đảm bảo phản hồi nhanh và ổn định trong điều khiển thời gian thực.

Input của khối: Dữ liệu hình ảnh từ camera, tín hiệu từ cảm biến, trạng thái hệ thống và các tín hiệu phản hồi từ ngoại vi.

Output của khối: Lệnh điều khiển tốc độ, góc lái, tín hiệu cảnh báo, tín hiệu điều khiển động cơ/servo và dữ liệu hiển thị trạng thái hệ thống.

* Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng một vi điều khiển duy nhất

Phương án này sử dụng STM32 hoặc Arduino để đảm nhiệm toàn bộ chức năng của hệ thống như đọc cảm biến, điều khiển động cơ và xử lý tín hiệu cơ bản. Giải pháp có ưu điểm là cấu trúc đơn giản, chi phí thấp và dễ triển khai. Tuy nhiên, khả năng tính toán hạn chế, không phù hợp cho các tác vụ xử lý ảnh và thuật toán AI trong hệ thống xe tự hành.

Phương án 2: Sử dụng Raspberry Pi làm bộ xử lý duy nhất

Raspberry Pi được sử dụng để thực hiện cả xử lý ảnh, AI và điều khiển ngoại vi. Phương án này có ưu điểm là năng lực tính toán cao, hỗ trợ tốt các thư viện như OpenCV và dễ tích hợp camera, màn hình hiển thị. Tuy nhiên, Raspberry Pi chạy hệ điều hành Linux nên khả năng xử lý thời gian thực chưa tối ưu, dễ xảy ra độ trễ khi điều khiển động cơ hoặc đọc cảm biến tốc độ cao.

Phương án 3: Kết hợp Raspberry Pi và STM32 kết hợp với TXS0180

Phương án này sử dụng Raspberry Pi làm bộ xử lý cấp cao và STM32 làm bộ điều khiển mức thấp. Raspberry Pi đảm nhiệm xử lý ảnh, AI và ra quyết định, trong khi STM32 thực hiện điều khiển động cơ, servo và giao tiếp cảm biến. Giải pháp này giúp tận dụng ưu điểm của cả hai nền tảng, tăng độ ổn định và khả năng mở rộng hệ thống, tuy nhiên cấu trúc phần cứng và giao tiếp sẽ phức tạp hơn. IC TXS0108 giúp MCU giao tiếp với cái driver điều khiển động cơ, bảo vệ MCU.

Phương án lựa chọn

Sau khi phân tích, nhóm lựa chọn phương án kết hợp Raspberry Pi và STM32F103C8T6 cho khối xử lý trung tâm. Raspberry Pi được sử dụng để xử lý ảnh, nhận diện làn đường, biển báo và vật cản, trong khi STM32 đảm nhiệm điều khiển

Giải pháp này giúp hệ thống cân bằng giữa khả năng xử lý thông minh và khả năng điều khiển ổn định. Đồng thời, việc phân chia nhiệm vụ giữa Raspberry Pi và STM32 giúp giảm tải cho bộ xử lý chính, tăng độ tin cậy khi vận hành và phù hợp với yêu cầu của mô hình xe tự hành ADAS.

Khối xử lý trung tâm được thiết kế dựa trên sự phối hợp giữa Raspberry Pi và STM32. Trong đó, Raspberry Pi đóng vai trò là bộ xử lý cấp cao, còn STM32 đóng vai trò là bộ điều khiển ngoại vi mức thấp. Hai bộ xử lý này được kết nối với nhau thông qua giao tiếp UART, cho phép Raspberry Pi gửi lệnh điều khiển xuống STM32 và STM32 có thể phản hồi trạng thái hệ thống khi cần.

Tham chiếu hình:

Cấu trúc kết nối của Raspberry Pi và STM32 được thể hiện trong Hình 3.6. Sơ đồ nguyên lý khối xử lý trung tâm.

Kết nối chính

Các kết nối chính trong khối xử lý trung tâm bao gồm:

Raspberry Pi kết nối với camera để thu nhận dữ liệu hình ảnh.

Raspberry Pi kết nối với khối hiển thị để hiển thị giao diện hoặc trạng thái hệ thống.

Raspberry Pi giao tiếp với STM32 thông qua UART.

STM32 kết nối với cảm biến khoảng cách, còi, LED, driver động cơ và servo.

Cả Raspberry Pi và STM32 dùng chung nguồn 5V hoặc nguồn chuyển đổi phù hợp, đồng thời có chung điểm GND để đảm bảo tín hiệu điều khiển ổn định.

3.2.3. Thiết kế khối động cơ

* Chức năng của khối

Khối động cơ là khối chấp hành chính của mô hình xe tự hành, có nhiệm vụ chuyển đổi các lệnh điều khiển từ khối xử lý trung tâm thành chuyển động thực tế của xe. Khối này đảm nhiệm hai chức năng quan trọng: tạo chuyển động tịnh tiến thông qua động cơ DC truyền động và điều khiển hướng di chuyển thông qua servo lái.

Trong hệ thống, khối động cơ nhận các tín hiệu điều khiển từ Raspberry Pi/STM32, bao gồm lệnh tốc độ, chiều quay và góc lái mong muốn. Sau đó, các tín hiệu này được đưa đến driver động cơ và servo để điều khiển xe di chuyển theo yêu cầu của thuật toán ADAS như giữ làn, tránh vật cản hoặc dừng khi phát hiện tình huống nguy hiểm.

Input: tín hiệu PWM điều khiển tốc độ, tín hiệu điều khiển chiều quay, tín hiệu PWM điều khiển góc lái, nguồn công suất cấp cho động cơ.

Output: chuyển động quay của động cơ DC, góc quay của servo lái và tín hiệu phản hồi từ encoder nếu có sử dụng điều khiển kín.

* Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng module L298N điều khiển động cơ DC

Phương án này sử dụng driver cầu H L298N để điều khiển tốc độ và chiều quay động cơ DC bằng tín hiệu PWM từ vi điều khiển. Ưu điểm dễ sử dụng và phổ biến, chi phí

thấp, phù hợp với mô hình nhỏ và tải nhẹ. Tuy nhiên module này không đem lại hiệu suất cao, dễ nóng khi tải lớn và khả năng chịu dòng hạn chế.

Phương án 2: Sử dụng module TB6612FNG điều khiển động cơ DC

Phương án này sử dụng driver TB6612FNG có kích thước nhỏ gọn và hiệu suất cao hơn L298N. Hiệu suất module tốt hơn L298N, ít tỏa nhiệt, điều khiển PWM ổn định. Nhược điểm là chịu dòng thấp, không phù hợp với động cơ tải lớn, hạn chế khi xe tăng tốc hoặc tải nặng.

Phương án 3: Sử dụng BTS7960 kết hợp servo và encoder

Phương án này sử dụng BTS7960 để điều khiển động cơ truyền động, kết hợp servo điều khiển lái và encoder phản hồi tốc độ. Ưu điểm Chịu dòng lớn và hoạt động ổn định, Điều khiển tốc độ và hướng lái chính xác, Hỗ trợ phản hồi tốc độ bằng encoder, Phù hợp với hệ thống xe tự hành. Nhược điểm mạch điều khiển phức tạp hơn, Cần nguồn công suất riêng, Cần xử lý nhiễu động cơ.

Phương án lựa chọn

Phương án được lựa chọn là sử dụng BTS7960 kết hợp động cơ DC, servo lái, encoder và mạch chuyển mức TXS0108E.

Linh kiện sử dụng

Module BTS7960.

Động cơ DC truyền động.

Servo điều khiển lái.

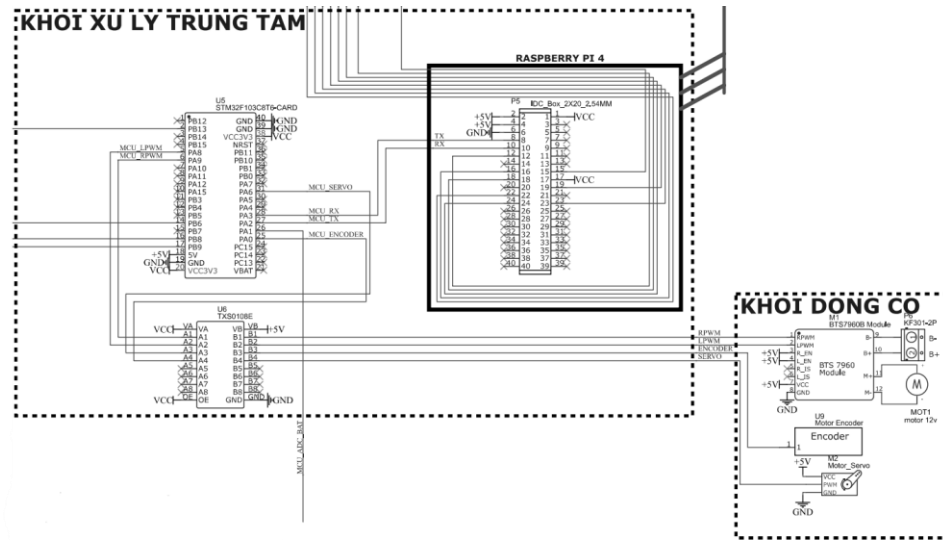
Encoder phản hồi tốc độ.

Lý do chọn BTS7960 phù hợp với động cơ công suất lớn của mô hình xe tự hành nhờ khả năng chịu dòng cao và điều khiển ổn định hơn L298N hoặc TB6612FNG. Servo giúp điều khiển góc lái chính xác bằng PWM, phù hợp với bài toán giữ làn và điều hướng.

Encoder cung cấp tín hiệu phản hồi tốc độ giúp hệ thống điều khiển ổn định hơn.

Vì vậy, phương án sử dụng BTS7960 kết hợp servo và encoder đáp ứng tốt yêu cầu điều khiển chuyển động của mô hình xe tự hành.

* Thiết kế và nguyên lý hoạt động



Hình 3.4 Sơ đồ nguyên lý của khối động cơ

Khối động cơ được thiết kế gồm hai phần chính: phần truyền động và phần điều hướng. Phần truyền động sử dụng động cơ DC kết hợp với driver BTS7960 để điều khiển tốc độ và chiều quay của xe. Phần điều hướng sử dụng servo để thay đổi góc lái. Ngoài ra, encoder được gắn với động cơ để cung cấp tín hiệu phản hồi phục vụ việc đo tốc độ hoặc điều khiển kín.

Kết nối chính

Các kết nối chính của khối động cơ bao gồm:

BTS7960 nhận nguồn công suất từ khối nguồn để cấp cho động cơ DC.

BTS7960 nhận tín hiệu điều khiển từ khối xử lý trung tâm thông qua các đường PWM và enable.

Động cơ DC được nối với ngõ ra công suất của BTS7960.

Servo nhận nguồn 5V, GND và tín hiệu PWM điều khiển góc lái.

Encoder gắn với động cơ tạo tín hiệu phản hồi đưa về khối xử lý.

Toàn bộ hệ thống sử dụng chung GND để bảo đảm tín hiệu điều khiển có cùng mức tham chiếu.

3.2.4. Thiết kế khối cảm biến

* Chức năng của khối

Khối cảm biến có vai trò thu thập thông tin khoảng cách giữa xe và vật cản phía trước, từ đó cung cấp dữ liệu đầu vào cho khối xử lý trung tâm để hỗ trợ các chức năng an toàn trong hệ thống xe tự hành. Trong đề tài này, khối cảm biến sử dụng cảm biến khoảng cách VL53L0X, kết nối với vi điều khiển STM32F103C8T6 để đo khoảng cách và phát hiện vật cản trong phạm vi hoạt động của xe.

Nhiệm vụ chính của khối cảm biến là đo khoảng cách đến vật thể phía trước, gửi dữ liệu đo được về STM32, hỗ trợ thuật toán phát hiện vật cản và đưa ra các quyết định như giảm tốc, dừng xe hoặc cảnh báo. Dữ liệu đầu vào của khối là môi trường vật lý phía trước xe, cụ thể là khoảng cách đến vật cản. Dữ liệu đầu ra là giá trị khoảng cách đã đo, được truyền về khối xử lý trung tâm thông qua giao tiếp số.

Input: khoảng cách thực tế từ xe đến vật cản phía trước.

Output: dữ liệu khoảng cách dạng số gửi về STM32 để xử lý.

Vai trò: hỗ trợ phát hiện vật cản và tăng độ an toàn khi xe vận hành.

* Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng cảm biến siêu âm HC-SR04

Phương án này sử dụng cảm biến siêu âm HC-SR04 để đo khoảng cách bằng cách phát sóng siêu âm và đo thời gian phản hồi của sóng sau khi gặp vật cản. Đây là loại cảm biến khoảng cách phổ biến trong các mô hình robot và xe tự hành đơn giản. Có ưu điểm giá thành thấp, dễ mua, dễ sử dụng, nguyên lý hoạt động đơn giản. Có thể kết nối trực tiếp với vi điều khiển thông qua chân trigger và echo, phù hợp với các bài toán phát hiện vật cản cơ bản. Tuy nhiên module này có kích thước tương đối lớn so với một số cảm biến quang học. Góc phát sóng rộng, độ chính xác bị ảnh hưởng bởi hình dạng và bề mặt vật cản. Khó đo chính xác với vật nhỏ, bề mặt nghiêng hoặc vật liệu hấp thụ âm. Tốc độ phản hồi và độ ổn định có thể không phù hợp với các tình huống cần đo nhanh. Cần xử lý thời gian xung echo, có thể chiếm tài nguyên timer của vi điều khiển.

Phương án 2: Sử dụng cảm biến hồng ngoại đo khoảng cách

Phương án này sử dụng cảm biến hồng ngoại để phát hiện vật cản hoặc ước lượng khoảng cách dựa trên cường độ ánh sáng phản xạ. Một số cảm biến hồng ngoại có đầu ra analog hoặc digital, dễ tích hợp với vi điều khiển. Có ưu điểm kích thước nhỏ gọn, Chi phí thấp, Dễ kết nối với vi điều khiển, Tốc độ phản hồi nhanh, Phù hợp cho phát hiện vật cản ở khoảng cách gần. Tuy nhiên, Độ chính xác phụ thuộc nhiều vào màu sắc và bề mặt vật cản, Dễ bị ảnh hưởng bởi ánh sáng môi trường, Khoảng đo thường ngắn, Một số loại chỉ cho biết có/không có vật cản, không cung cấp khoảng cách chính xác, Không phù hợp nếu hệ thống cần giá trị khoảng cách ổn định để điều khiển.

Phương án 3: Sử dụng cảm biến ToF VL53L0X

Phương án này sử dụng cảm biến VL53L0X, là cảm biến đo khoảng cách theo nguyên lý Time-of-Flight. Cảm biến phát tia laser hồng ngoại và đo thời gian phản xạ để tính khoảng cách đến vật thể. Module này giao tiếp với vi điều khiển thông qua chuẩn I2C. Có ưu điểm kích thước nhỏ gọn, dễ tích hợp lên mô hình xe. Cho dữ liệu khoảng cách dạng số. Giao tiếp I2C đơn giản, chỉ cần hai đường tín hiệu chính SDA và SCL. Độ chính xác tốt hơn so với cảm biến hồng ngoại đơn giản. Ít phụ thuộc hơn vào cường độ phản xạ so với một số cảm biến IR cơ bản. Phù hợp với chức năng phát hiện vật cản phía trước trong mô hình xe tự hành. Tuy nhiên Giá thành cao hơn cảm biến siêu âm hoặc cảm biến hồng ngoại đơn giản. Khoảng đo có giới hạn, không phù hợp cho phát hiện vật ở khoảng cách quá xa. Cần cấu hình phần mềm để khởi tạo cảm biến và đọc dữ liệu qua I2C. Nếu dùng nhiều cảm biến VL53L0X trên cùng bus I2C, cần quản lý địa chỉ hoặc chân XSHUT.

Phương án lựa chọn

Phương án được lựa chọn là sử dụng cảm biến ToF VL53L0X kết nối với STM32F103C8T6 thông qua giao tiếp I2C.

Các linh kiện chính trong khối cảm biến bao gồm:

Cảm biến khoảng cách VL53L0X.

Vi điều khiển STM32F103C8T6 trong khối xử lý trung tâm.

Nguồn cấp 3.3V hoặc 5V tùy theo module VL53L0X sử dụng.

Đường giao tiếp I2C gồm SDA và SCL.

Các chân phụ như XSHUT và GPIO1 nếu cần điều khiển bật/tắt cảm biến hoặc nhận tín hiệu ngắt.

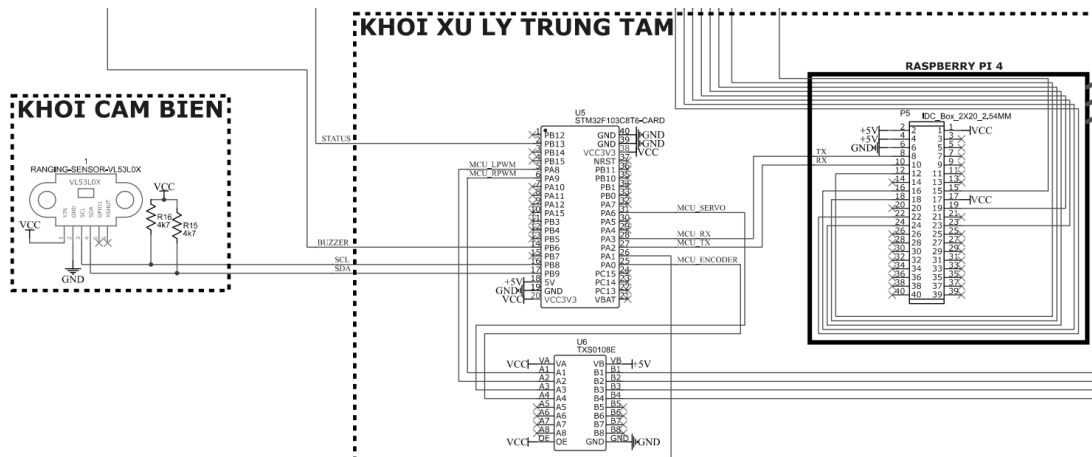
Cảm biến VL53L0X được lựa chọn vì phù hợp với yêu cầu phát hiện vật cản của mô hình xe tự hành. So với cảm biến siêu âm, VL53L0X có kích thước nhỏ gọn hơn, dễ bố trí ở phía trước xe và ít bị ảnh hưởng bởi hình dạng cơ khí của module. So với cảm biến hồng ngoại đơn giản, VL53L0X cung cấp dữ liệu khoảng cách dạng số ổn định hơn, thuận lợi cho việc xử lý trong vi điều khiển.

Về khả năng tích hợp, VL53L0X sử dụng chuẩn giao tiếp I2C nên chỉ cần hai đường tín hiệu chính để truyền dữ liệu. Điều này giúp giảm số lượng chân kết nối giữa cảm biến và STM32, đồng thời phù hợp với kiến trúc hệ thống nhúng có nhiều ngoại vi. Trong sơ đồ nguyên lý, cảm biến được đặt trong khối cảm biến và kết nối trực tiếp với STM32F103C8T6 thuộc khối xử lý trung tâm, giúp STM32 có thể đọc dữ liệu khoảng cách và xử lý phản ứng nhanh ở mức điều khiển thấp.

Về hiệu năng, VL53L0X đủ đáp ứng nhu cầu phát hiện vật cản trong mô hình xe tự hành ở phạm vi gần. Dữ liệu đo được có thể được sử dụng để cảnh báo, giảm tốc hoặc dừng xe khi phát hiện vật cản phía trước. Ngoài ra, cảm biến này có kích thước nhỏ, tiêu thụ điện năng thấp và dễ lập trình với các thư viện hỗ trợ sẵn, phù hợp với yêu cầu nghiên cứu và thử nghiệm của đề tài.

Do đó, phương án sử dụng VL53L0X + STM32F103C8T6 + giao tiếp I2C là lựa chọn phù hợp cho khối cảm biến của hệ thống.

* Thiết kế và nguyên lý hoạt động



Hình 3.5 Sơ đồ nguyên lý của khối cảm biến

Khối cảm biến được thiết kế để kết nối trực tiếp với vi điều khiển STM32F103C8T6. Cảm biến VL53L0X được bố trí ở phía trước xe nhằm đo khoảng cách đến vật cản trong hướng di chuyển. Dữ liệu đo được được gửi về STM32 thông qua giao tiếp I2C để phục vụ xử lý và ra quyết định điều khiển.

Kết nối chính

Dựa trên sơ đồ nguyên lý, module VL53L0X có các nhóm kết nối chính sau:

Nguồn cấp: chân VIN của cảm biến được cấp nguồn từ hệ thống, chân GND nối mass chung.

Giao tiếp dữ liệu: hai đường SDA và SCL của cảm biến được kết nối với các chân giao tiếp I2C của STM32.

Tín hiệu điều khiển phụ: chân XSHUT có thể được dùng để bật/tắt hoặc reset cảm biến bằng phần mềm.

Tín hiệu ngắt: chân GPIO1 có thể được dùng làm tín hiệu báo ngắt khi có dữ liệu đo hoặc sự kiện khoảng cách, nếu hệ thống có sử dụng.

Trong thiết kế này, STM32 đóng vai trò là bộ điều khiển đọc dữ liệu cảm biến. Cảm biến VL53L0X đóng vai trò là thiết bị ngoại vi cung cấp dữ liệu khoảng cách cho khối xử lý trung tâm.

3.2.5. Thiết kế khối camera

* Chức năng của khối

Khối camera có vai trò thu nhận hình ảnh môi trường phía trước xe, cung cấp dữ liệu đầu vào cho các thuật toán xử lý ảnh và trí tuệ nhân tạo trong hệ thống ADAS. Đây là nguồn dữ liệu quan trọng giúp xe nhận biết làn đường, biển báo giao thông, đèn tín hiệu hoặc vật cản trong quá trình vận hành.

Trong đề tài này, khối camera sử dụng Pi Camera module kết nối trực tiếp với Raspberry Pi. Dữ liệu hình ảnh từ camera được Raspberry Pi xử lý để thực hiện các chức năng như giữ làn đường, nhận diện biển báo, phát hiện vật cản và hỗ trợ đưa ra quyết định điều khiển.

Input: hình ảnh thực tế từ môi trường phía trước xe.

Output: luồng hình ảnh hoặc khung hình được truyền đến Raspberry Pi để xử lý.

* Các phương án thiết kế và lựa chọn linh kiện

Phương án 1: Sử dụng cảm biến đơn giản thay cho camera

Phương án này sử dụng cảm biến hồng ngoại, siêu âm hoặc line tracking để phát hiện vật cản và vạch đường cơ bản. Giải pháp có ưu điểm là chi phí thấp, dễ kết nối và xử lý đơn giản. Tuy nhiên, hệ thống không thể thu nhận hình ảnh, nhận diện biển báo hay xử lý các tình huống giao thông phức tạp, nên không phù hợp với yêu cầu ADAS.

Phương án 2: Sử dụng USB Camera

Camera USB được kết nối với Raspberry Pi thông qua cổng USB để thu nhận hình ảnh phục vụ xử lý bằng OpenCV. Phương án này dễ triển khai và phù hợp cho các bài toán xử lý ảnh cơ bản. Tuy nhiên, camera USB thường chiếm cổng kết nối, kích thước lớn hơn và độ ổn định phụ thuộc vào từng loại camera.

Phương án 3: Sử dụng Pi Camera module

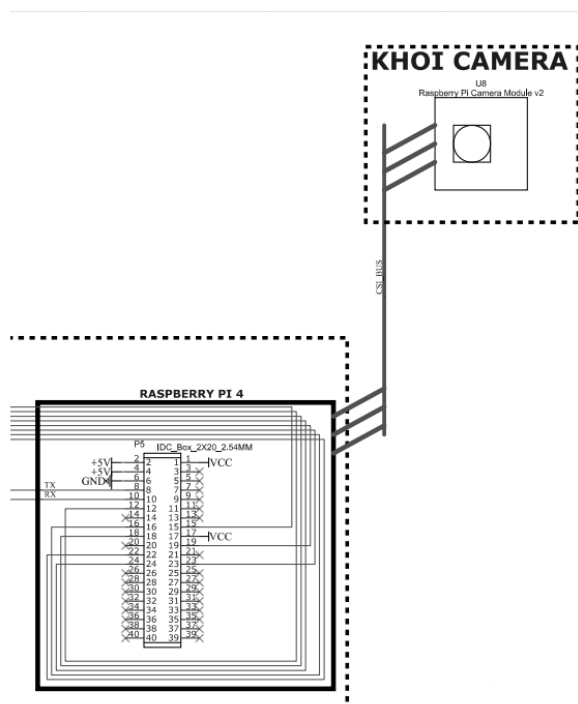
Phương án này sử dụng Pi Camera module kết nối trực tiếp với Raspberry Pi qua cổng CSI chuyên dụng. Camera có khả năng truyền dữ liệu ổn định, độ trễ thấp và phù hợp cho các ứng dụng xử lý ảnh nhúng. Ngoài ra, kết nối bằng cáp dẹt giúp hệ thống gọn gàng và dễ tích hợp trên mô hình xe.

Phương án lựa chọn

Sau khi phân tích, nhóm lựa chọn sử dụng Pi Camera module cho khối camera của hệ thống. Camera được kết nối trực tiếp với Raspberry Pi thông qua cổng CSI và cáp dẹt chuyên dụng.

Giải pháp này đáp ứng tốt yêu cầu xử lý ảnh của hệ thống ADAS như phát hiện làn đường, nhận diện biển báo giao thông và hỗ trợ phát hiện vật cản. Đồng thời, Pi Camera có khả năng tích hợp tốt với Raspberry Pi, độ ổn định cao, kích thước nhỏ gọn và được hỗ trợ mạnh bởi các thư viện như OpenCV và Picamera2.

* Thiết kế và nguyên lý hoạt động



Hình 3.6 Sơ đồ nguyên lý của khối camera

Khối camera được thiết kế để kết nối trực tiếp với Raspberry Pi thông qua cổng camera chuyên dụng. Camera được đặt ở phía trước mô hình xe nhằm thu nhận hình ảnh môi trường phía trước, phục vụ cho các thuật toán ADAS như giữ làn đường, nhận diện biển báo, phát hiện đèn giao thông hoặc vật cản.

Tham chiếu hình: Kết nối của khối camera được thể hiện trong Sơ đồ nguyên lý khối camera.

Kết nối chính

Kết nối chính của khối camera gồm:

Pi Camera module.

Cáp dẹt camera.

Cổng camera trên Raspberry Pi.

Nguồn và tín hiệu điều khiển được hỗ trợ thông qua giao diện camera của Raspberry Pi.

Cơ cấu gá đặt camera trên thân xe để đảm bảo góc nhìn ổn định.

Camera cần được cố định chắc chắn để hạn chế rung lắc trong quá trình xe di chuyển, vì rung động có thể làm giảm chất lượng ảnh và ảnh hưởng đến kết quả xử lý thuật toán.

3.2.6. Thiết kế khối hiển thị

*** Chức năng của khối**

Khối hiển thị và cảnh báo có vai trò trực quan hóa thông tin vận hành và phát tín hiệu cảnh báo chủ động cho hệ thống xe tự hành. Thông qua khối này, người dùng có thể giám sát thời gian thực các trạng thái hoạt động của xe trong quá trình thử nghiệm, bao gồm chế độ hành trình, kết quả xử lý ảnh từ thuật toán ADAS, trạng thái nhận diện làn đường/đèn giao thông, cũng như nhận biết các trạng thái hệ thống của vi điều khiển (MCU) và các nguy cơ va chạm thông qua tín hiệu âm thanh/ánh sáng.

Input: Dữ liệu hình ảnh, trạng thái AI, kết quả thuật toán từ Raspberry Pi 4; dữ liệu cảm biến khoảng cách, tín hiệu đo lường xung nhịp hệ thống nội bộ từ vi điều khiển STM32F103C8T6.

Output: Giao diện đồ họa trực quan trên màn hình LCD TFT; trạng thái nhấp của LED STATUS báo hiệu tình trạng MCU; âm thanh cảnh báo từ còi chip (Buzzer) khi phát hiện vật cản nguy hiểm.

*** Các phương án thiết kế và lựa chọn linh kiện**

Phương án 1: Sử dụng LED báo trạng thái

Phương án này sử dụng LED đơn hoặc LED RGB để hiển thị các trạng thái cơ bản của hệ thống như nguồn, lỗi hoặc cảnh báo. Giải pháp có ưu điểm là mạch đơn giản,

chi phí thấp và dễ điều khiển. Tuy nhiên, khả năng hiển thị thông tin hạn chế và không phù hợp cho việc quan sát dữ liệu debug hoặc kết quả xử lý ảnh.

Phương án 2: Sử dụng màn hình OLED giao tiếp I2C

Màn hình OLED kích thước nhỏ được sử dụng để hiển thị văn bản hoặc thông tin trạng thái. Phương án này có ưu điểm là nhỏ gọn, tiêu thụ điện năng thấp và giao tiếp đơn giản qua I2C. Tuy nhiên, kích thước hiển thị nhỏ và không phù hợp để hiển thị hình ảnh hoặc giao diện trực quan của hệ thống ADAS.

Phương án 3: Sử dụng màn hình LCD TFT 2.4 inch giao tiếp SPI kết hợp với Transistor kích Led và Còi

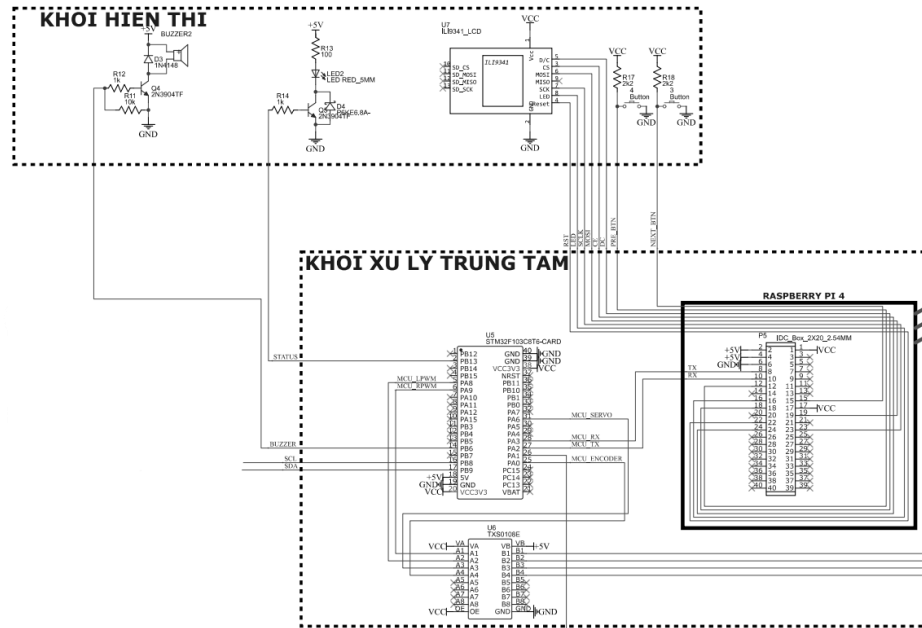
Phương án này sử dụng màn hình LCD TFT màu 2.4 inch kết nối với Raspberry Pi qua giao tiếp SPI. Màn hình có khả năng hiển thị giao diện trạng thái, dữ liệu debug và hình ảnh xử lý từ hệ thống, đồng thời bố trí các mạch còi báo chủ động (Active Buzzer) và LED trạng thái thứ cấp điều khiển độc lập bởi MCU qua transistor công suất. Giải pháp này phù hợp với các ứng dụng nhúng cần hiển thị trực quan, tuy nhiên yêu cầu cấu hình driver và sử dụng nhiều chân giao tiếp hơn. Hệ thống cảnh báo bằng âm thanh và ánh sáng hoạt động độc lập giúp giảm tải cho bộ xử lý trung tâm và tăng độ tin cậy.

Phương án lựa chọn

Sau khi phân tích, nhóm lựa chọn màn hình LCD TFT 2.4 inch giao tiếp SPI kết hợp với Transistor kích Led và Còi cho khối hiển thị. Màn hình sử dụng IC điều khiển ILI9341 hoặc tương đương và giao tiếp trực tiếp với Raspberry Pi thông qua SPI.

Giải pháp này đáp ứng tốt yêu cầu hiển thị của hệ thống ADAS như trạng thái vận hành, cảnh báo vật cản, thông tin điều khiển và dữ liệu debug. Ngoài ra, màn hình có kích thước phù hợp với mô hình xe tự hành, dễ tích hợp, chi phí hợp lý và được hỗ trợ bởi nhiều thư viện trên Raspberry Pi.

* Thiết kế và nguyên lý hoạt động



Hình 3.7 Sơ đồ nguyên lý của khối hiển thị

Khối hiển thị được kết nối trực tiếp với Raspberry Pi thông qua giao tiếp SPI. Raspberry Pi đóng vai trò là bộ xử lý tạo dữ liệu hiển thị, trong khi LCD TFT đóng vai trò thiết bị đầu ra trực quan. Nội dung hiển thị là giao diện trạng thái, thông tin debug, cảnh báo hoặc hình ảnh đã được xử lý.

Tham chiếu hình:

Kết nối của khối hiển thị được thể hiện trong Sơ đồ nguyên lý khối hiển thị.

Kết nối chính

Các kết nối chính giữa Raspberry Pi và LCD TFT gồm:

Đường nguồn cấp cho màn hình.

Đường GND dùng chung với hệ thống.

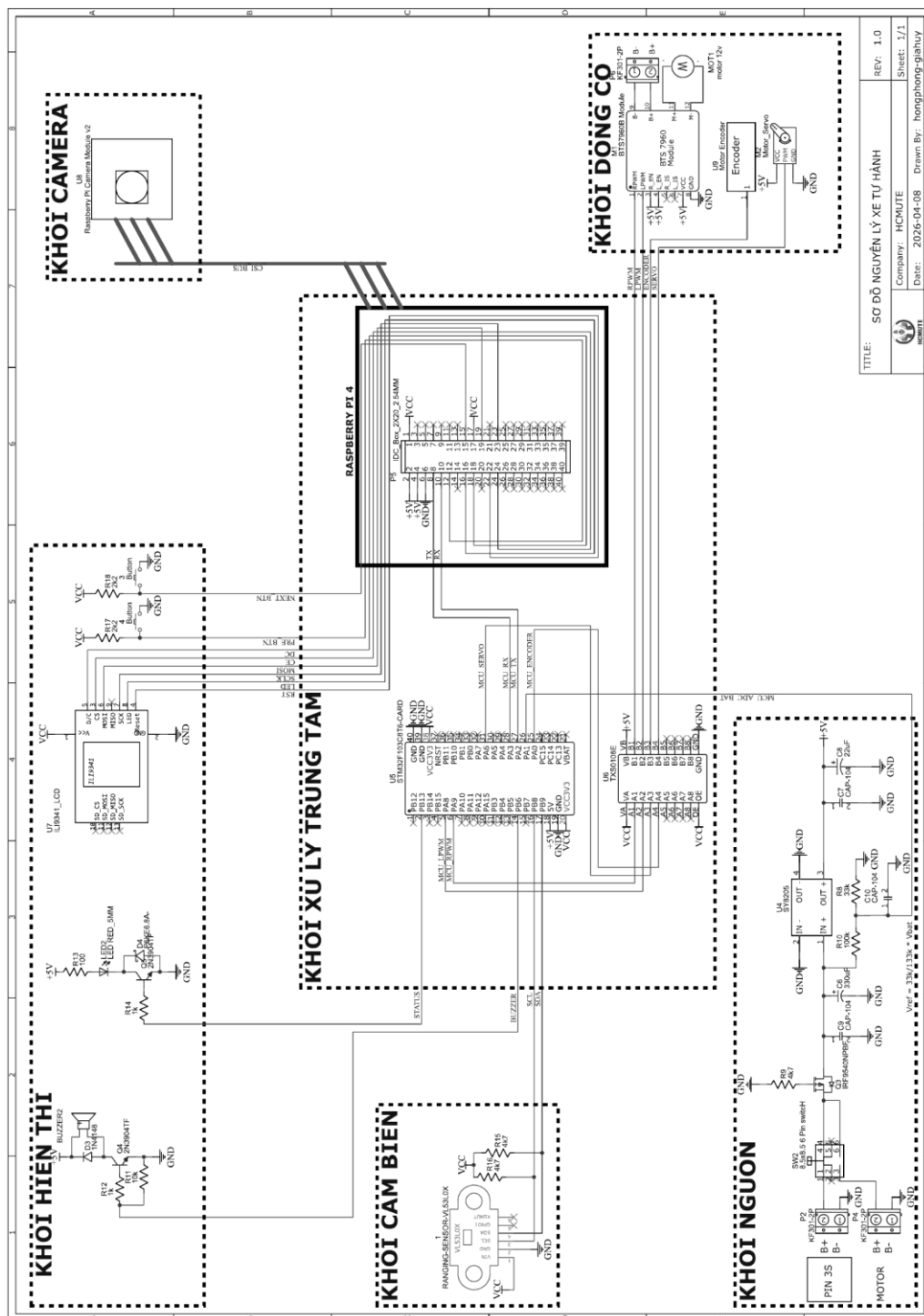
Đường SPI gồm xung clock và dữ liệu từ Raspberry Pi đến LCD.

Các chân điều khiển như chọn chip, reset màn hình, chọn dữ liệu/lệnh và điều khiển đèn nền.

Trong thiết kế này, LCD chủ yếu nhận dữ liệu từ Raspberry Pi để hiển thị, do đó đường đọc ngược từ LCD về Raspberry Pi không phải là yêu cầu bắt buộc.

3.2.7. Sơ đồ nguyên lý toàn hệ thống

Sau khi hoàn tất quá trình thiết kế và kiểm tra hoạt động của hệ thống thì sơ đồ nguyên lý được hoàn thiện như trong hình sau đây:



Hình 3.8 Sơ đồ nguyên lý hệ thống

3.3. LỰA CHỌN VÀ HUẤN LUYỆN MODULE AI DỰ ĐOÁN

3.3.1. Lựa chọn mô hình

Khởi nhận diện là phần quan trọng nhất của hệ thống ADAS (Advanced Driver Assistance System), chịu trách nhiệm phân tích luồng video từ camera theo thời gian thực và trả về tọa độ cũng như trạng thái của đèn giao thông. Do hệ thống được triển khai trên máy tính nhúng Raspberry Pi với tài nguyên tính toán (CPU/RAM) hạn chế, việc lựa chọn mô hình đòi hỏi sự cân bằng khắt khe giữa tốc độ và độ chính xác. Dưới đây là các phương án được cân nhắc:

Phương án 1: YOLO-FastestV2 (Mô hình học sâu siêu nhẹ cho thiết bị Edge) [11]

Phiên bản tối ưu hóa cực hạn của họ YOLO, thiết kế đặc biệt để chạy trên các vi xử lý ARM không có GPU.

Ưu điểm: Tốc độ suy luận cực nhanh, tiêu tốn rất ít RAM và CPU. Kích thước file trọng số siêu nhỏ (< 1MB). Dễ dàng xuất sang định dạng ONNX để chạy trực tiếp bằng thư viện cv::dnn của OpenCV bằng C++.

Nhược điểm: Cấu trúc mạng bị cắt giảm nhiều (chỉ còn 2 scale output) nên cần tinh chỉnh bộ Anchors (khung neo) kỹ lưỡng để bù đắp độ chính xác.

Phương án 2: YOLOv8n / YOLOv11n (Mô hình SOTA độ chính xác cao)

Các phiên bản YOLO mới nhất mang lại hiệu năng phát hiện vật thể xuất sắc trên các tập dữ liệu lớn.

Ưu điểm: Độ chính xác (mAP) rất cao, nhận diện tốt các vật thể nhỏ ở khoảng cách xa, cấu trúc hiện đại.

Nhược điểm: Quá nặng đối với CPU của Raspberry Pi (chỉ đạt khoảng 2–5 FPS), gây trễ tín hiệu điều khiển. Yêu cầu bộ nhớ lớn và môi trường triển khai phức tạp hơn.

Phương án 3: Haar Cascades / HOG + SVM (Thị giác máy tính truyền thống)

Phương pháp phát hiện đối tượng dựa trên đặc trưng hình học truyền thống của OpenCV.

Ưu điểm: Không cần GPU để huấn luyện, tốc độ xử lý trên CPU rất nhanh, cực kỳ nhẹ.

Nhược điểm: Độ chính xác rất thấp khi điều kiện ánh sáng thay đổi (chói nắng, ban đêm) hoặc góc nhìn nghiêng. Tạo ra quá nhiều nhiễu (False Positives) trong môi trường sa bàn phức tạp.

Phương án được chọn: YOLO-FastestV2

Tốc độ suy luận nhanh nhất, đáp ứng yêu cầu Real-time cho xe tự hành trên phần cứng Raspberry Pi OS.

Độ chính xác đủ tốt sau khi áp dụng thuật toán K-Means để gom cụm bộ Anchors tối ưu riêng cho kích thước đèn giao thông.

Dễ dàng xuất sang định dạng ONNX và tích hợp trực tiếp vào mã nguồn C++ qua cv::dnn, loại bỏ hoàn toàn sự phụ thuộc vào các thư viện nặng nề như PyTorch trên thiết bị nhúng [12].

Bảng 3.1 . So sánh 3 mô hình YOLO-FastestV2, YOLOv11n, Haar Cascades

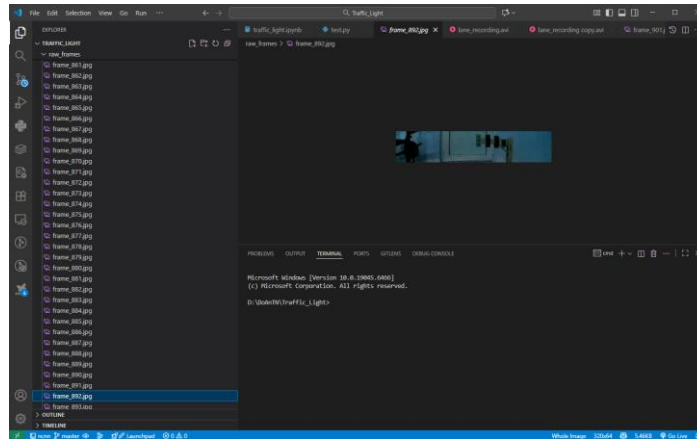
STT	Tiêu chí	YOLO-FastestV2 (Được chọn)	YOLOv11n	Haar Cascades
1	Loại mô hình	Deep Learning (Edge)	Deep Learning (SOTA)	Computer Vision truyền thống
2	Tốc độ suy luận (CPU Pi)	Tốt (~20-30 FPS)	Kém (~2-5 FPS)	Rất nhanh (>30 FPS)
3	Độ chính xác (mAP/Acc)	Khá (>85% sau tinh chỉnh)	Rất cao (>90%)	Thấp (<60% ngoài trời)
4	Kích thước mô hình	< 1MB	~6MB	Vài chục KB
5	Giao thức triển khai	OpenCV DNN (ONNX)	PyTorch / TFLite	OpenCV thuần
6	Độ ổn định môi trường	Cao (Kháng nhiễu tốt)	Cao	Thấp (Dễ bị nhiễu sáng)

3.3.2. Quá trình huấn luyện mô hình YOLO-FastestV2

Thu thập dữ liệu:

Dữ liệu được thu thập từ hình ảnh camera hành trình mô phỏng trên sa bàn và các tập dữ liệu mở.

Tập trung cắt vùng quan tâm (ROI) cố định với tỷ lệ 320x64 (kích thước siêu dẹt) để tối ưu hóa trường nhìn của xe, loại bỏ bầu trời và mặt đường không cần thiết.

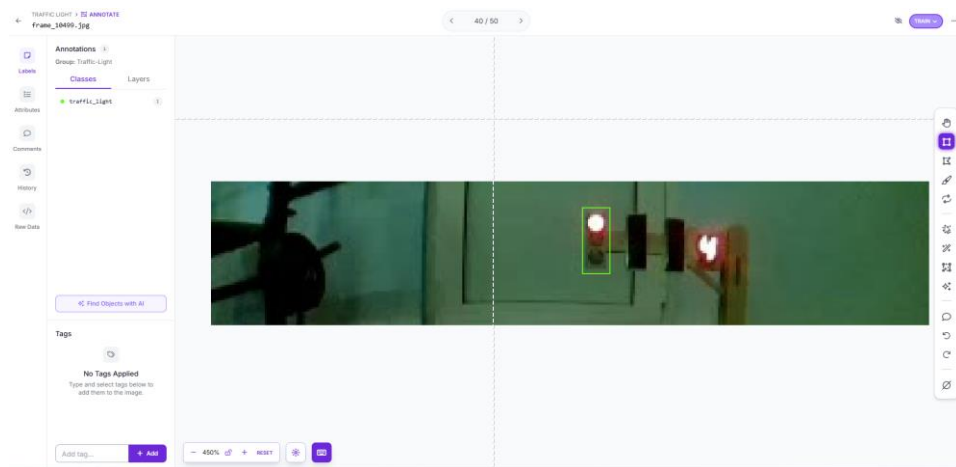


Hình 3.9 Cắt ảnh đúng kích thước 320x64

Gán nhãn:

Sử dụng công cụ LabelImg / Roboflow để gán nhãn bounding box.

Hệ thống sử dụng chiến lược gom cụm: Chỉ gán duy nhất 1 lớp (class) là "traffic_light". Trọng trách phân biệt màu (Xanh/Đỏ/Vàng) được đẩy sang module xử lý hình thái học (HSV Masking) sau khi cắt được hộp ROI, giúp tăng tốc độ mạng nơ-ron lên mức tối đa.



Hình 3.10 Dán nhãn cho ảnh

Tăng cường dữ liệu (Data Augmentation):

Resize: Chuẩn hóa đầu vào về dạng lưới Tensor tương thích với mô hình.

Exposure: Điều chỉnh độ sáng (-20% đến +20%) để mô phỏng điều kiện môi trường ánh sáng thay đổi.

Blur: Thêm hiệu ứng mờ chuyển động (Motion Blur) lên đến 2px để mô phỏng rung lắc khi xe tự hành di chuyển.

Noise: Thêm nhiễu ngẫu nhiên để tăng tính khái quát cho mô hình.

Huấn luyện và tinh chỉnh:

Môi trường: WSL Ubuntu 22.04.

Framework: PyTorch (Dựa trên repository YOLO-FastestV2 của tác giả dog-qiuqiu).

Tinh chỉnh (Fine-tuning): Chạy thuật toán K-Means trên tập dữ liệu để tìm ra 6 Anchors phù hợp nhất với hình dáng của cột đèn (Stride 16 và Stride 32), thay thế hoàn toàn bộ Anchors mặc định của tập COCO.

Tham số: epochs = 300, batch-size = 32, kết thúc sớm (early stopping) nếu Loss hội tụ.

Code thực thi huấn luyện và trích xuất:

```
# 1. Tải mã nguồn YOLO-FastestV2
!git clone https://github.com/dog-qiuqiu/Yolo-FastestV2.git
%cd Yolo-FastestV2

# 2. Cài đặt các thư viện phụ thuộc
!pip install -r requirements.txt
!pip install onnx onnx-simplifier
Python
# 3. Chuẩn bị file cấu hình dữ liệu (traffic_light.data)
import os

data_config = """
[name]
```

```

name=traffic_light

[train]
train=data/train.txt
valid=data/val.txt
classes=1
"""

with open("data/traffic_light.data", "w") as f:
    f.write(data_config)

# Cập nhật số lớp (classes=1) trong file kiến trúc mạng
# (Đã cấu hình lại bộ Anchors K-Means riêng trong file yolo-fastestv2.yaml)
Bash
# 4. Bắt đầu huấn luyện mô hình
!python3 train.py --data data/traffic_light.data \
    --batch-size 32 \
    --epochs 300 \
    --weight model/backbone/backbone.pth

# 5. Xuất mô hình sang định dạng ONNX để triển khai trên Raspberry Pi
!python3 pytorch2onnx.py --data data/traffic_light.data \
    --weight weights/best.pth \
    --output weights/yolo-fastestv2.onnx

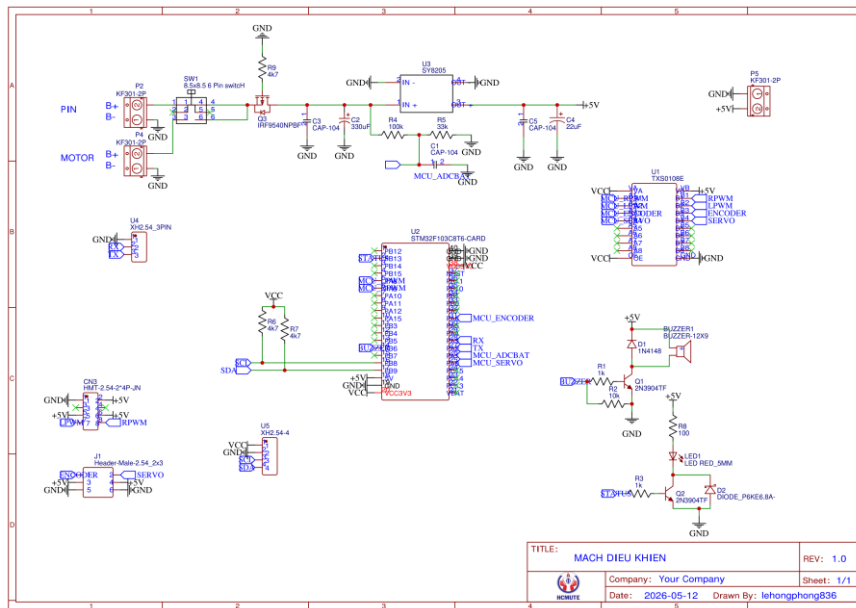
```

3.4. THIẾT KẾ PCB CHO XE TỰ HÀNH

3.4.1. Thiết kế sơ đồ nguyên lý

Để loại bỏ sự thiếu ổn định của các dây cắm testboard và tối ưu hóa không gian, toàn bộ hệ thống điều khiển mức thấp được tích hợp lên một bo mạch in (PCB) chuyên

dụng thiết kế trên nền tảng EasyEDA. Sơ đồ nguyên lý được chia thành các khối chức năng rõ ràng:



Hình 3.11 Sơ đồ nguyên lý mạch điều khiển

Khối nguồn và bảo vệ: Sử dụng Terminal KF301-2P cho ngõ vào Pin (PIN) và ngõ ra Động cơ (MOTOR). Nguồn đầu vào đi qua công tắc SW1 và mạch bảo vệ chống cắm ngược cực sử dụng P-Channel MOSFET IRF9540 (Q3). Sau đó, điện áp được đưa vào IC nguồn xung hạ áp SY8205 (U3) để cung cấp đường nguồn 5V ổn định, dòng cao cho hệ thống logic.

Khối Vi điều khiển và Giao tiếp: Trái tim của mạch là module STM32F103C8T6 (U2). Mạch tích hợp IC dịch mức logic TXS0108E (U1) để giao tiếp an toàn giữa mức 3.3V của MCU và mức 5V của các thiết bị ngoại vi (Servo, Encoder, PWM Động cơ).

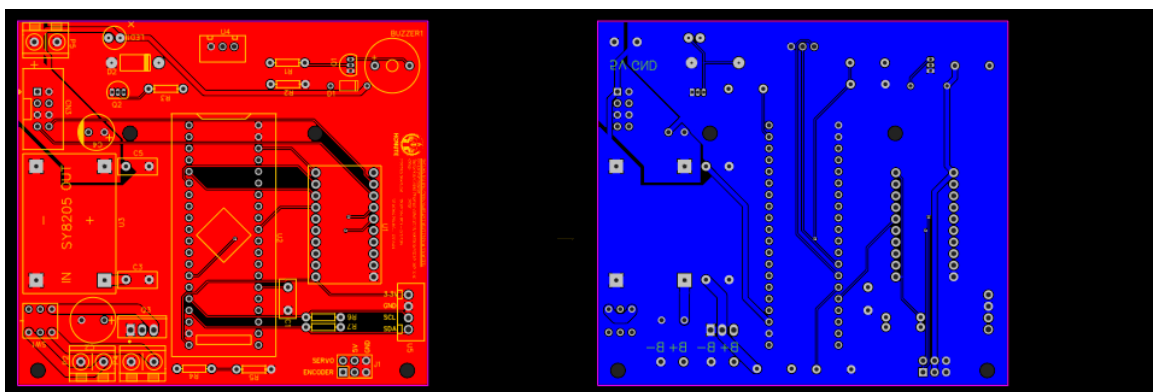
Khối Ngoại vi và Cảnh báo: Tích hợp trực tiếp các Header chuyên dụng như U4 (UART kết nối Raspberry Pi), U5 (I2C kết nối TOF VL53L0X), và J1 (chân cắm Servo/Encoder). Hệ thống cảnh báo (Buzzer, LED Status) được kích dòng an toàn thông qua các Transistor NPN 2N3904 (Q1, Q2).

3.4.2. Thiết kế Layout và Kỹ thuật đi dây (Routing)

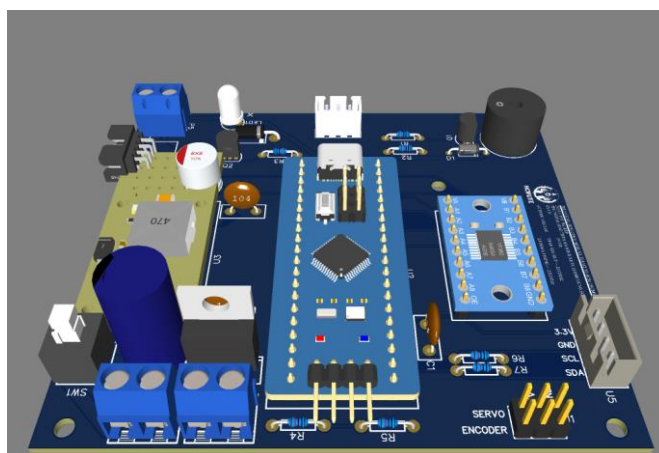
Bo mạch được thiết kế 2 lớp (Top Layer và Bottom Layer) tuân thủ các nguyên tắc thiết kế mạch hỗn hợp (Mixed-signal PCB) để chống nhiễu điện từ (EMI):

Phủ đồng (Copper Pouring): Mặt trên (Top Layer - Lớp đỏ) được phủ đồng toàn bộ và kết nối với đường GND chung của hệ thống. Kỹ thuật này tạo ra một mặt phẳng tham chiếu (Ground Plane) hoàn hảo, cung cấp đường dẫn hồi tiếp dòng điện (return path) ngắn nhất và hỗ trợ tản nhiệt trực tiếp cho khu vực IC SY8205.

Định tuyến dòng cao (High-Current Traces): Các đường dẫn điện áp từ Pin đến ngõ ra MOTOR (Terminal P2 và P4) được bố trí ở mặt dưới (Bottom Layer - Lớp xanh) với độ rộng đường mạch (trace width) cực lớn. Điều này đảm bảo mạch chịu được dòng khởi động và dòng xả liên tục của động cơ DC mà không gây sụt áp hay sinh nhiệt làm cháy board.



Hình 3.12 Layout mặt trước, sau của PCB MACH DIEU KHIEN



Hình 3.13 Mô phỏng 3D của PCB

3.5. THIẾT KẾ SA BÀN CHO XE TỰ HÀNH

3.5.1. Quy mô và cấu trúc của sa bàn

Sa bàn là môi trường thử nghiệm vật lý giả lập, được thiết kế chuyên biệt để đánh giá toàn diện năng lực của các thuật toán nhận thức (Perception) và điều khiển (Control). Sa bàn có kích thước tổng thể là 2500mm x 2500mm, mang thiết kế dạng vòng khép kín (closed-loop) với cấu trúc hình vuông được bo tròn mềm mại ở 4 góc.

Bề mặt sa bàn sử dụng nền tối màu (xám/đen) để tạo độ tương phản quang học cao nhất. Làn đường di chuyển (1 làn duy nhất) được giới hạn bởi hai vạch kẻ đường nét liền màu trắng ở cả mép trong và mép ngoài, mô phỏng chuẩn xác điều kiện đường nhựa thực tế, tạo điều kiện thuận lợi nhất cho thuật toán Canny Edge Detection và phân vùng màu.

3.5.2. Các phân đoạn quỹ đạo thử nghiệm

Thiết kế hình học của sa bàn được phân bổ nhằm tạo ra các bài test (test cases) đa dạng cho xe:

Phân đoạn đường thẳng: Giúp kiểm tra khả năng tăng tốc độ ổn định của xe, đánh giá mức độ nhiễu và độ trễ của hệ thống truyền thông UART khi thuật toán PID không phải can thiệp góc lái quá nhiều.

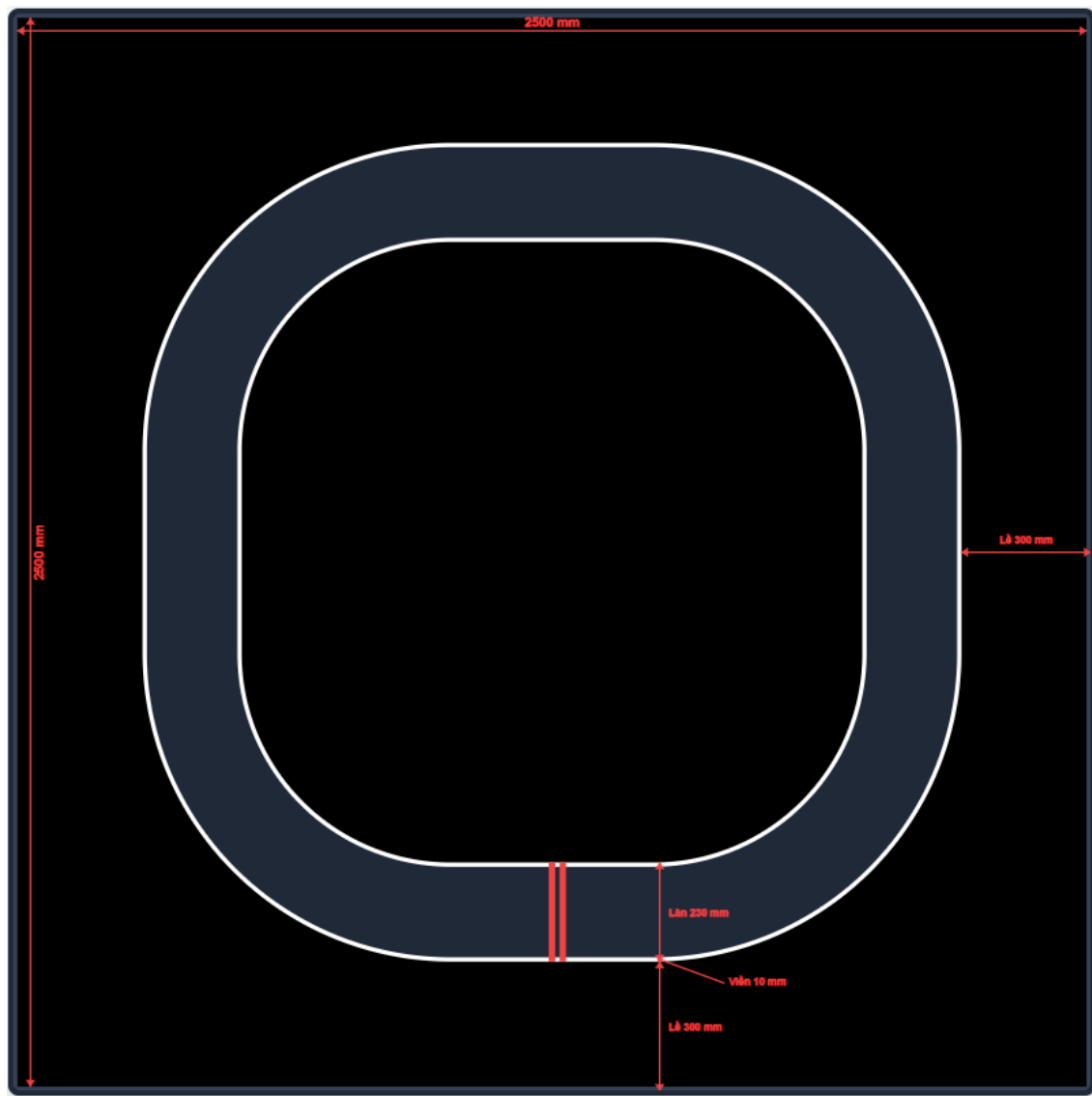
Phân đoạn đường cong (Các góc bo tròn): Bốn góc của liên tục là bài kiểm tra khắc nghiệt cho thuật toán bám quỹ đạo. Tại đây, hệ thống phải liên tục tính toán sự thay đổi của phương trình đa thức làn đường và xuất tín hiệu PWM cho servo nhằm bù đắp sai số ngang (Cross-Track Error) để xe không bị lao ra khỏi vạch trắng.

Mô phỏng tình huống dừng đỗ và giao lộ

Điểm nhấn đặc biệt trên sa bàn là sự xuất hiện của cụm vạch kẻ ngang màu đỏ (vạch đôi) nằm ở giữa đoạn đường thẳng dưới cùng.

Chức năng: Vạch đỏ này đóng vai trò là vạch dừng (Stop line) mô phỏng khu vực giao lộ hoặc trạm kiểm soát.

Tích hợp ADAS: Đây là vị trí lý tưởng để đặt các mô hình đèn tín hiệu giao thông thu nhỏ. Khi xe tự hành tiến đến khu vực này, luồng camera sẽ bắt dính tín hiệu đèn và chuyển cho mô hình AI YOLO-FastestV2 phân loại. Xe bắt buộc phải thực hiện thuật toán giảm tốc dần và phanh khẩn cấp ngay trước vạch đỏ nét đôi này, thể hiện khả năng phản ứng thời gian thực với hạ tầng giao thông thông minh.



Hình 3.14 Sa bàn kích thước 2.5m x 2.5m

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG PHẦN MỀM

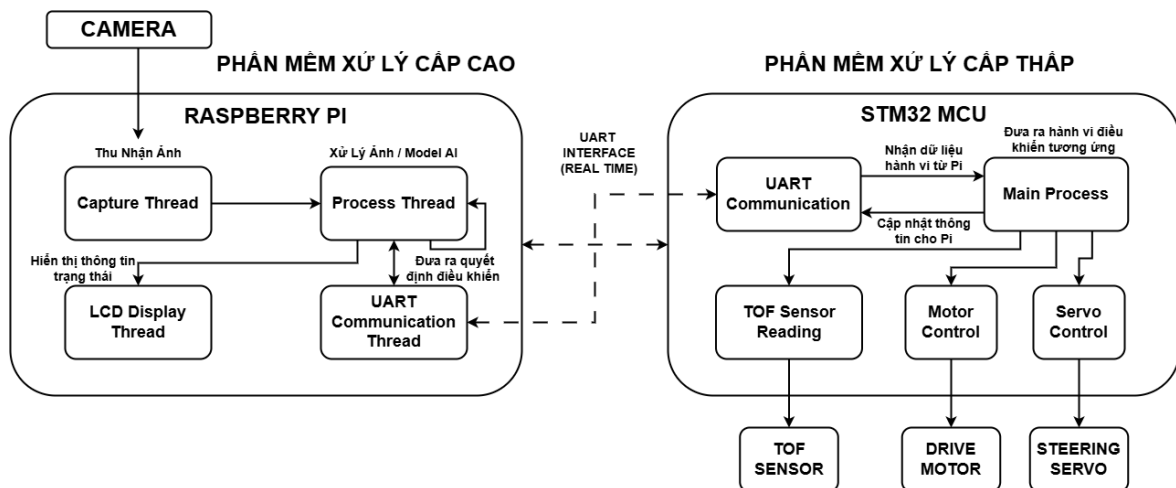
4.1. TỔNG QUAN KIẾN TRÚC PHẦN MỀM HỆ THỐNG

4.1.1. Phân chia phần mềm trên Raspberry Pi và STM32

Chương này trình bày thiết kế và triển khai phần mềm cho hệ thống xe tự hành. Hệ thống gồm hai khối chính: Raspberry Pi thực hiện các tác vụ nhận thức môi trường và ra quyết định điều khiển, trong khi STM32 đảm nhiệm điều khiển thời gian thực các cơ cấu chấp hành. Phần mềm Raspberry Pi được xây dựng theo mô hình đa luồng nhằm tối ưu hiệu năng xử lý và đảm bảo khả năng vận hành ổn định của hệ thống.

4.1.2. Sơ đồ kiến trúc phần mềm tổng thể

Sơ đồ kiến trúc phần mềm tổng thể của hệ thống được trình bày trong Hình 4.1. Sơ đồ thể hiện mối quan hệ giữa các khối phần mềm chính trên Raspberry Pi và STM32, đồng thời mô tả luồng dữ liệu từ camera đến khối xử lý cấp cao, sau đó truyền xuống khối điều khiển cấp thấp để điều khiển các cơ cấu chấp hành.



Hình 4.1 Sơ đồ kiến trúc tổng thể

Trong sơ đồ, hệ thống phần mềm được chia thành hai vùng chính:

Phần mềm xử lý cấp cao trên Raspberry Pi.

Phần mềm xử lý cấp thấp trên STM32 MCU.

Raspberry Pi đóng vai trò xử lý cấp cao, thực hiện các tác vụ liên quan đến thị giác máy tính và ra quyết định điều khiển. Phần mềm được thiết kế theo mô hình đa luồng gồm các luồng chính: thu nhận ảnh từ camera (Capture Thread), xử lý ảnh và nhận diện môi trường (Process Thread), hiển thị thông tin hệ thống trên màn hình LCD (Display Thread) và truyền thông với STM32 qua UART (UART Thread).

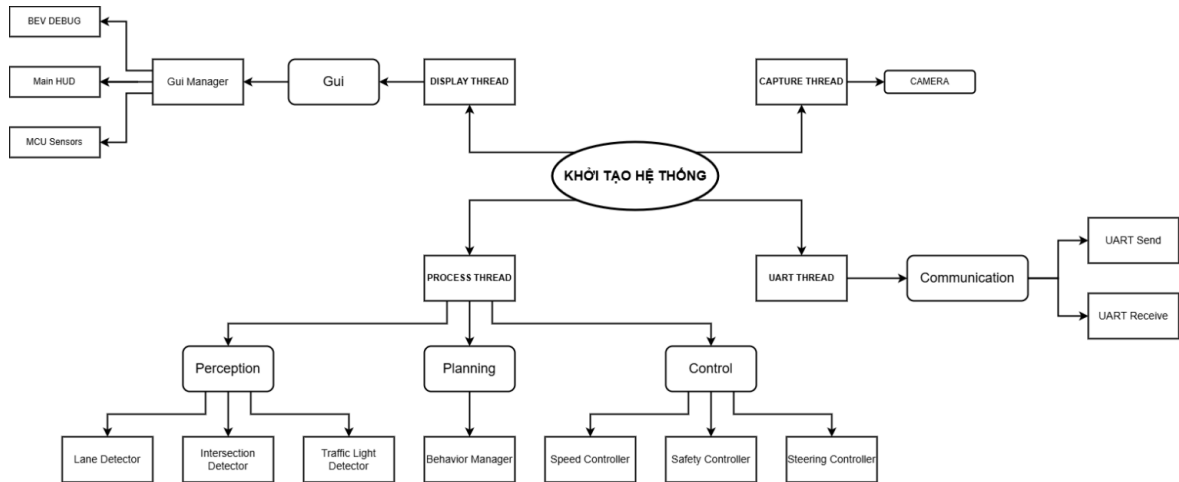
STM32 đóng vai trò điều khiển thời gian thực, trực tiếp làm việc với các cảm biến và cơ cấu chấp hành. Các chức năng chính bao gồm giao tiếp UART với Raspberry Pi, đọc dữ liệu cảm biến TOF, xử lý logic điều khiển, điều khiển động cơ DC, servo lái và còi cảnh báo. STM32 liên tục nhận lệnh điều khiển từ Raspberry Pi đồng thời giám sát khoảng cách vật cản thông qua cảm biến TOF. Khi phát hiện vật cản ở khoảng cách nguy hiểm, STM32 có thể ưu tiên dừng xe hoặc thực hiện các hành động an toàn nhằm giảm nguy cơ va chạm.

Việc phân chia phần mềm giữa Raspberry Pi và STM32 giúp tách biệt nhiệm vụ xử lý ảnh và điều khiển thời gian thực, từ đó giảm tải cho từng bộ xử lý và nâng cao độ ổn định của hệ thống. Raspberry Pi tập trung vào các thuật toán AI và nhận thức môi trường, trong khi STM32 đảm nhiệm điều khiển phần cứng và các chức năng an toàn có yêu cầu phản hồi nhanh. Kiến trúc này phù hợp với mô hình Edge AI, đồng thời thuận lợi cho việc mở rộng, bảo trì và kiểm thử hệ thống.

4.2. LẬP TRÌNH PHẦN MỀM TRÊN RASPBERRY PI

4.2.1. Tổng quan chương trình điều khiển trên Raspberry Pi

Chương trình trên Raspberry Pi được thiết kế theo mô hình đa luồng nhằm cho phép các tác vụ thu nhận ảnh, xử lý dữ liệu, hiển thị giao diện và truyền thông UART hoạt động song song. Sau khi khởi tạo, hệ thống tạo bốn luồng chính gồm Capture Thread, Process Thread, Display Thread và UART Thread.



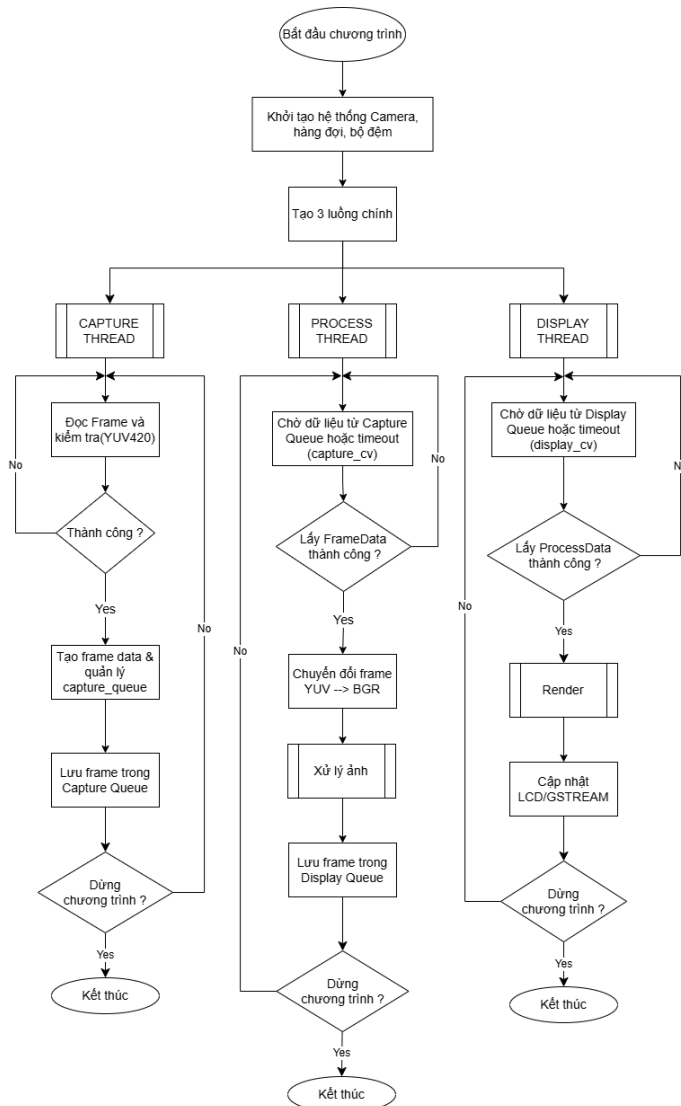
Hình 4.2 Sơ đồ tổng quan chương trình điều khiển trên Raspberry Pi

Nhìn chung, kiến trúc chương trình trên Raspberry Pi được tổ chức theo hướng tách biệt rõ ràng giữa các nhiệm vụ: thu nhận dữ liệu, xử lý nhận thức, lập kế hoạch hành vi, điều khiển, hiển thị và truyền thông. Cách thiết kế này giúp chương trình dễ quản lý, dễ mở rộng và đáp ứng tốt hơn yêu cầu xử lý thời gian thực của hệ thống xe tự lái mô hình.

4.2.2. Xử lý đa luồng trên Raspberry Pi

Dựa trên lưu đồ ở Hình 4.3, phần mềm trên Raspberry Pi được tổ chức theo kiến trúc xử lý đa luồng dạng pipeline nhằm bảo đảm khả năng xử lý thời gian thực. Khi chương trình khởi động, hệ thống tiến hành khởi tạo camera, các hàng đợi dữ liệu (queue), bộ đệm và các cơ chế đồng bộ. Sau đó, ba luồng chính gồm Capture Thread, Process Thread và Display Thread được tạo và hoạt động song song.

Capture Thread đảm nhiệm việc thu nhận dữ liệu từ camera. Luồng này liên tục đọc các khung hình ở định dạng YUV420 từ thiết bị camera. Nếu quá trình đọc thành công, dữ liệu ảnh sẽ được đóng gói thành cấu trúc FrameData và đưa vào Capture Queue. Sau khi lưu dữ liệu vào hàng đợi, luồng tiếp tục quay lại bước thu nhận khung hình mới. Quá trình này lặp lại liên tục cho đến khi hệ thống nhận yêu cầu dừng chương trình.



Hình 4.3 Lưu đồ giải thuật xử lý đa luồng

Process Thread chịu trách nhiệm xử lý ảnh. Luồng này chờ dữ liệu từ Capture

Queue thông qua cơ chế condition variable với thời gian chờ (timeout) xác định. Khi nhận được một FrameData hợp lệ, dữ liệu ảnh được chuyển đổi từ định dạng YUV sang BGR để phục vụ các thuật toán xử lý ảnh. Sau đó, hệ thống thực hiện các tác vụ như nhận diện làn đường, phát hiện giao lộ, nhận dạng đèn giao thông và tính toán lệnh điều khiển xe. Kết quả xử lý được đóng gói thành ProcessData và lưu vào Display Queue để chuyển sang giai đoạn hiển thị.

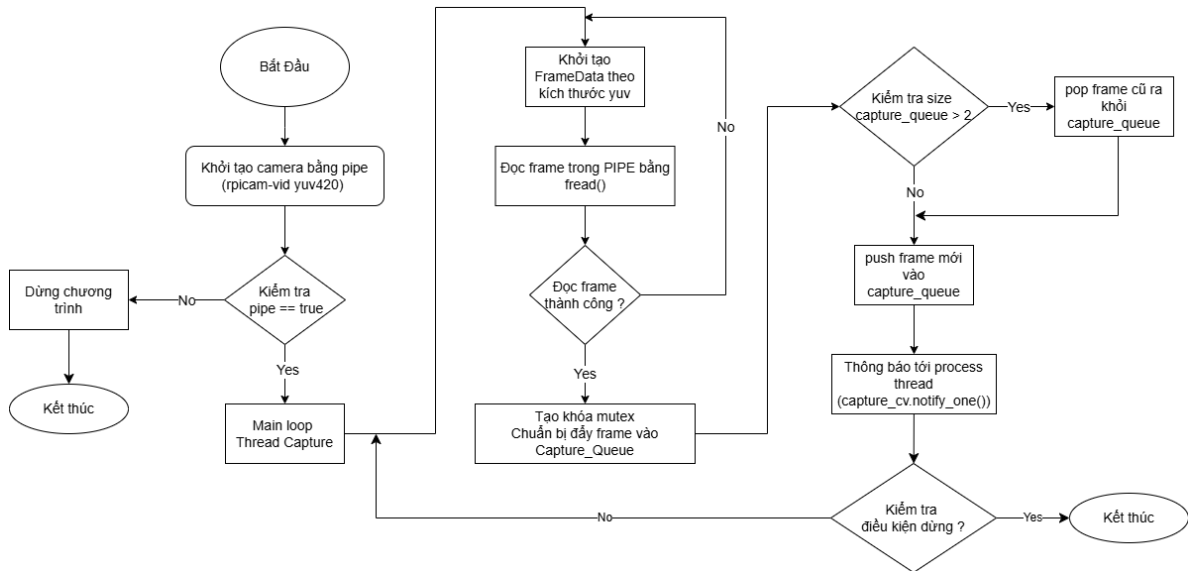
Display Thread thực hiện chức năng hiển thị và giám sát hệ thống.

Luồng này chờ dữ liệu từ Display Queue thông qua condition variable. Khi nhận được ProcessData hợp lệ, kết quả sẽ được render thành khung hình hiển thị, sau đó cập nhật lên màn hình LCD hoặc truyền tới luồng GStreamer phục vụ việc theo dõi từ xa và kiểm thử. Sau khi hoàn tất việc hiển thị, luồng tiếp tục chờ dữ liệu mới từ hàng đợi. Ba luồng hoạt động độc lập nhưng được liên kết thông qua các hàng đợi dữ liệu, giúp tách biệt tốc độ thực thi của từng tác vụ và tránh hiện tượng nghẽn hệ thống khi một công đoạn xử lý chậm hơn các công đoạn còn lại. Việc sử dụng queue kết hợp với mutex, condition variable và biến atomic cho phép đồng bộ dữ liệu an toàn giữa các

luồng, đồng thời bảo đảm hệ thống có thể thu nhận ảnh, xử lý dữ liệu và hiển thị kết quả một cách liên tục với độ trễ thấp.

4.2.3. Luồng thu nhận hình ảnh từ camera

Luồng thu nhận hình ảnh từ camera, hay Capture Thread, có nhiệm vụ đọc liên tục dữ liệu hình ảnh từ camera và cung cấp frame đầu vào cho luồng xử lý ảnh. Trong hệ thống, camera được khởi động tại hàm main() thông qua lệnh `rpicam-vid` với định dạng ảnh YUV420, độ phân giải 320×240 và tốc độ khung hình 30 FPS. Dữ liệu ảnh từ camera được đọc trực tiếp thông qua pipe bằng hàm `fread()`. Hoạt động của chương trình được thể hiện qua lưu đồ hình 4.4.



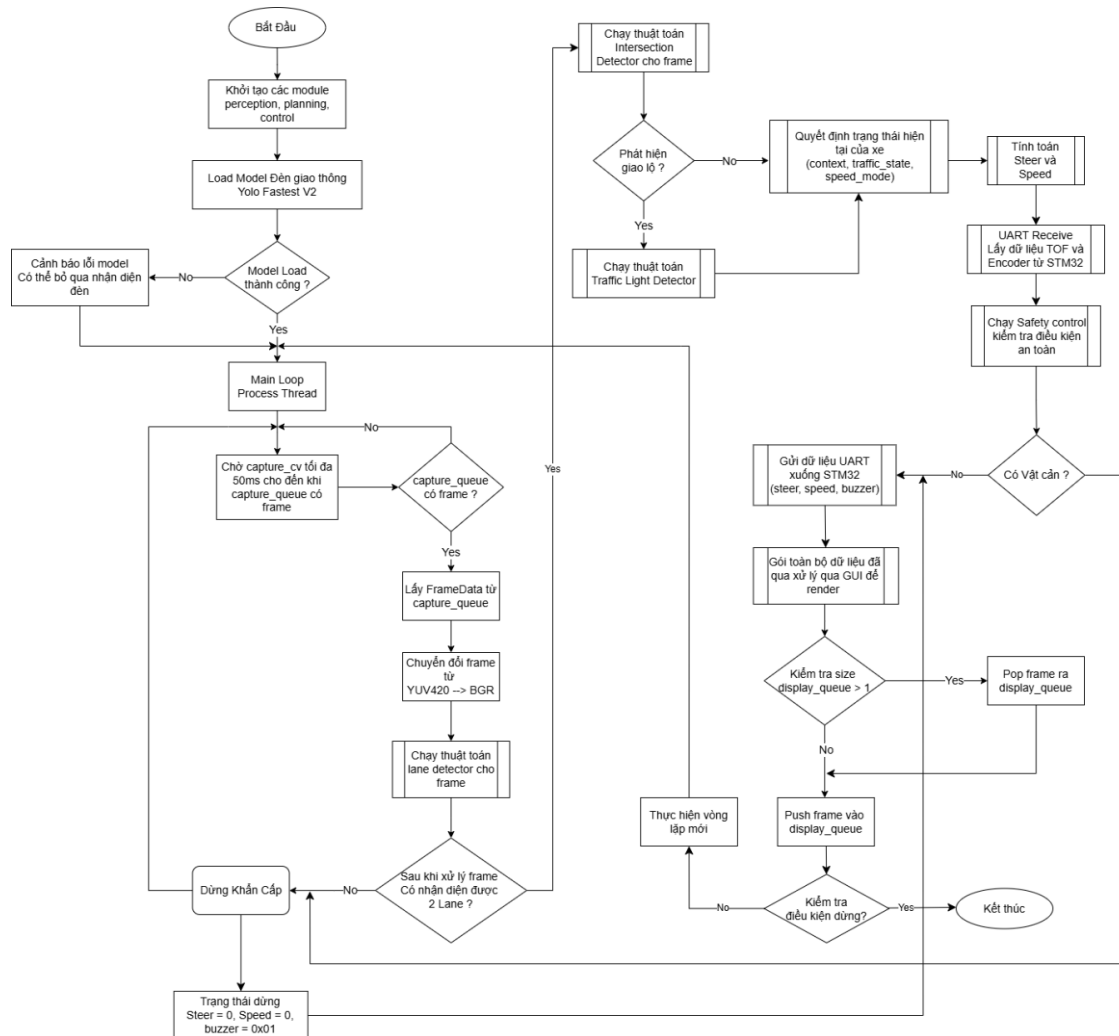
Hình 4.4 Lưu đồ giải thuật luồng thu nhận hình ảnh từ camera

Sau khi đọc thành công, mỗi khung hình được đưa vào hàng đợi `capture_queue` để truyền sang Process Thread. Cơ chế mutex được sử dụng để đảm bảo an toàn khi nhiều luồng cùng truy cập dữ liệu. Để giảm độ trễ xử lý, kích thước hàng đợi được giới hạn; các frame cũ sẽ bị loại bỏ khi hàng đợi đầy nhằm ưu tiên xử lý dữ liệu mới nhất. Khi có dữ liệu mới, Capture Thread sử dụng condition variable để thông báo cho Process Thread bắt đầu xử lý. Đồng thời, luồng cũng thống kê số khung hình thu nhận được trong mỗi giây nhằm phục vụ việc giám sát hiệu năng hệ thống. Nhờ cơ chế này,

Capture Thread đảm bảo quá trình thu nhận ảnh diễn ra liên tục, ổn định và cung cấp dữ liệu thời gian thực cho các thuật toán xử lý ảnh trên Raspberry Pi.

4.2.4. Luồng xử lý chính và đưa ra quyết định

Process Thread là luồng xử lý trung tâm của hệ thống, thực hiện toàn bộ quá trình nhận thức môi trường, ra quyết định điều khiển và giao tiếp với STM32. Khi chương trình bắt đầu, hệ thống khởi tạo các module perception, planning và control, đồng thời nạp mô hình nhận diện đèn giao thông YOLO Fastest V2. Nếu mô hình được tải thành công, chương trình chuyển sang vòng lặp xử lý chính; ngược lại, hệ thống phát cảnh báo lỗi và tiếp tục hoạt động mà không sử dụng chức năng nhận diện đèn giao thông.



Hình 4.5 Lưu đồ giải thuật luồng xử lý chính và đưa ra quyết định

Trong quá trình vận hành, Process Thread liên tục chờ dữ liệu từ capture_queue với thời gian chờ tối đa 50 ms. Khi có khung hình mới, dữ liệu được lấy ra khỏi hàng đợi và chuyển đổi từ định dạng YUV420 sang BGR để phục vụ cho các thuật toán xử lý ảnh của OpenCV. Sau đó, thuật toán Lane Detector được thực hiện nhằm phát hiện làn đường và xác định vị trí của xe trên mặt đường.

Sau khi xử lý làn đường, hệ thống kiểm tra kết quả nhận diện. Nếu không phát hiện đủ hai làn đường hợp lệ, cơ chế dừng khẩn cấp được kích hoạt, các giá trị điều khiển được đưa về trạng thái an toàn với góc lái bằng 0, tốc độ bằng 0 và buzzer được kích hoạt cảnh báo. Nếu phát hiện làn đường thành công, hệ thống tiếp tục thực hiện thuật toán Intersection Detector để xác định sự xuất hiện của khu vực giao lộ. Khi không phát hiện giao lộ, hệ thống sử dụng trạng thái hiện tại của xe gồm các thông tin như trạng thái điều khiển, trạng thái giao thông và chế độ tốc độ để đưa ra quyết định vận hành. Ngược lại, nếu phát hiện giao lộ, thuật toán Traffic Light Detector sẽ được kích hoạt để nhận diện trạng thái đèn giao thông. Kết quả nhận diện sau đó được sử dụng để cập nhật trạng thái vận hành của xe trước khi chuyển sang bước điều khiển.

Từ trạng thái vận hành hiện tại, bộ điều khiển tính toán giá trị góc lái (Steering) và tốc độ (Speed) phù hợp. Đồng thời, Process Thread nhận dữ liệu phản hồi từ STM32 thông qua luồng UART Receive, bao gồm khoảng cách đo được từ cảm biến TOF và dữ liệu encoder. Các thông tin này được chuyển đến Safety Controller để kiểm tra điều kiện an toàn trong quá trình di chuyển. Nếu phát hiện vật cản hoặc nguy cơ mất an toàn, Safety Controller sẽ can thiệp vào lệnh điều khiển nhằm giảm tốc hoặc dừng xe. Sau khi hoàn tất quá trình kiểm tra, các giá trị điều khiển cuối cùng gồm góc lái, tốc độ và trạng thái buzzer được gửi xuống STM32 thông qua giao tiếp UART.

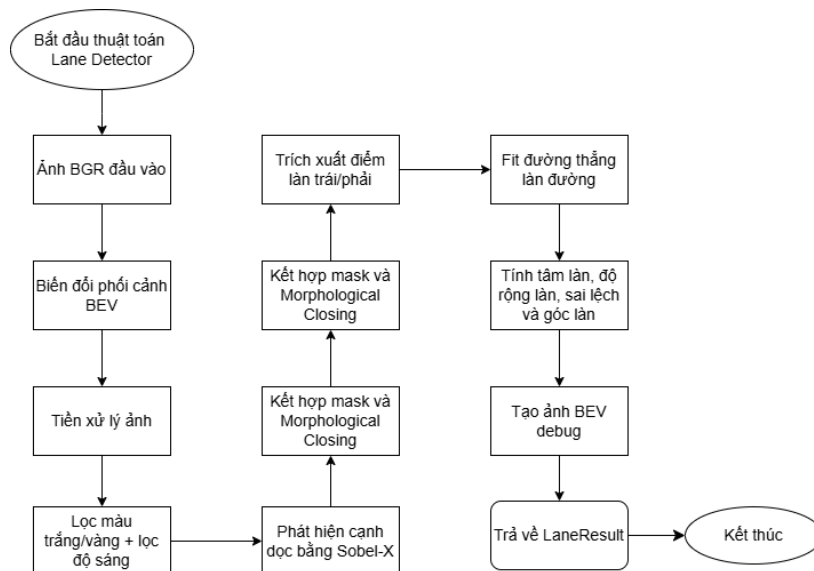
Song song với việc điều khiển xe, toàn bộ dữ liệu xử lý và kết quả nhận diện được đóng gói thành một khung dữ liệu phục vụ hiển thị. Trước khi đưa vào display_queue, hệ thống kiểm tra kích thước hàng đợi; nếu số lượng phần tử vượt quá ngưỡng cho phép, khung hình cũ nhất sẽ bị loại bỏ nhằm tránh hiện tượng tích tụ dữ liệu và tăng độ trễ. Sau đó, khung dữ liệu mới được đưa vào hàng đợi để Display Thread thực hiện

hiển thị. Cuối mỗi chu kỳ, hệ thống kiểm tra điều kiện dừng chương trình; nếu chưa có yêu cầu kết thúc, Process Thread quay trở lại đầu vòng lặp và tiếp tục xử lý khung hình tiếp theo.

Thông qua chuỗi xử lý từ nhận ảnh, nhận diện môi trường, đánh giá an toàn, tính toán điều khiển đến gửi lệnh xuống STM32, Process Thread đóng vai trò là trung tâm điều phối toàn bộ hoạt động của xe tự hành, bảo đảm hệ thống vận hành liên tục và đáp ứng các yêu cầu thời gian thực.

4.2.5. Thuật toán phát hiện làn đường

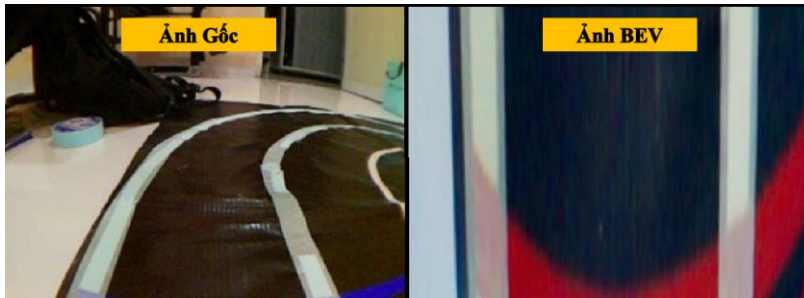
Thuật toán phát hiện làn đường được cài đặt trong lớp LaneDetector, có nhiệm vụ xác định vị trí làn đường phía trước xe, tính toán độ lệch so với tâm làn và cung cấp dữ liệu cho bộ điều khiển hướng lái. Đầu vào của thuật toán là ảnh màu từ camera, đầu ra là cấu trúc LaneResult chứa các thông tin như trạng thái phát hiện làn, vị trí tâm làn, độ rộng làn đường, sai lệch chuẩn hóa và góc định hướng của làn.



Hình 4.6 Lưu đồ giải thuật của thuật toán phát hiện làn đường

định tâm làn và ước lượng hướng di chuyển của xe [9].

Quá trình xử lý gồm các bước chính như Hình 4.6. Trước tiên, ảnh đầu vào được biến đổi phối cảnh sang góc nhìn từ trên xuống (Bird's Eye View - BEV) như Hình 4.7. Phép biến đổi này giúp hai vạch làn trở nên gần song song, làm đơn giản hóa quá trình xác

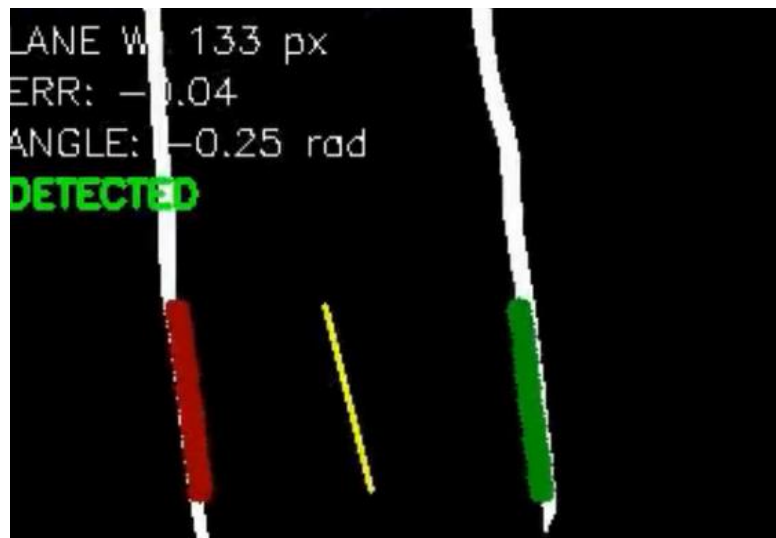


Hình 4.7 Biến đổi phối cảnh Bird's Eye View

Tiếp theo, ảnh BEV được tiền xử lý nhằm làm nổi bật các vạch làn đường. Hệ thống sử dụng bộ lọc Gaussian để giảm

nhiễu, kết hợp phân tích màu sắc trong không gian HLS và độ sáng của ảnh để tạo ảnh nhị phân chứa các vùng có khả năng thuộc vạch làn. Sau đó, toán tử Sobel-X được áp dụng để phát hiện các cạnh dọc đặc trưng của vạch kẻ đường. Kết quả tiếp tục được xử lý bằng các phép toán hình thái học nhằm loại bỏ nhiễu và nối các đoạn vạch bị đứt.

Từ ảnh nhị phân thu được, thuật toán tiến hành trích xuất các điểm thuộc làn trái và làn phải trong vùng quan sát gần xe. Các điểm này được sử dụng để xấp xỉ đường thẳng đại diện cho mỗi làn đường. Dựa trên vị trí của hai làn, hệ thống tính toán tâm làn và độ rộng làn



Hình 4.8 Biến đổi ảnh BEV nhị phân

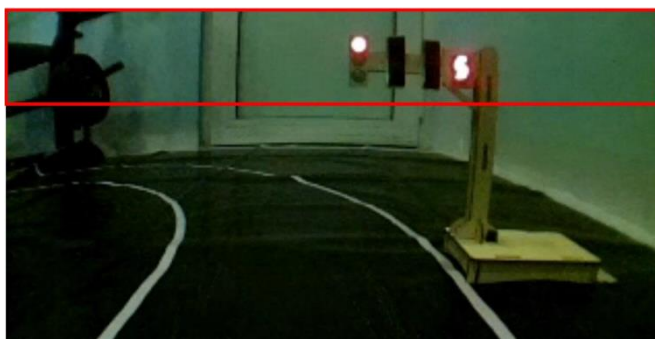
đường. Trong trường hợp chỉ phát hiện được một bên làn, tâm làn được ước lượng dựa trên độ rộng làn tiêu chuẩn.

Sau khi xác định được tâm làn, thuật toán tính sai lệch chuẩn hóa giữa tâm làn và tâm ảnh. Giá trị sai lệch này nằm trong khoảng $[-1, 1]$ và được sử dụng làm đầu vào cho bộ điều khiển lái. Đồng thời, góc định hướng của làn đường cũng được tính toán nhằm đánh giá xu hướng cong hoặc lệch hướng của đường.

Nhờ kết hợp các kỹ thuật biến đổi phối cảnh, lọc màu, phân tích độ sáng, phát hiện cạnh và xử lý hình thái học, thuật toán có thể phát hiện làn đường với độ ổn định cao trong thời gian thực. Cách tiếp cận này phù hợp với nền tảng Raspberry Pi nhờ yêu cầu tính toán thấp nhưng vẫn đáp ứng tốt bài toán giữ làn cho mô hình xe tự hành.

4.2.6. Thuật toán nhận diện đèn giao thông

Thuật toán nhận diện đèn giao thông được cài đặt trong lớp TrafficLightDetector, có nhiệm vụ phát hiện vị trí đèn giao thông trong ảnh và xác định trạng thái hiện tại của đèn gồm đỏ, vàng, xanh hoặc không xác định. Khác với thuật



Hình 4.9 Vùng ROI nhận diện đèn

toán phát hiện làn đường sử dụng chủ yếu các kỹ thuật xử lý ảnh truyền thống, thuật toán này kết hợp mô hình học sâu YOLO-FastestV2 với phương pháp phân tích màu sắc trong không gian HLS. Lưu đồ giải thuật của thuật toán phát hiện đèn giao thông được mô tả như Hình 4.11

Để giảm khối lượng tính toán, hệ thống chỉ xử lý một vùng quan tâm (ROI) nằm ở phía trên ảnh như Hình 4.9, nơi đèn giao thông thường xuất hiện trong môi trường thử nghiệm. Mô hình YOLO-FastestV2 được lưu dưới định dạng ONNX và được thực thi trên Raspberry Pi thông qua thư viện OpenCV DNN. Khi khởi động, mô hình được nạp vào bộ nhớ và sẵn sàng thực hiện suy luận trên các khung hình đầu vào. Trong quá trình hoạt động, ảnh trong vùng ROI được chuẩn hóa và đưa vào mạng nơ-ron để phát hiện đối tượng đèn giao thông. Kết quả đầu ra của mô hình bao gồm các bounding box ứng viên cùng độ tin cậy tương ứng. Thuật toán tiếp tục sử dụng kỹ thuật Non-Maximum Suppression (NMS) để loại bỏ các bounding box chồng lấp và lựa chọn vùng phát hiện có độ tin cậy cao nhất. Sau khi xác định được vị trí đèn giao thông, hệ thống tiến hành phân tích màu sắc bên trong bounding box nhằm xác định trạng thái đèn. Vùng ảnh chứa đèn được chuyển sang không gian màu HLS, trong đó các thành

phần màu đỏ, vàng và xanh được tách riêng bằng các ngưỡng màu thích nghi theo điều kiện ánh sáng. Thuật toán đếm số lượng điểm ảnh thuộc từng màu và xác định trạng thái đèn dựa trên màu chiếm ưu thế.

Kết quả nhận diện được lưu trong cấu trúc TrafficLightRaw, bao gồm trạng thái phát hiện đèn, màu đèn và các thông tin phục vụ hiển thị. Đồng thời, bounding box và nhãn trạng thái được vẽ trực tiếp lên ảnh



Hình 4.10 Kết quả nhận diện đèn

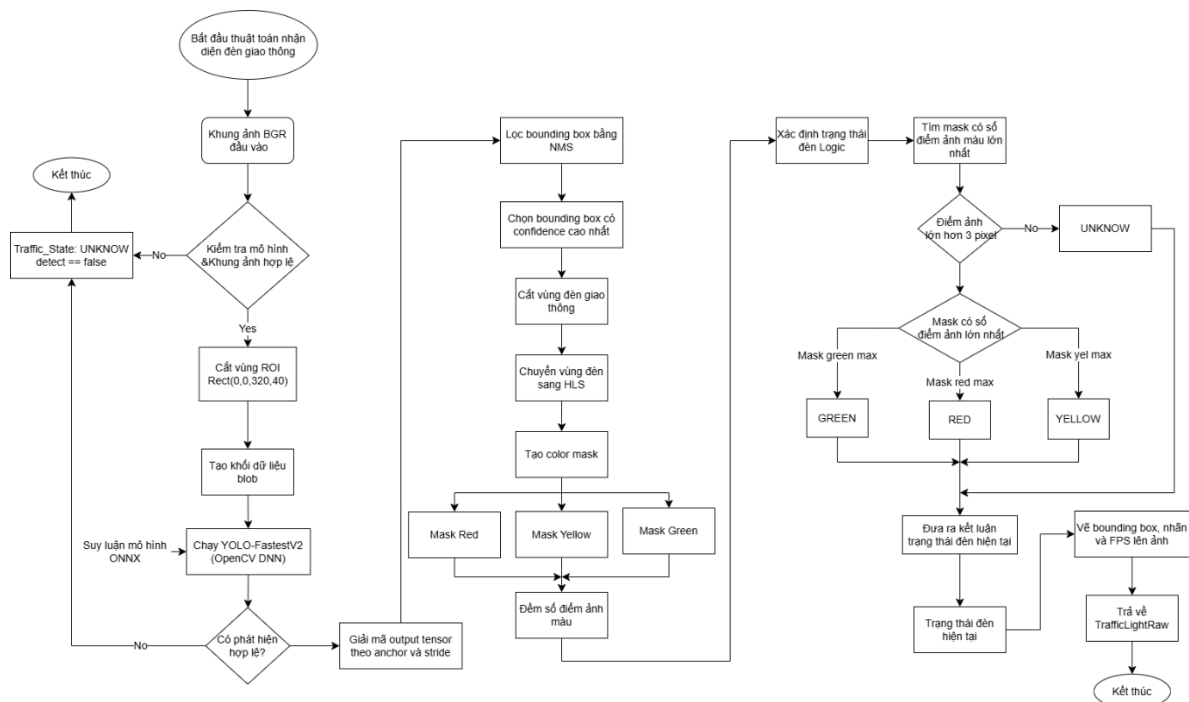
để hỗ trợ giám sát và đánh giá thuật toán như Hình 4.10.

Thông tin nhận diện đèn giao thông được gửi đến khối BehaviorManager để hỗ trợ ra quyết định điều khiển. Dựa trên trạng thái đèn, hệ thống có thể thực hiện các hành vi tương ứng như dừng xe khi gặp đèn đỏ, giảm tốc khi gặp đèn vàng hoặc tiếp tục di chuyển khi đèn xanh.

Bảng 4.1 Kết quả bảng nhận diện màu đèn giao thông

Trạng thái	Màu hiển thị
RED	Đỏ
YELLOW	Vàng
GREEN	Xanh
UNKNOWN	Xanh lam/vàng nhạt

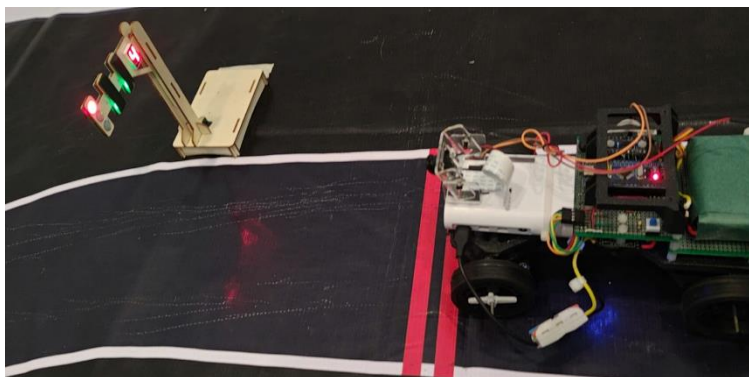
Việc kết hợp mô hình YOLO-FastestV2 với phân tích màu trong không gian HLS giúp thuật toán vừa đảm bảo khả năng phát hiện chính xác vị trí đèn giao thông, vừa xác định được trạng thái màu với chi phí tính toán phù hợp cho nền tảng Raspberry Pi, đáp ứng yêu cầu xử lý thời gian thực của hệ thống xe tự hành.



Hình 4.11 Lưu đồ giải thuật cho thuật toán nhận diện đèn giao thông

4.2.7. Thuật toán phát hiện khu vực giao lộ

Thuật toán phát hiện khu vực giao lộ được cài đặt trong lớp IntersectionDetector, có nhiệm vụ nhận biết sự xuất hiện của vạch dừng trên mặt đường để xác định xe đang tiếp cận giao lộ. Đầu vào của thuật toán là ảnh Bird's Eye View (BEV) cùng thông tin tâm làn và độ rộng làn đường từ khối phát hiện làn. Kết quả đầu ra là cấu trúc IntersectionResult, trong đó biến stop_line_detected cho biết trạng thái phát hiện giao lộ. Luồng hoạt động của thuật toán có thể được mô tả như Hình 4.15.



Hình 4.12 Khu vực giao lộ

Trong mô hình thử nghiệm, vạch dừng được biểu diễn bằng màu đỏ như Hình 4.12. Do đó, thuật toán tập trung tìm kiếm các vùng màu đỏ nằm giữa hai làn đường. Trước tiên, một vùng quan tâm (ROI)

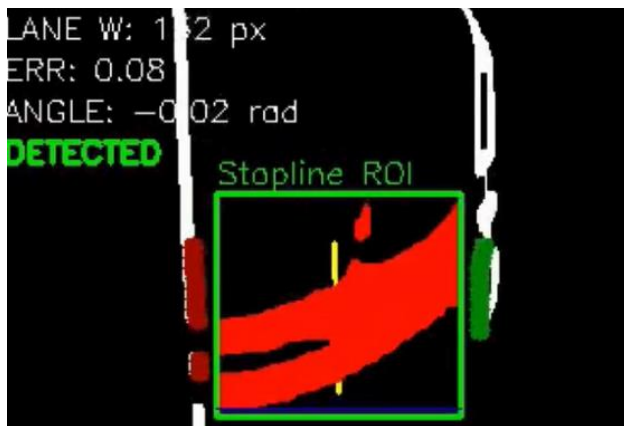
được xác định dựa trên vị trí tâm làn và độ rộng làn đường nhằm giới hạn phạm vi xử lý, giảm nhiễu và tiết kiệm tài nguyên tính toán như Hình 4.13.

Sau khi xác định ROI, ảnh được làm mịn bằng bộ lọc Gaussian và chuyển sang không gian màu HLS. Thuật toán sử dụng các ngưỡng màu để tách riêng vùng màu đỏ, sau đó áp dụng phép toán hình thái học (Morphological Closing) nhằm loại bỏ nhiễu và nối các đoạn vạch bị đứt gãy.



Hình 4.13 Vùng ROI nhận diện giao lộ

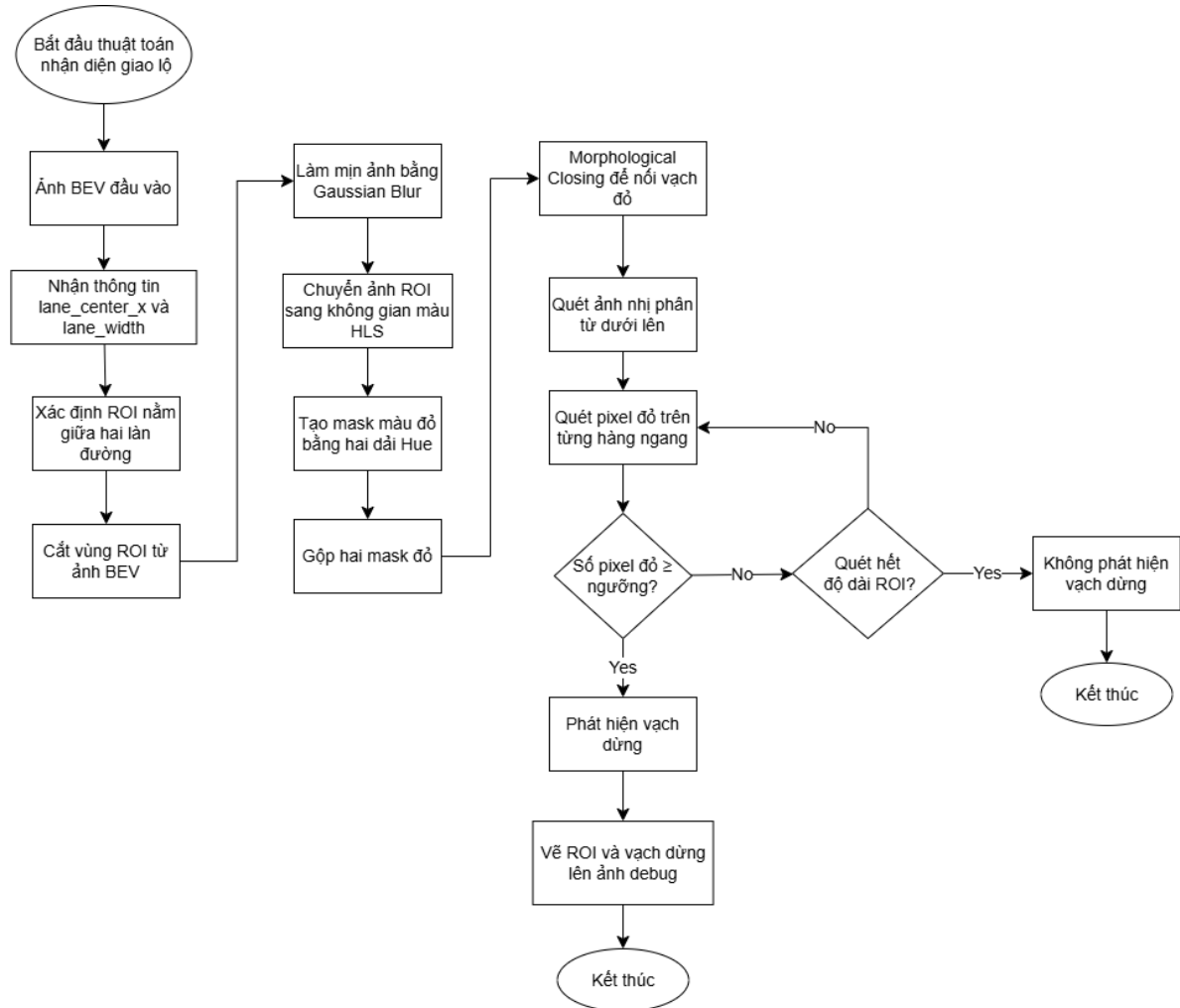
Từ ảnh nhị phân thu được, hệ thống quét các hàng pixel theo hướng từ dưới lên để tìm vạch dừng gần xe nhất. Tại mỗi hàng, số lượng điểm ảnh màu đỏ được thống kê và so sánh với ngưỡng xác định trước. Nếu số điểm ảnh vượt ngưỡng, hàng đó được xem là ứng viên của vạch dừng. Để tăng độ tin cậy và hạn chế phát hiện nhầm do nhiễu, thuật toán sử dụng cơ chế xác nhận qua nhiều khung hình liên tiếp. Chỉ khi vạch đỏ được phát hiện liên tục trong một số frame nhất định, trạng thái stop_line_detected mới được kích hoạt. Cơ chế này giúp nâng cao độ ổn định của hệ thống trong điều kiện vận hành thực tế.



Hình 4.14 Kết quả khi nhận diện được vạch giao lộ

Kết quả phát hiện giao lộ được truyền đến khối BehaviorManager để hỗ trợ ra quyết định điều khiển. Khi phát hiện vạch dừng, hệ thống có thể chuyển sang trạng thái tiếp cận giao lộ, kích hoạt chức năng nhận diện đèn giao thông và thực hiện các hành vi phù hợp như giảm tốc hoặc dừng xe.

Nhờ tận dụng thông tin làn đường và đặc trưng màu sắc của vạch dừng, thuật toán có độ phức tạp thấp, dễ triển khai trên Raspberry Pi và đáp ứng tốt yêu cầu xử lý thời gian thực của hệ thống xe tự hành.



Hình 4.15 Lưu đồ giải thuật cho thuật toán phát hiện giao lộ

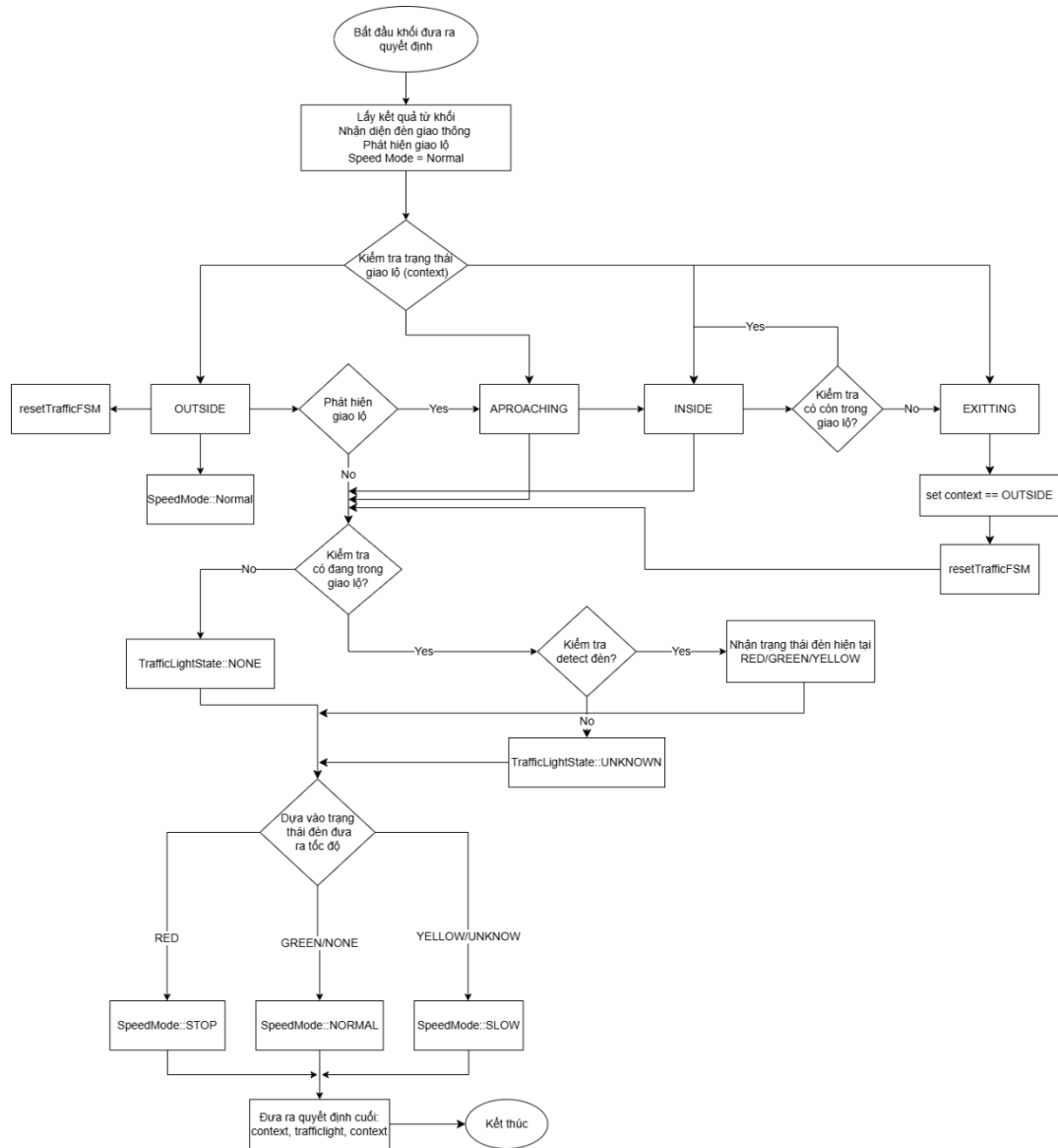
4.2.8. Khối quyết định hành vi điều khiển

Khối quyết định hành vi điều khiển (Behavior Manager) đóng vai trò trung tâm trong hệ thống điều hướng của xe tự hành. Khối này tiếp nhận dữ liệu từ các module nhận thức môi trường như phát hiện giao lộ và nhận diện đèn giao thông, sau đó đưa ra quyết định điều khiển tốc độ phù hợp với từng tình huống vận hành. Lưu đồ luồng hoạt động của khối được mô tả như Hình 4.16.

Để đảm bảo tính ổn định và hạn chế ảnh hưởng của nhiễu nhận dạng, thuật toán được xây dựng theo mô hình máy trạng thái hữu hạn (Finite State Machine – FSM), bao gồm hai thành phần chính: FSM ngữ cảnh giao lộ và FSM đèn giao thông. FSM ngữ cảnh giao lộ được sử dụng để xác định vị trí của xe so với giao lộ, gồm bốn trạng thái: OUTSIDE, APPROACHING, INSIDE và EXITING. Việc chuyển trạng thái được thực hiện dựa trên kết quả phát hiện vạch dừng kết hợp với bộ đếm thời gian nhằm tránh dao động do nhiễu ảnh. Khi xe ở trạng thái APPROACHING hoặc INSIDE, FSM đèn giao thông được kích hoạt để xác định trạng thái đèn gồm RED, YELLOW, GREEN, UNKNOWN hoặc NONE. Để tăng độ tin cậy, trạng thái đèn chỉ được xác nhận khi được phát hiện liên tiếp trong nhiều khung hình. Trong trường hợp tạm thời mất nhận dạng, hệ thống sử dụng trạng thái hợp lệ gần nhất trong một khoảng thời gian giới hạn để duy trì tính ổn định.

Dựa trên trạng thái đèn giao thông đã được xác nhận, hệ thống đưa ra hành vi điều khiển tương ứng: dừng xe khi gặp đèn đỏ hoặc đèn vàng, tiếp tục di chuyển khi đèn xanh, giảm tốc khi trạng thái đèn chưa xác định và duy trì tốc độ bình thường khi không có giao lộ.

Kết quả của Behavior Manager được gửi đến các bộ điều khiển chuyển động để điều chỉnh tốc độ và hành vi vận hành của xe. Việc kết hợp FSM với cơ chế xác nhận nhiều khung hình giúp hệ thống hoạt động ổn định và đáng tin cậy hơn trong điều kiện thực tế.



Hình 4.16 Lưu đồ giải thuật cho khối quyết định hành vi điều khiển

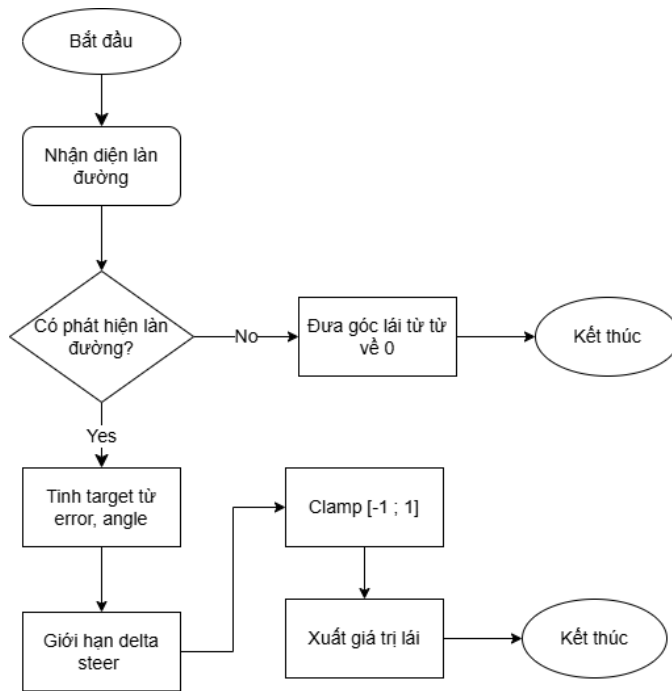
Kết quả của Behavior Manager được gửi đến các bộ điều khiển chuyển động để điều chỉnh tốc độ và hành vi vận hành của xe. Việc kết hợp FSM với cơ chế xác nhận nhiều khung hình giúp hệ thống hoạt động ổn định và đáng tin cậy hơn trong điều kiện thực tế.

4.2.9. Thuật toán điều khiển tốc độ và hướng lái

Khối điều khiển tốc độ và hướng lái có nhiệm vụ chuyển đổi thông tin từ các khối nhận thức môi trường thành lệnh điều khiển thực tế cho xe tự hành. Dữ liệu đầu vào

bao gồm kết quả phát hiện làn đường, phát hiện giao lộ, quyết định hành vi và khoảng cách vật cản từ cảm biến ToF. Đầu ra của khối là các lệnh điều khiển góc lái và tốc độ gửi đến STM32 thông qua giao tiếp UART.

a. Thuật toán điều khiển hướng lái



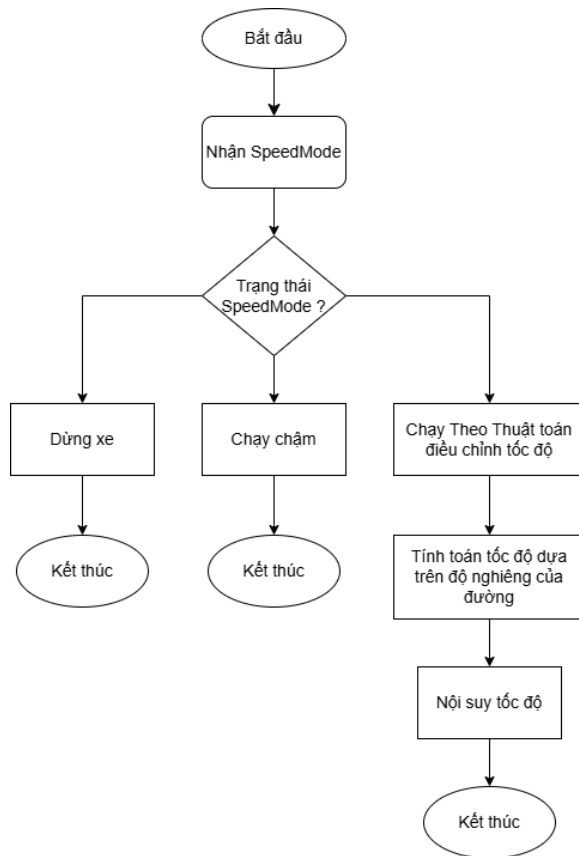
Hình 4.17 Lưu đồ giải thuật cho thuật toán điều khiển hướng lái

đoạn đường cong. Để tránh hiện tượng đánh lái đột ngột, sự thay đổi góc lái được giới hạn và làm mượt theo từng chu kỳ xử lý trước khi gửi đến servo. Trong trường hợp không phát hiện được làn đường, hệ thống sẽ đưa góc lái dần về vị trí trung tâm nhằm duy trì trạng thái an toàn và giảm nguy cơ mất kiểm soát.

b. Thuật toán điều khiển tốc độ

Tốc độ xe được điều chỉnh dựa trên trạng thái hành vi do Behavior Manager cung cấp. Hệ thống hỗ trợ ba chế độ điều khiển chính: STOP, SLOW và NORMAL.

Trong chế độ NORMAL, tốc độ được điều chỉnh theo độ cong của làn đường. Khi xe di chuyển trên đoạn đường thẳng, tốc độ được duy trì ở mức cao hơn; ngược lại, khi

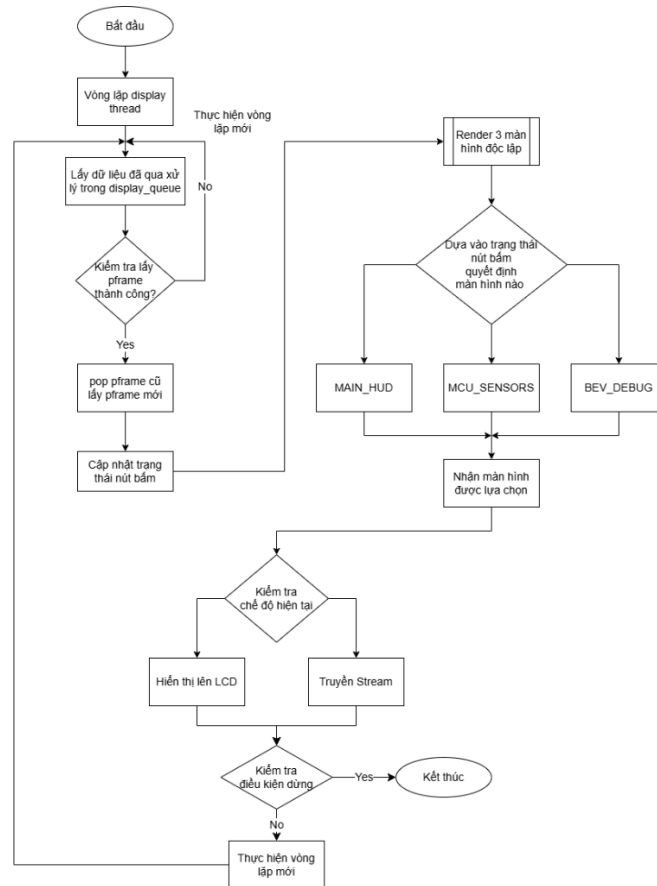


Hình 4.18 Lưu đồ giải thuật cho thuật toán điều khiển tốc độ

phát hiện đường cong, tốc độ sẽ tự động giảm để tăng độ ổn định và hạn chế nguy cơ lệch làn. Nếu hệ thống không phát hiện được làn đường, tốc độ được đưa về 0 nhằm đảm bảo an toàn.

4.2.10. Luồng hiển thị thông tin lên LCD và GSTREAM

Khởi hiển thị có nhiệm vụ trực quan hóa dữ liệu vận hành của xe tự hành và hỗ trợ giám sát hệ thống theo thời gian thực. Chức năng này được triển khai, hoạt động độc lập với các luồng xử lý khác nhằm đảm bảo hiệu năng thời gian thực.



Hình 4.19 Lưu đồ giải thuật cho luồng hiển thị LCD và Gstreamer.

Hệ thống hỗ trợ ba chế độ hiển thị chính

Main HUD hiển thị hình ảnh camera cùng các thông số vận hành quan trọng như tốc độ, góc lái, trạng thái giao lộ, trạng thái đèn giao thông, điện áp pin, khoảng cách ToF và tốc độ động cơ. Khi phát hiện vật cản nguy hiểm, màn hình sẽ hiển thị cảnh báo để hỗ trợ người vận hành.



Hình 4.20 Màn hình HUD

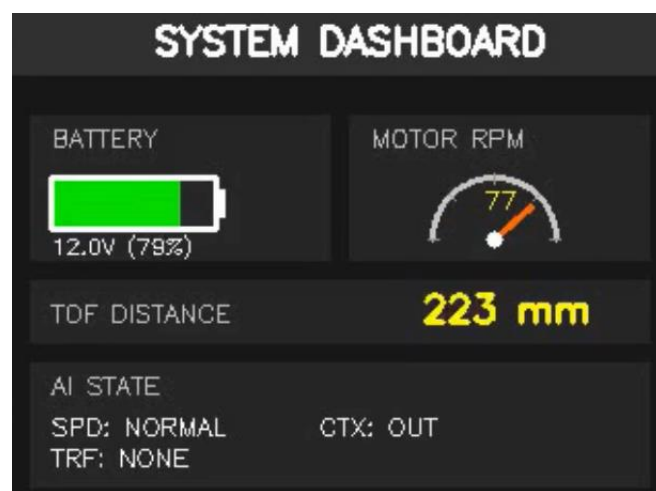
BEV Debug hiển thị ảnh Bird's Eye View phục vụ kiểm tra thuật toán phát hiện làn đường, kèm các thông số như độ rộng làn, sai lệch tâm làn và góc định hướng của làn đường.

Dashboard hiển thị trạng thái phần cứng và cảm biến của hệ thống, bao gồm mức pin, tốc độ động cơ, khoảng cách ToF và các thông tin giám sát khác.

Ngoài việc hiển thị trực tiếp trên màn hình LCD, hệ thống còn hỗ trợ truyền video thời gian thực qua mạng LAN bằng GStreamer [13]. Video được mã hóa theo chuẩn H.264 và truyền qua giao thức RTP/UDP, cho phép giám sát từ xa và hỗ trợ quá trình kiểm thử, đánh



Hình 4.21 Màn hình BEV



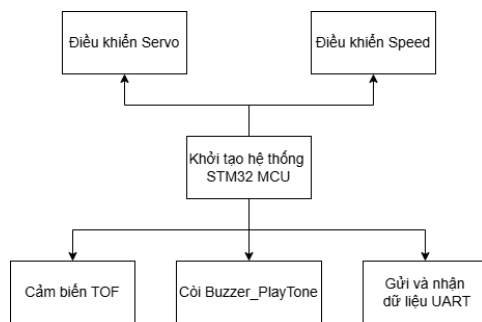
Hình 4.22 Màn hình HUD

giá hệ thống. Bên cạnh đó, Display Thread còn hỗ trợ ghi lại video của các giao diện hiển thị bằng OpenCV VideoWriter để phục vụ phân tích và đánh giá kết quả thực nghiệm. Nhờ khả năng hiển thị trực quan, hỗ trợ debug và truyền hình ảnh thời gian thực, khối hiển thị góp phần quan trọng trong quá trình giám sát, đánh giá và tối ưu hệ thống xe tự hành.

4.3. LẬP TRÌNH PHẦN MỀM TRÊN STM32

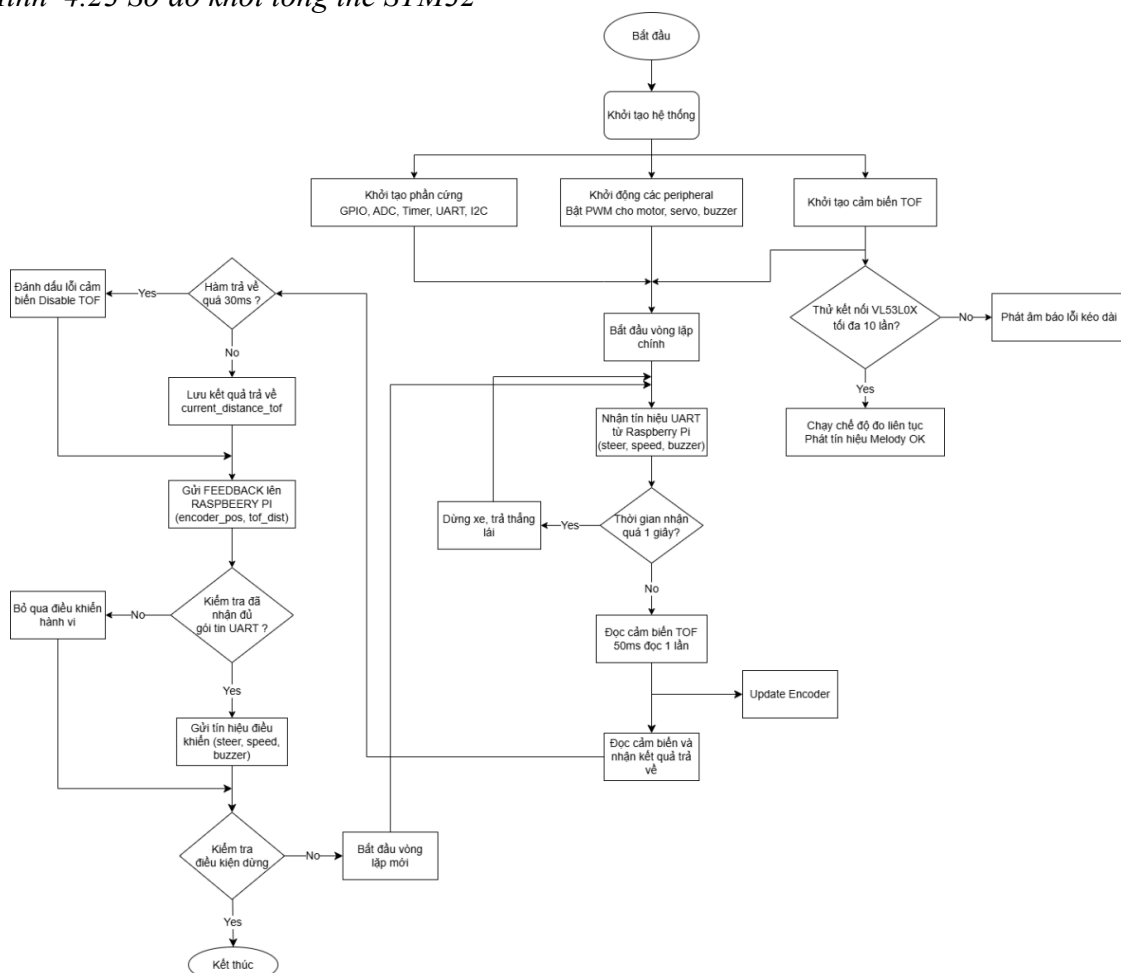
4.3.1. Tổng quan chương trình điều khiển trên STM32

Hình 4.23 mô tả sơ đồ khối tổng thể của chương trình điều khiển trên STM32. Trong hệ thống, STM32 đóng vai trò bộ điều khiển thời gian thực, thực hiện giao tiếp với Raspberry Pi, đọc dữ liệu cảm biến và điều khiển các cơ cấu chấp hành gồm động cơ, servo lái và buzzer.



Hình 4.23 Sơ đồ khối tổng thể STM32

Chương trình trên STM32 được xây dựng theo mô hình vòng lặp chính (super-loop), bảo đảm xử lý liên tục các tác vụ giao tiếp, thu thập dữ liệu cảm biến và điều khiển hệ thống. Lưu đồ hoạt động được trình bày ở Hình 4.24.



Hình 4.24 Lưu đồ giải thuật trên STM32

Sau khi khởi động, STM32 thực hiện khởi tạo các ngoại vi như GPIO, ADC, Timer, UART và I2C, đồng thời kích hoạt các kênh PWM điều khiển động cơ, servo và buzzer. Cảm biến khoảng cách VL53L0X được kiểm tra kết nối và đưa vào chế độ đo liên tục nếu khởi tạo thành công.

Trong quá trình hoạt động, STM32 thực hiện các chức năng chính:

Nhận lệnh điều khiển từ Raspberry Pi thông qua UART.

Đọc dữ liệu cảm biến TOF và encoder.

Điều khiển tốc độ động cơ, góc lái servo và buzzer.

Thực hiện cơ chế an toàn (fail-safe), tự động dừng xe và đưa bánh lái về vị trí trung tâm khi mất tín hiệu điều khiển hoặc phát hiện vật cản ở khoảng cách nguy hiểm.

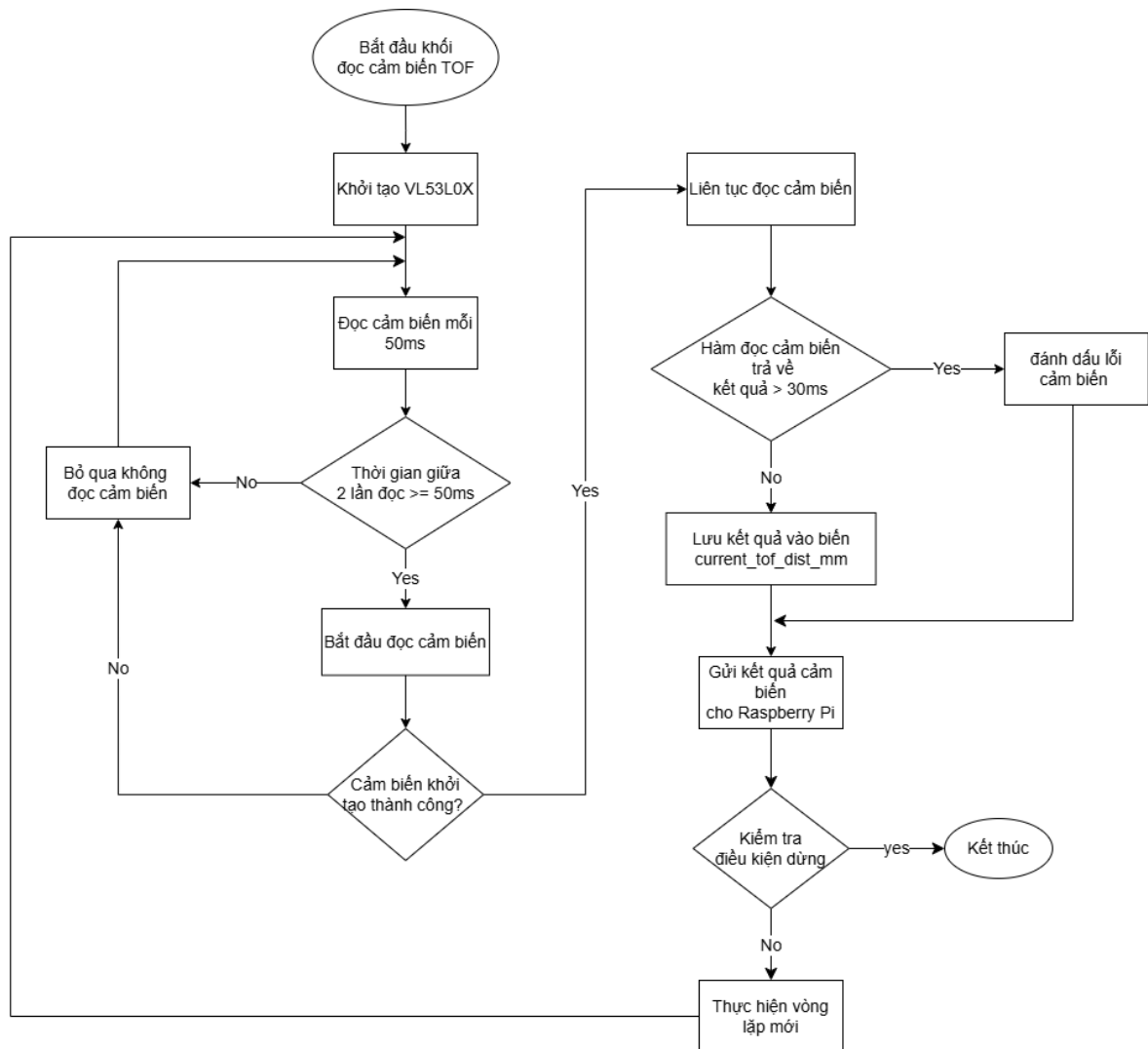
Gửi dữ liệu phản hồi về Raspberry Pi, bao gồm điện áp pin, khoảng cách TOF và tốc độ encoder.

Với kiến trúc đơn giản và tách biệt rõ ràng giữa các chức năng, chương trình trên STM32 đáp ứng tốt yêu cầu điều khiển thời gian thực và đảm bảo độ ổn định cho hệ thống xe tự hành.

4.3.2. Đọc cảm biến khoảng cách TOF

Khối cảm biến khoảng cách được triển khai trên STM32F103C8T6, sử dụng cảm biến VL53L0X giao tiếp qua chuẩn I2C1 với tốc độ 100 kHz. Nhiệm vụ chính của khối là đo khoảng cách vật cản phía trước xe và cung cấp dữ liệu cho hệ thống điều khiển. Sau khi khởi động, STM32 thực hiện khởi tạo cảm biến VL53L0X. Nếu khởi tạo thành công, cảm biến được đưa vào chế độ đo liên tục; ngược lại hệ thống sẽ phát cảnh báo bằng buzzer để thông báo lỗi cảm biến hoặc lỗi giao tiếp I2C. Trong quá trình hoạt động, STM32 đọc dữ liệu khoảng cách theo chu kỳ 50 ms (tương đương khoảng 20 Hz). Giá trị đo được được lưu vào biến `current_tof_dist_mm` và sử dụng cho các chức năng điều khiển, giám sát an toàn và truyền thông với Raspberry Pi. Để tăng độ tin cậy của hệ thống, chương trình tích hợp cơ chế chống treo khi đọc cảm biến. Nếu thời gian đọc dữ liệu vượt quá ngưỡng cho phép, hệ thống sẽ xem cảm biến

gặp lỗi, ngừng đọc ở các chu kỳ tiếp theo và gán giá trị khoảng cách mặc định nhằm tránh ảnh hưởng đến hoạt động chung của xe.

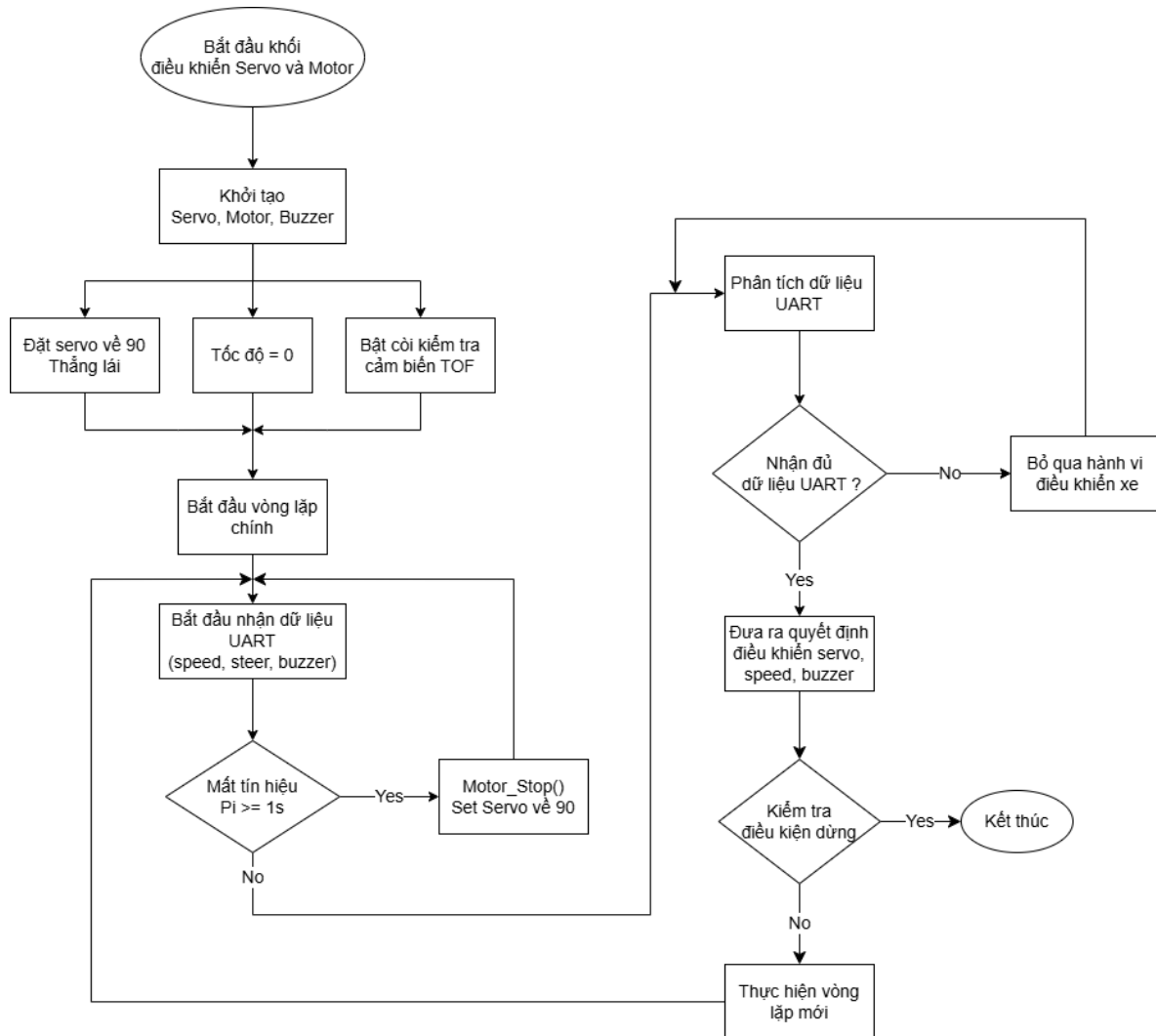


Hình 4.25 Lưu đồ giải thuật của thuật toán đọc cảm biến

Dữ liệu khoảng cách thu được được đóng gói vào gói phản hồi UART và gửi định kỳ về Raspberry Pi. Thông tin này được sử dụng để giám sát vật cản phía trước, hiển thị lên giao diện người dùng và hỗ trợ các cơ chế điều khiển an toàn của hệ thống xe tự hành.

4.3.3. Điều khiển động cơ DC và Servo lái

Khởi động điều khiển động cơ và servo được triển khai trên STM32, có nhiệm vụ nhận lệnh điều khiển từ Raspberry Pi, xử lý dữ liệu và điều khiển các cơ cấu chấp hành của xe. Luồng hoạt động của thuật toán được mô tả trong Hình 4.26.



Hình 4.26 Lưu đồ giải thuật thuật toán điều khiển động cơ DC và Servo

Sau khi khởi động, STM32 tiến hành khởi tạo các ngoại vi phục vụ điều khiển gồm động cơ DC, servo lái và buzzer. Hệ thống đưa servo về vị trí trung tâm (90°), đặt tốc độ động cơ bằng 0 và phát tín hiệu kiểm tra trạng thái cảm biến ToF. Sau khi hoàn tất quá trình khởi tạo, chương trình chuyển sang vòng lặp điều khiển chính.

Trong mỗi chu kỳ xử lý, STM32 liên tục nhận dữ liệu điều khiển từ Raspberry Pi thông qua giao tiếp UART. Dữ liệu nhận được bao gồm giá trị tốc độ, góc lái và trạng

thái buzzer. Trước khi thực hiện điều khiển, hệ thống kiểm tra trạng thái kết nối với Raspberry Pi. Nếu mất tín hiệu điều khiển trong khoảng thời gian quy định, cơ chế an toàn (failsafe) được kích hoạt, động cơ sẽ dừng và servo được đưa về vị trí trung tâm nhằm tránh các tình huống mất kiểm soát.

Khi dữ liệu UART được nhận đầy đủ và hợp lệ, STM32 tiến hành phân tích gói lệnh, sau đó cập nhật các giá trị điều khiển tương ứng cho động cơ DC, servo lái và buzzer. Giá trị tốc độ được chuyển đổi thành tín hiệu PWM để điều khiển driver BTS7960, trong khi giá trị góc lái được chuyển đổi thành độ rộng xung PWM phù hợp với servo. Nếu dữ liệu UART không hợp lệ hoặc chưa nhận đủ, hệ thống bỏ qua chu kỳ điều khiển hiện tại và tiếp tục chờ dữ liệu mới. Quá trình trên được lặp lại liên tục trong suốt thời gian hoạt động của xe, giúp STM32 duy trì điều khiển thời gian thực và đảm bảo phản hồi nhanh đối với các lệnh từ Raspberry Pi.

4.4. GIAO TIẾP UART GIỮA RASPBERRY PI VÀ STM32

Trong hệ thống, Raspberry Pi và STM32 giao tiếp với nhau thông qua chuẩn UART. Đây là kênh truyền dữ liệu chính giữa tầng xử lý cấp cao và tầng điều khiển mức thấp. Raspberry Pi đảm nhiệm việc xử lý ảnh, thực thi thuật toán ADAS và sinh ra lệnh điều khiển, trong khi STM32 nhận lệnh để điều khiển trực tiếp động cơ, servo, buzzer và đọc các tín hiệu phản hồi từ cảm biến.

Giao tiếp UART được cấu hình với tốc độ truyền 115200 bps, định dạng truyền 8 bit dữ liệu, không parity, 1 stop bit. Raspberry Pi mở cổng UART bằng thư viện termios, cấu hình chế độ truyền nhận không đồng bộ và sử dụng cơ chế đọc/ghi trực tiếp qua file descriptor của Linux. Ở phía STM32, UART2 được sử dụng để nhận lệnh điều khiển và gửi dữ liệu phản hồi về Raspberry Pi.

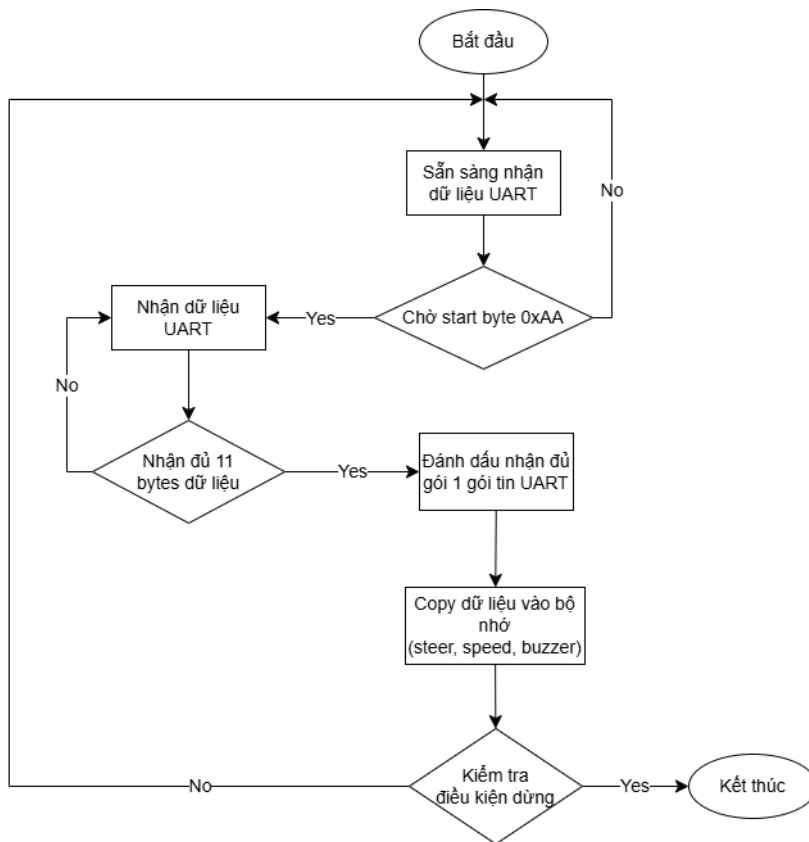
4.4.1. Chiều truyền từ Raspberry Pi xuống STM32

Trong hệ thống, Raspberry Pi gửi các lệnh điều khiển xuống STM32 thông qua giao tiếp UART. Dữ liệu được đóng gói trong cấu trúc `ControlPacket_t`, bao gồm các thông tin điều khiển như góc lái, tốc độ, chế độ hoạt động và trạng thái buzzer. Mỗi gói tin

sử dụng byte bắt đầu 0xAA và byte kết thúc 0x55 để đánh dấu khung truyền, giúp tăng khả năng đồng bộ dữ liệu.

Ở phía Raspberry Pi, hàm `uart_send()` thực hiện đóng gói các tham số điều khiển và gửi xuống STM32 theo từng chu kỳ xử lý. Các giá trị góc lái và tốc độ được tính toán từ các thuật toán điều khiển trước đó, trong khi chế độ hoạt động và trạng thái buzzer được xác định bởi khối quyết định hành vi.

Để đảm bảo độ tin cậy khi nhận dữ liệu, STM32 sử dụng cơ chế state machine gồm ba trạng thái: chờ byte bắt đầu, nhận dữ liệu và chờ byte kết thúc. Cách tiếp cận này giúp hệ thống đồng bộ chính xác khung truyền và hạn chế lỗi do nhiễu hoặc mất byte trong quá trình truyền thông.

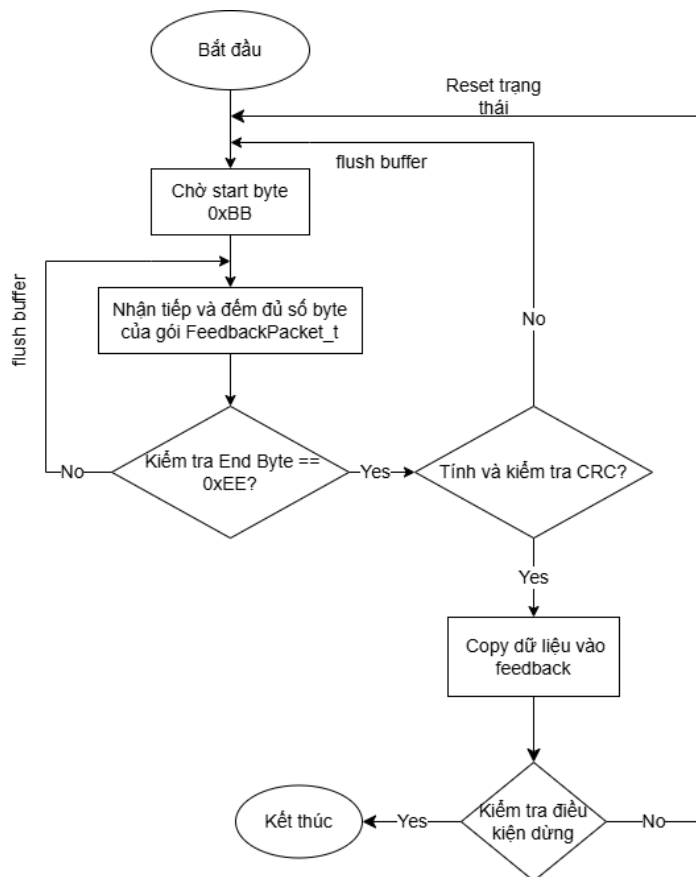


Hình 4.27 Sơ đồ luồng nhận dữ liệu từ STM32

Khi nhận được một gói lệnh hợp lệ, dữ liệu được lưu vào bộ đệm điều khiển và kích hoạt cờ xử lý để vòng lặp chính cập nhật động cơ, servo lái và buzzer. Đồng thời, STM32 ghi nhận thời điểm nhận lệnh cuối cùng thông qua biến `last_uart_tick`. Nếu trong một khoảng thời gian xác định không nhận được gói lệnh mới từ Raspberry Pi, cơ chế failsafe sẽ được kích hoạt, tự động dừng động cơ và đưa servo về vị trí trung tâm nhằm đảm bảo an toàn cho hệ thống.

4.4.2. Chiều phản hồi từ STM32 lên Raspberry Pi

Bên cạnh việc nhận lệnh điều khiển, STM32 còn định kỳ gửi dữ liệu phản hồi về Raspberry Pi thông qua UART nhằm cung cấp thông tin trạng thái thực tế của xe. Dữ liệu được đóng gói trong cấu trúc `FeedbackPacket_t`, bao gồm điện áp pin, giá trị encoder và khoảng cách đo được từ cảm biến ToF. Mỗi gói tin sử dụng byte bắt đầu `0xBB` và byte kết thúc `0xEE` để đánh dấu khung truyền.



Hình 4.28 Sơ đồ luồng nhận dữ liệu từ Pi

hợp lệ, chương trình cập nhật dữ liệu vào biến dùng chung `mcu_status`. Do quá trình nhận UART được thực hiện trên một luồng riêng, cơ chế mutex được sử dụng để đồng bộ truy cập dữ liệu giữa các luồng, tránh xảy ra xung đột hoặc sai lệch dữ liệu.

Thông tin phản hồi từ STM32 được sử dụng cho các chức năng hiển thị, giám sát hệ thống, ghi log và hỗ trợ các thuật toán điều khiển cấp cao trên Raspberry Pi.

4.4.3. Cơ chế an toàn trong giao tiếp UART

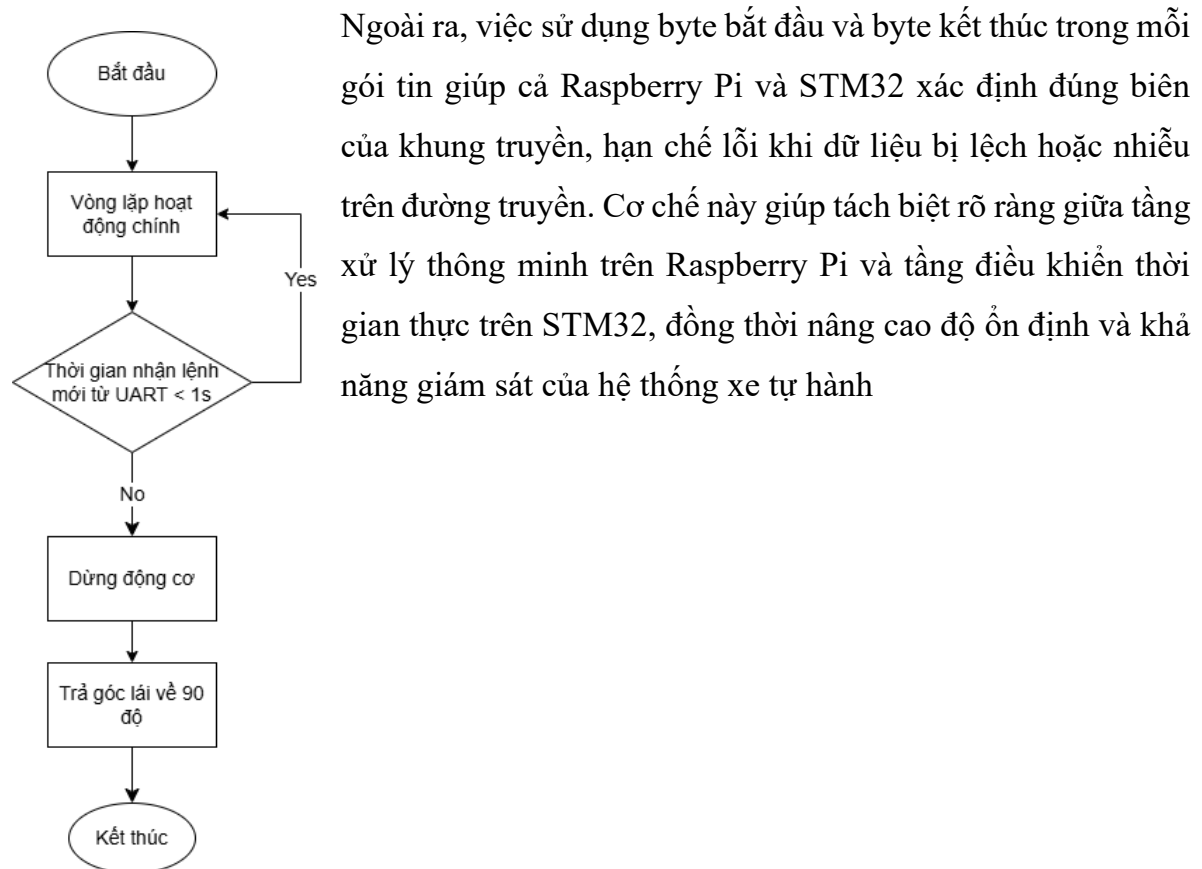
Ở phía STM32, hàm `Send_Feedback()` thực hiện đọc dữ liệu cảm biến hiện tại, đóng gói thành gói phản hồi và gửi lên Raspberry Pi. Các thông tin này giúp hệ thống giám sát tình trạng nguồn, trạng thái chuyển động của xe và khoảng cách tới vật cản phía trước.

Tại Raspberry Pi, dữ liệu phản hồi tương tự chiều truyền điều khiển, Raspberry Pi sử dụng cơ chế state machine để đồng bộ khung dữ liệu và đảm bảo tính chính xác trong quá trình nhận.

Khi nhận được gói phản hồi

Để tăng độ an toàn, STM32 sử dụng biến `last_uart_tick` để giám sát thời gian nhận lệnh cuối cùng từ Raspberry Pi. Nếu STM32 không nhận được gói lệnh hợp lệ trong một khoảng thời gian nhất định, hệ thống sẽ tự động dừng động cơ và đưa servo về vị trí trung tâm.

Cơ chế này giúp xe không tiếp tục di chuyển khi mất kết nối UART hoặc khi Raspberry Pi bị treo.



Hình 4.29 Sơ đồ luồng xử lý an toàn trong giao tiếp UART

CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG

5.1. KẾT QUẢ THI CÔNG PHẦN CỨNG VÀ CƠ KHÍ

Quá trình hoàn thiện phần cứng là bước bản lề để chuyển đổi từ các bản vẽ thiết kế lý thuyết sang một mô hình thực thể có khả năng vận hành. Mục tiêu của giai đoạn này là đảm bảo sự chắc chắn về mặt cơ khí, tính thẩm mỹ, gọn gàng trong cách bố trí dây dẫn và sự phân tách hợp lý giữa các phân khu nguồn/logic để đảm bảo ổn định lâu dài.

5.1.1. Hoàn thiện cơ khí xe tự hành

Mô hình xe được thiết kế với tỉ lệ 1:16. Khung gầm của xe tự hành được thiết kế và định hình để đáp ứng hai tiêu chí: chịu tải trọng tốt và có không gian lắp đặt linh hoạt cho các tầng phần cứng khác nhau.

Vật liệu và kết cấu: Khung xe được gia công bằng phương pháp in 3D chính xác từ vật liệu nhựa, tạo ra các lỗ bắt vít chuẩn theo thiết kế. Kết cấu khung dạng nhiều tầng được cố định qua các trụ đỡ vững chắc, giúp tách biệt hoàn toàn không gian của phần cứng công suất phía dưới và phần cứng xử lý logic phía trên.



Hình 5.1 Kết cấu của mô hình xe tự hành



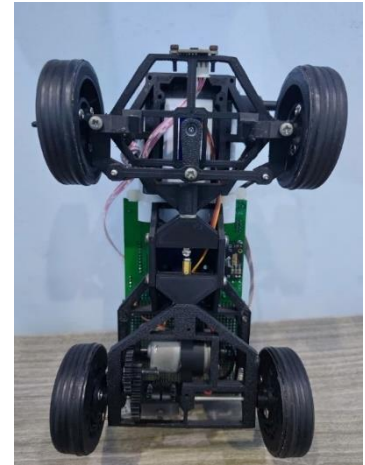
Hình 5.2 Hệ thống dẫn động cầu sau của xe tự hành

Cơ cấu truyền động và Encoder: Hệ thống dẫn động cầu sau sử dụng động cơ DC giảm tốc (màu xám). Động cơ được cố định vào khung thông qua các gá in 3D chắc chắn. Trục động cơ truyền lực đến bánh xe sau thông qua một hệ bánh răng giảm tốc bổ, giúp tăng mô-men xoắn để xe khởi động và di chuyển

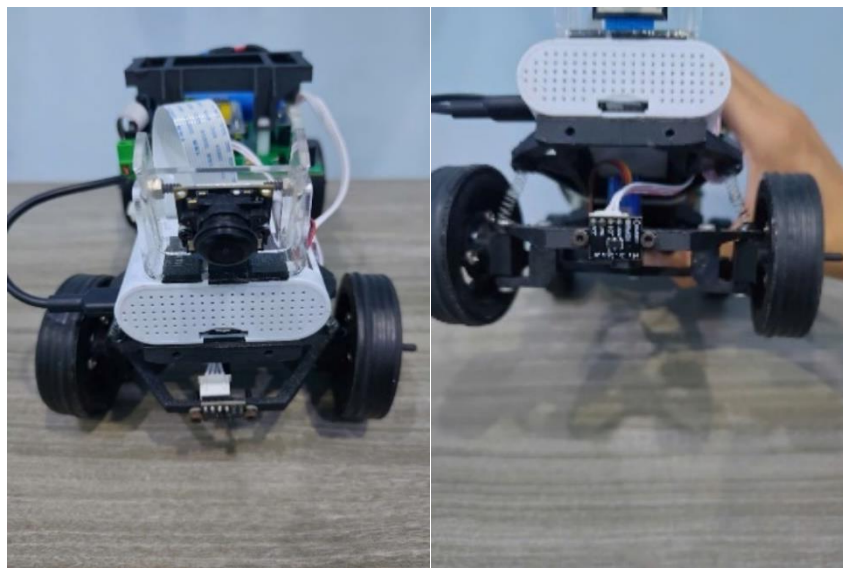
êm ái hơn trên sa bàn. Cảm biến Encoder từ tính được tích hợp ngay phía sau động cơ để thu thập vận tốc bánh xe.

Cơ cấu lái và giảm xóc: Cơ cấu lái Ackermann sử dụng động cơ Servo RC nối với hệ thanh giằng và nhựa. Phía trước có hệ thống lò xo giảm xóc giúp xe di chuyển ổn định trên các bề mặt không bằng phẳng.

Vị trí cảm biến và camera: Camera được đặt trên hộp đựng Raspberry pi 4 và cúi xuống mặt đất 1 góc $\sim 30^\circ$, giúp camera có thể bám làn tốt hơn, vẫn có thể thấy đủ rõ đèn giao thông ở giao lộ. Cảm biến TOF được đặt đáy đầu xe, giúp cảm biến có thể đo đạc chính xác ngay tại đầu xe, tránh va chạm.



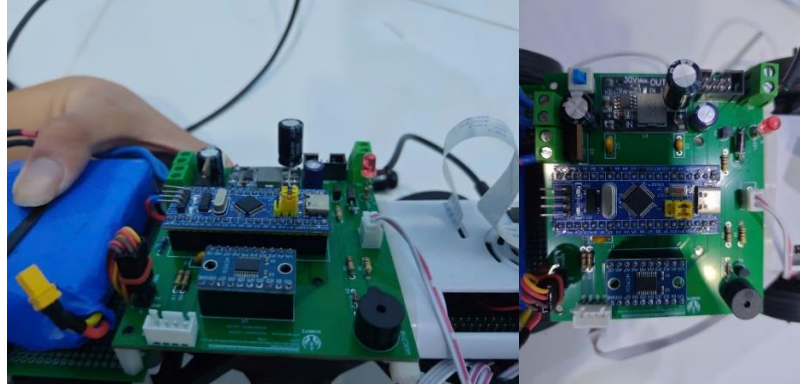
Hình 5.3 Hệ thống đánh lái của xe tự hành



Hình 5.4 Vị trí đặt camera và cảm biến ToF

5.1.2. Kết quả thiết kế và gia công mạch in

Mạch in (PCB) được thiết kế đóng vai trò là "trạm trung chuyển" tín hiệu và nguồn điện, kết nối hệ thống máy tính nhúng (Raspberry Pi), vi điều khiển (STM32) và các cơ cấu chấp hành.



Hình 5.5 Hình ảnh mạch PCB thành phẩm

5.2. ĐÁNH GIÁ ĐỘ ỔN ĐỊNH CỦA GIAO TIẾP VÀ THỜI GIAN THỰC

5.2.1. Đánh giá tốc độ đáp ứng của cảm biến

Đánh giá thời gian thu thập khung hình camera

Để đảm bảo khả năng phản ứng theo thời gian thực của hệ thống tự hành, độ trễ thu thập hình ảnh đầu vào (Read Latency) được đo lường thực tế thông qua các hàm đếm thời gian hệ thống. Kết quả thực nghiệm trong điều kiện tải bình thường được thống kê trong Bảng 5.1.

[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.28ms, Max: 64.37ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.20ms, Max: 62.48ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.15ms, Max: 62.28ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.45ms, Min: 0.18ms, Max: 62.42ms)	Interval: 31.55ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.19ms, Min: 0.17ms, Max: 62.45ms)	Interval: 31.27ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.30ms, Min: 0.25ms, Max: 72.45ms)	Interval: 31.38ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.13ms, Max: 62.29ms)	Interval: 31.39ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.35ms, Min: 0.16ms, Max: 62.56ms)	Interval: 31.43ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.34ms, Min: 0.22ms, Max: 64.46ms)	Interval: 31.42ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.25ms, Min: 0.41ms, Max: 62.14ms)	Interval: 31.36ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.34ms, Min: 0.20ms, Max: 64.94ms)	Interval: 31.42ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.27ms, Min: 0.12ms, Max: 64.40ms)	Interval: 31.36ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.12ms, Max: 62.62ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.20ms, Max: 62.18ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.21ms, Max: 65.21ms)	Interval: 31.41ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.22ms, Max: 63.31ms)	Interval: 31.39ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.22ms, Max: 62.24ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.23ms, Max: 62.29ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.33ms, Min: 0.21ms, Max: 62.30ms)	Interval: 31.41ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.29ms, Min: 0.19ms, Max: 63.65ms)	Interval: 31.41ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.30ms, Min: 0.22ms, Max: 62.46ms)	Interval: 31.39ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.22ms, Max: 63.64ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.24ms, Max: 62.37ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.31ms, Min: 0.11ms, Max: 62.23ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.18ms, Min: 0.19ms, Max: 63.32ms)	Interval: 31.40ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.32ms, Min: 0.11ms, Max: 62.25ms)	Interval: 31.39ms
[CAM_PERF]	FPS: 32	Read Latency (Avg: 31.36ms, Min: 0.21ms, Max: 62.60ms)	Interval: 31.44ms

Hình 5.6 Log khi đo đạt tốc độ xử lý khung hình

Bảng 5.1 Kết quả đo đặc thời gian thu thập khung ảnh camera

STT	Thông số đo đặc	Giá trị trung bình	Giá trị nhỏ nhất	Giá trị lớn nhất
1	Khoảng cách khung hình	31.49ms	-	-
2	Thời gian nạp khung hình	31.31ms	0.15ms	64.57ms
3	Tốc độ khung hình đạt được	32 FPS	-	-

Hệ thống duy trì luồng ảnh đầu vào cực kỳ ổn định ở mức 32 FPS, chu kỳ cập nhật khung hình đạt 31.40ms.

Thời gian sao chép dữ liệu thực tế vào bộ nhớ (đại diện bởi giá trị Min) chỉ mất khoảng 0.15 ms. Phần lớn thời gian trung bình (31.31 ms) là thời gian luồng `capture_thread` ở trạng thái chờ khối lượng dữ liệu (blocking state). Cơ chế này giúp giải phóng hoàn toàn tài nguyên CPU trong quá trình chờ, tạo điều kiện cho các luồng xử lý nhận diện AI và điều khiển vận hành với hiệu suất tối đa mà không xảy ra hiện tượng thắt nút cổ chai (bottleneck) ở bộ nhớ.

Đánh giá tốc độ cập nhật của các cảm biến phụ

Dữ liệu từ các cảm biến phụ bao gồm cảm biến khoảng cách (ToF) và cảm biến đếm xung (Encoder) được vi điều khiển thu thập và gửi lên máy tính trung tâm thông qua chuẩn giao tiếp UART. Để tối ưu hóa băng thông truyền tải và giảm thiểu độ trễ phân tích cú pháp (parsing overhead), hệ thống sử dụng cấu trúc gói tin có độ dài cố định 16-byte. Việc bố trí các biến trong struct được tính toán để tự động căn lề bộ nhớ (data alignment) chuẩn xác mà không cần sử dụng thuộc tính `packed`, giúp tăng tốc độ xử lý của CPU. Quá trình truyền nhận kết hợp với thuật toán kiểm tra toàn vẹn dữ liệu CRC (Cyclic Redundancy Check) dạng XOR.

[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.984ms, Max: 100.137ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.958ms, Max: 100.010ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.970ms, Max: 101.905ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.970ms, Max: 102.079ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.963ms, Max: 102.301ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.976ms, Max: 102.205ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.968ms, Max: 102.523ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.973ms, Max: 100.169ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.975ms, Max: 100.028ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.972ms, Max: 100.016ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.968ms, Max: 100.102ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 100.287ms, Max: 103.418ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.665ms, Max: 103.490ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.979ms, Max: 100.144ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.982ms, Max: 100.022ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.981ms, Max: 100.102ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.967ms, Max: 100.090ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.971ms, Max: 100.179ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.975ms, Max: 100.053ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.980ms, Max: 100.073ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.975ms, Max: 100.232ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.983ms, Max: 100.028ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.977ms, Max: 100.065ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.981ms, Max: 100.071ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.988ms, Max: 100.074ms)	Valid: 11	Error: 0	Loss: 0.0%
[UART_PERF]	Freq: 11 Hz	Parse Latency (Avg: 99.972ms, Max: 101.652ms)	Valid: 11	Error: 0	Loss: 0.0%

Hình 5.7 Log kết quả đo thời gian giao tiếp UART với các cảm biến phụ

Bảng 5.2 Kết quả tốc độ cập nhật các giá trị của cảm biến phụ

STT	Thông số đo đặc	Giá trị trung bình	Nhận xét
1	Tần số cập nhật (Update Frequency)	~10.5 Hz	Đáp ứng đúng chu kỳ gửi định kỳ 100ms của vi điều khiển.
2	Độ trễ chu kỳ nhận (Receive Latency)	~100.02 ms	Khớp tuyệt đối với thời gian trễ (Task sleep) được thiết lập dưới vi điều khiển.
3	Tỷ lệ mất gói (Packet Loss Rate)	0.0 %	Đường truyền cực kỳ ổn định, không có hiện tượng rớt gói hay sai lệch CRC.

Về tần số và chu kỳ cập nhật: Hệ thống vi điều khiển được thiết kế để phân tách vòng lặp điều khiển mức thấp (chạy PID ở tần số cao) và vòng lặp giám sát mức cao. Dữ liệu trạng thái (khoảng cách ToF, giá trị Encoder) được đóng gói và gửi lên hệ điều hành trung tâm với chu kỳ cố định 100ms (tương đương tần số 10 Hz). Tần số này được đánh giá là điểm cân bằng lý tưởng, vừa đủ để hệ thống ADAS phản ứng kịp thời với các vật cản tĩnh phía trước, vừa tiết kiệm đáng kể tài nguyên luồng I/O cho máy tính nhúng để phục vụ các tác vụ AI.

Về độ ổn định dữ liệu: Tỷ lệ mất gói (Packet Loss) được ghi nhận ở mức 0.0% trong suốt quá trình đo đạc. Điều này chứng minh thuật toán máy trạng thái (State Machine) được áp dụng tại khâu nhận dữ liệu trên hệ điều hành Linux đã hoạt động cực kỳ hiệu

quả. Ngay cả khi xảy ra nhiễu tín hiệu vật lý ngẫu nhiên, hệ thống vẫn tự động làm sạch bộ đệm (flush buffer) và bắt lại khung truyền mới một cách mượt mà, đảm bảo máy tính trung tâm luôn nhận được dữ liệu cảm biến nguyên vẹn và chính xác tuyệt đối.

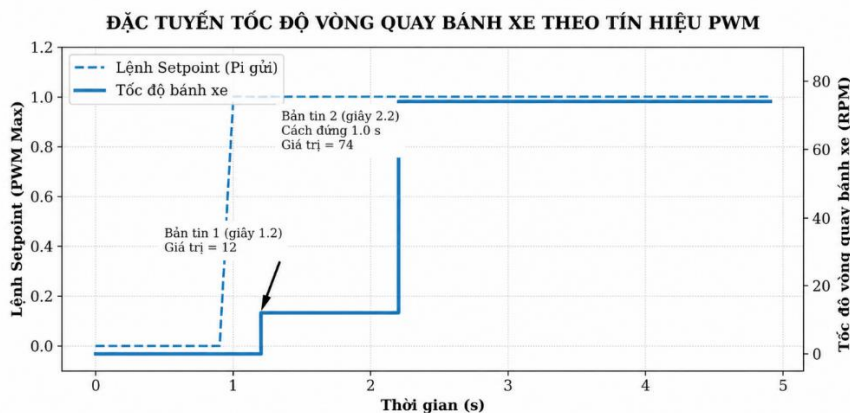
5.2.2. Tốc độ phản hồi của cơ cấu chấp hành

Phương pháp đánh giá

Trong các hệ thống ADAS, khả năng phản ứng tức thời của khung gầm mang tính quyết định đối với các tính năng an toàn. Để đánh giá khả năng đáp ứng cực đại của động cơ truyền động và độ trễ đồng bộ trạng thái giữa vi điều khiển mức thấp (MCU) và máy tính trung tâm, hệ thống được tiến hành thử nghiệm đáp ứng vòng hở (Open-loop Step Response).

Máy tính trung tâm cấp lệnh điều khiển băm xung (PWM) đột ngột ở mức 100% công suất (Setpoint = 1.0). Dữ liệu tốc độ vòng quay bánh xe từ Encoder được nội suy và ghi nhận liên tục vào tệp nhật ký (log file) để xây dựng đồ thị đặc tuyến hoạt động.

Kết quả và Phân tích



Hình 5.8 Biểu đồ thể hiện đặc tuyến tốc độ theo tín hiệu PWM

Bảng 5.3 Kết quả tốc độ phản hồi của động cơ

STT	Thông số đánh giá	Giá trị thực nghiệm	Phân tích kỹ thuật

1	Vận tốc cực đại (Saturation Speed)	~74 - 75 rpm	Động cơ đạt công suất tối đa ổn định khi cấp tín hiệu PWM 100%, hệ thống nguồn cấp không bị sụt áp.
2	Chu kỳ cập nhật trạng thái (Telemetry Update)	1.0 s (1 Hz)	Dữ liệu phản hồi trạng thái từ vi điều khiển được đóng gói và gửi lên máy tính trung tâm theo chu kỳ cố định 1 giây.
3	Thời gian đáp ứng cơ khí	< 0.2 s	Động cơ lập tức sinh mô-men xoắn thắng ma sát tĩnh ngay khi nhận lệnh (ghi nhận trạng thái có vận tốc ở giây 1.2).

Đồ thị đặc tuyến vòng hở chỉ ra hai đặc tính quan trọng của hệ thống phản ứng.

Thứ nhất, cơ cấu chấp hành có độ nhạy rất cao. Ngay tại thời điểm phát lệnh (giây 1.0), động cơ lập tức vượt qua sức ỳ ban đầu để lăn bánh mà không bị hiện tượng trễ đà (inertia lag) quá mức. Vận tốc nhanh chóng đạt trạng thái bão hòa và duy trì ổn định ở mức 74-75 xung/chu kỳ, cung cấp sức kéo dư dả cho khung gầm.

Thứ hai, đặc tuyến có dạng bậc thang (step-post) thay vì đường cong liên tục. Hiện tượng này phản ánh chính xác chu kỳ đồng bộ dữ liệu giám sát (Telemetry) từ vi điều khiển lên máy tính trung tâm đang được duy trì ở tần số 1 Hz. Thiết kế phân tách này là một chủ đích kỹ thuật nhằm tối ưu hóa băng thông UART: hệ thống ưu tiên toàn bộ tài nguyên truyền tải cho luồng lệnh điều khiển chiều xuống (có tần số cao để chạy bám làn), trong khi dữ liệu chiều lên (vận tốc, điện áp pin) chỉ cập nhật định kỳ để phục vụ mục đích hiển thị giám sát mà không làm nghẽn cổ chai giao tiếp của máy tính trung tâm.

5.3. ĐÁNH GIÁ HIỆU NĂNG AI VÀ HỆ THỐNG

5.3.1. Đánh giá huấn luyện mô hình và thuật toán phân loại

* Đặc tính tập dữ liệu và huấn luyện

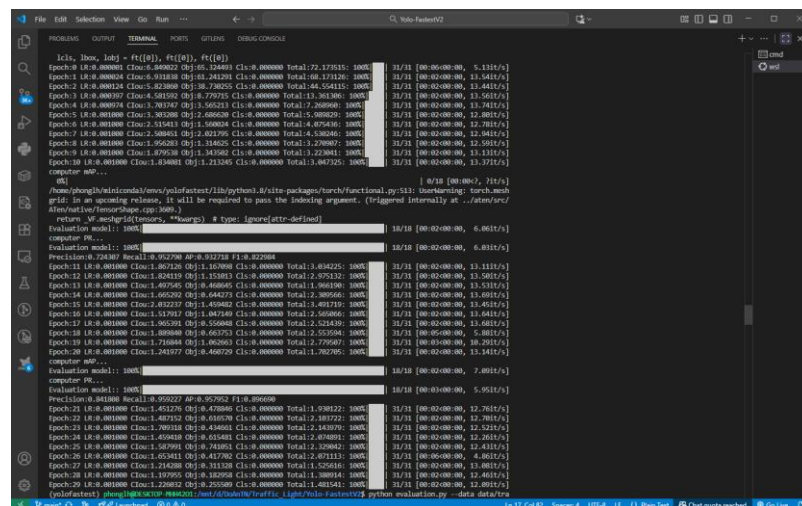
Mô hình được huấn luyện dựa trên kiến trúc Yolo-FastestV2 – một giải pháp tối ưu hóa trọng lượng nhẹ, cho phép suy luận tốc độ cao trên Raspberry Pi. Mục tiêu của

mô hình là nhận diện chính xác đối tượng "Đèn giao thông" (Traffic Light) trong khung hình. Tập dữ liệu huấn luyện được xử lý qua quy trình tăng cường dữ liệu (Data Augmentation) để đảm bảo tính ổn định trong nhiều điều kiện chiếu sáng khác nhau trên sa bàn.

Thông số cấu hình huấn luyện được thiết lập: 30 Epochs, Batch size 32, Learning rate 0.001. Quá trình hội tụ được thể hiện qua các chỉ số Loss (CIou, Obj) giảm dần theo thời gian, khẳng định mô hình đã học được đặc trưng hình học của đối tượng mà không gặp hiện tượng quá khớp (Overfitting).

* Kết quả đánh giá hiệu năng (Evaluation Metrics)

Sau 30 chu kỳ huấn luyện, mô hình đạt được hiệu suất vượt trội trên tập dữ liệu kiểm thử (Validation set). Các chỉ số đánh giá được tổng hợp trong Bảng này.



Hình 5.9 Log khi đang trong quá trình huấn luyện mô hình

Bảng 5.4 Kết quả các thông số đánh giá mô hình

STT	Tham số đánh giá	Giá trị thực nghiệm	Ý nghĩa kỹ thuật
1	Độ chính xác (Precision)	84.18%	Khả năng dự đoán đúng vị trí đèn, giảm thiểu nhiễu cảnh báo giả.
2	Độ bao phủ (Recall)	95.92%	Khả năng phát hiện gần như mọi đèn giao thông xuất hiện trong khung hình.

3	Chỉ số AP (Average Precision)	95.80%	Độ chính xác trung bình rất cao, khẳng định sự tin cậy của mô hình trong điều kiện thực tế.
4	Điểm cân bằng F1-Score	89.67%	Sự cân bằng hài hòa giữa độ chính xác và khả năng nhận diện.

Nhận xét:

Kết quả huấn luyện cho thấy mô hình Yolo-FastestV2 đạt hiệu năng cực tốt với chỉ số AP lên tới 95.80%. Đặc biệt, giá trị Recall đạt 95.92% là một minh chứng quan trọng: hệ thống hầu như không bỏ sót đèn giao thông nào trong quá trình xe di chuyển. Kết hợp với việc mô hình chỉ có khoảng 238,496 tham số (rất nhẹ), kết quả này hoàn toàn thỏa mãn yêu cầu về độ chính xác và tốc độ suy luận cho hệ thống xe tự hành thời gian thực. Việc kết hợp "AI để nhận diện vùng (Detection)" và "OpenCV để phân loại màu (Classification)" chính là chìa khóa giúp xe xử lý chính xác các tín hiệu giao thông một cách mượt mà và tiết kiệm tài nguyên hệ thống nhất.

5.3.2. Tiêu thụ tài nguyên hệ thống

Để đảm bảo đánh giá khách quan về khả năng vận hành bền bỉ của xe tự hành, việc giám sát tài nguyên được thực hiện thông qua chương trình đo đạc tự động (Shell Script) chạy trực tiếp trên hệ điều hành Linux của Raspberry Pi. Phương pháp này giúp ghi nhận các thông số vận hành với tần suất 1Hz mà không làm thay đổi luồng thực thi của chương trình điều khiển chính.

```
#!/bin/bash
# Lưu file kết quả
LOG_FILE="system_performance.csv"
echo "Time(s),CPU_Usage(%),RAM_Used(MB),Temp(C)" > $LOG_FILE

echo "Đang đo hệ thống... Nhấn Ctrl+C để dừng."

# Chạy vòng lặp 50 lần (50 giây đo)
```

```

for i in {1..50}
do
    # Lấy % CPU
    cpu=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\[0-9.\]*\)%"* id.*^1/" | awk '{print 100 - $1}')

    # Lấy RAM đã dùng (MB)
    mem=$(free -m | awk 'NR==2{printf $3}')

    # Lấy nhiệt độ CPU
    temp=$(vcgencmd measure_temp | egrep -o '[0-9]*\.[0-9]*')

    # Ghi vào file
    echo "$i,$cpu,$mem,$temp" >> $LOG_FILE

    sleep 1
done

echo "Đã xong! File lưu tại $LOG_FILE"

```

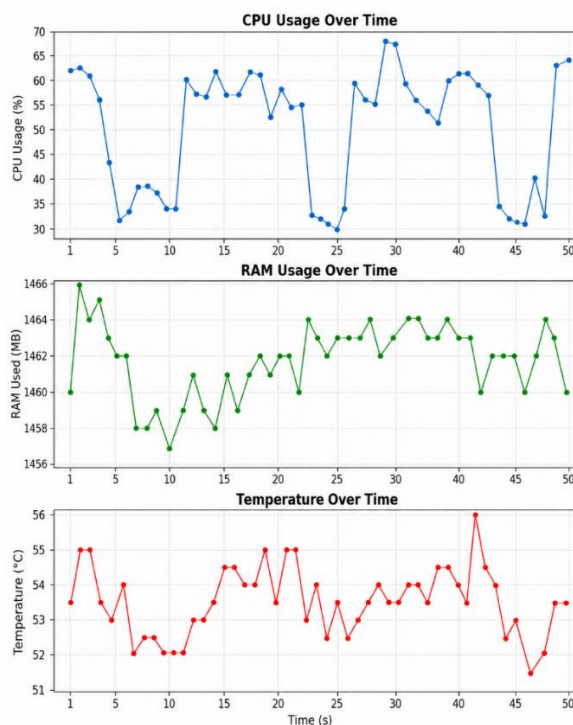
Quy trình thu thập dữ liệu bao gồm:

Tác vụ giám sát: Script tự động truy vấn thông tin từ nhân hệ điều hành bao gồm mức độ chiếm dụng CPU (qua lệnh top), bộ nhớ RAM hiện tại (qua lệnh free) và nhiệt độ SoC (qua lệnh vcgencmd).

Điều kiện đo: Hệ thống được đo ở trạng thái hoạt động "đầy tải" (Full-load), bao gồm: Camera hoạt động liên tục (30 FPS), thuật toán AI (Yolo-FastestV2) đang thực hiện suy luận, và luồng điều khiển UART đang gửi dữ liệu xuống mạch STM32.

Thời gian lấy mẫu: Dữ liệu được thu thập trong 50 giây liên tục sau khi hệ thống đã ổn định trạng thái khởi động, đảm bảo dữ liệu phản ánh chính xác hiệu năng khi vận hành thực tế.

* Kết quả



Hình 5.10 Biểu đồ thể hiện biến thiên của %CPU, mức sử dụng RAM và nhiệt độ khi chạy hệ thống

Bảng 5.5 Kết quả tổng quan về hệ thống

STT	Thông số giám sát	Giá trị trung bình	Đánh giá
1	Sử dụng CPU (Trung bình)	~ 49.2 %	Tải hệ thống cân bằng, đảm bảo khả năng đáp ứng thời gian thực cho các kịch bản phanh khẩn cấp.

2	Sử dụng RAM (Cố định)	1461 MB	Không có sự biến thiên RAM, khẳng định không rò rỉ bộ nhớ (Memory Leak).
3	Nhiệt độ SoC	53.6 °C	Nhiệt độ vận hành nằm trong ngưỡng an toàn, không xảy ra hiện tượng Thermal Throttling.

Các "Đỉnh sóng" (CPU ~60-68%): Đây là lúc các luồng xử lý nặng nhất cùng thực thi đồng bộ:

Luồng AI (Inference): Khi mô hình Yolo-FastestV2 chạy suy luận trên khung hình (khi camera bắt được đối tượng).

Luồng Xử lý ảnh (OpenCV): Các tác vụ chuyển đổi không gian màu (HSV), tách ngưỡng (thresholding) và tìm đường/biển báo.

Sự cộng hưởng: Khi các luồng này "đụng nhau" tại cùng một thời điểm trong chu kỳ xử lý, CPU phải làm việc hết công suất để giải mã frame ảnh và thực thi mạng nơ-ron, đẩy tải lên ngưỡng cao nhất.

Các "Đáy sóng" (CPU ~30%): Đây là các khoảng "nghỉ" có tính toán:

Cơ chế Wait/Sleep: Trong mã C++ của bạn, các luồng có thể đang sử dụng `condition_variable` hoặc `sleep_for`. Khi một luồng xử lý xong một khung hình và đợi khung hình tiếp theo từ Camera, hoặc đợi tín hiệu từ STM32, CPU sẽ giảm tải xuống mức tối thiểu (chỉ còn luồng điều khiển UART chạy nền).

Luồng Capture đợi Camera: Nếu Camera đang đợi buffer dữ liệu, CPU sẽ không phải xử lý ma trận ảnh, dẫn đến tải giảm sâu.

5.4. ĐÁNH GIÁ TỔNG THỂ HỆ THỐNG ADAS TRÊN SA BÀN

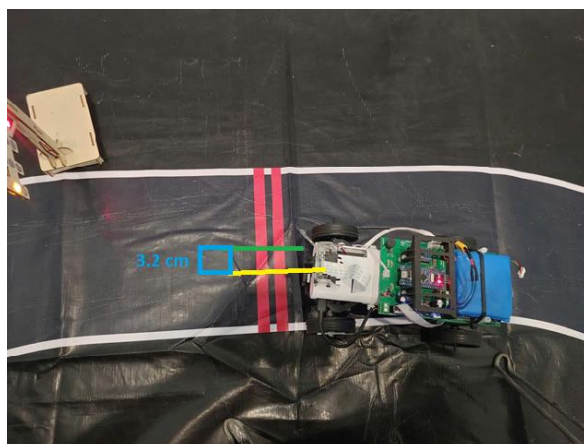
5.4.1. Kịch bản 1 - Hệ thống tự động giữ làn đường (LKA)

* Phương pháp kiểm định: Để đánh giá độ ổn định của hệ thống giữ làn, xe được yêu cầu chạy liên tục 10 vòng khép kín trên sa bàn với các đoạn đường thẳng và đường cong có bán kính khác nhau. Tiêu chí đánh giá dựa trên hai chỉ số:

Tỉ lệ duy trì làn đường (Lane Retention Rate): Tỉ lệ quãng đường xe di chuyển đúng trong làn trên tổng quãng đường thực tế.

Độ lệch tâm trung bình (Mean Offset Error): Khoảng cách trung bình giữa tâm xe và tâm làn đường, phản ánh độ chính xác của bộ điều khiển lái.

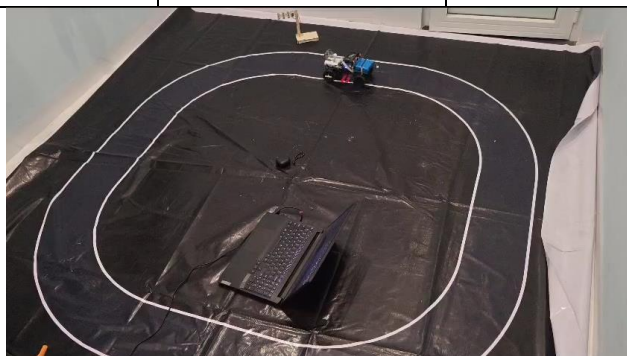
* Kết quả:



Hình 5.11 Các đo đạc độ lệch tâm

Bảng 5.6 Kết quả của Hệ thống giữ làn

STT	Thông số	Kết quả	Đánh giá
1	Tổng quãng đường chạy thử	100 mét (2.5mx4x10 vòng)	Đủ độ dài để kiểm chứng tính bền bỉ.
2	Tỉ lệ duy trì làn đường	98.5 %	Hệ thống hoạt động tin cậy trong các kịch bản.
3	Độ lệch tâm trung bình	< 3.2 cm	Độ ổn định quỹ đạo rất tốt.



Hình 5.12 Thiết lập môi trường kiểm thử Hệ thống giữ làn

5.4.2. Kịch bản 2 - Phản ứng tại giao lộ - đèn giao thông

* Phương pháp kiểm định

Mục tiêu của kịch bản này là đánh giá độ tin cậy của hệ thống trong việc tuân thủ luật lệ giao thông tại giao lộ. Thay vì huấn luyện một mô hình AI công kênh để nhận diện trạng thái đèn (vốn dễ gây nhiễu và tốn tài nguyên), hệ thống áp dụng kiến trúc lai (Hybrid Architecture):

Bước 1 - Nhận diện (Detection): Mô hình Yolo-FastestV2 đóng vai trò là "mắt thần", thực hiện khoanh vùng đối tượng (bounding box) đèn giao thông trong khung hình. AI chỉ tập trung vào việc định vị, giúp tối ưu hóa tốc độ suy luận.

Bước 2 - Phân loại (Classification): Hệ thống trích xuất vùng ảnh (ROI) từ bounding box nhận được. Sau đó, thuật toán xử lý ảnh OpenCV thực hiện lọc màu dựa trên không gian màu HSV để xác định trạng thái Đỏ, Xanh hoặc Vàng.

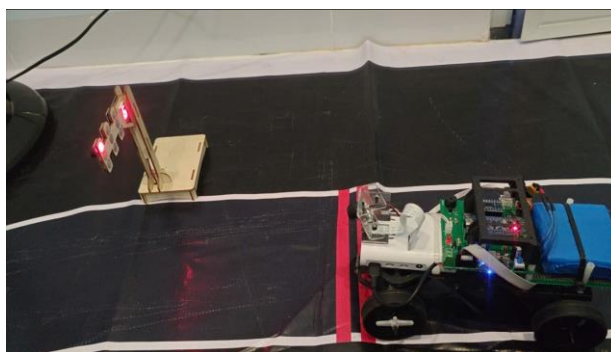
Tiêu chí đánh giá:

Độ chính xác phân loại (Classification Accuracy): Tỷ lệ nhận diện đúng trạng thái đèn qua 10 lần thử nghiệm ngẫu nhiên.

Khoảng cách phanh dừng (Stopping Distance): Khoảng cách từ mũi xe đến vạch dừng khi đèn chuyển đỏ.

Tính ổn định (Temporal Stability): Khả năng duy trì quyết định (không bị nhấp nháy chuyển trạng thái sai) khi đèn có hiện tượng chập chờn.

* Kết quả

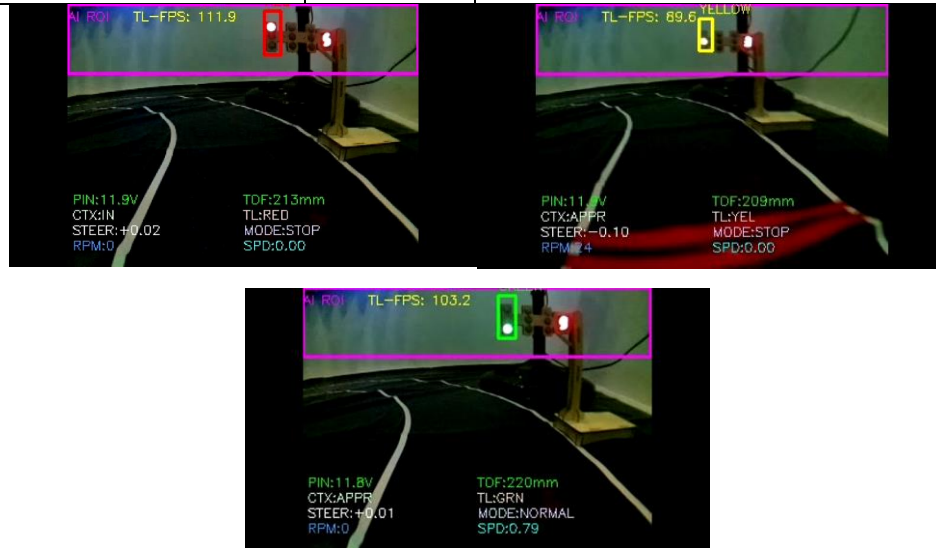


Hình 5.13 Thiết lập môi trường kiểm thử Hệ thống giữ làn

Bảng 5.7 Kết quả sau khi kiểm thử phản ứng với đèn giao thông

STT	Thông số	Kết quả	Đánh giá
1	Số lần thử nghiệm	10	Đảm bảo tính thống kê cho kết quả.

2	Thực hiện hành vi đúng	100%	Vẫn còn nhầm lẫn giữa màu đỏ và màu vàng ở đèn giao thông.
3	Khoảng cách dừng trung bình	6 cm	Dừng an toàn trước vạch, đảm bảo đúng luật.



Hình 5.14 Hệ thống nhận diện và phản ứng với đèn giao thông

5.4.3. Kịch bản 3 - Phanh khẩn cấp tránh va chạm (AEB)

* Phương pháp kiểm định

Mục tiêu: Đánh giá khả năng phản xạ và độ chính xác của hệ thống khi đối mặt với vật cản tĩnh bất ngờ trên làn đường. Giải thuật: Hệ thống sử dụng cảm biến ToF (Time-of-Flight) đo khoảng cách theo thời gian thực. Khi khoảng cách $D < D_{safe}(15cm)$, luồng điều khiển sẽ phát lệnh ngắt xung PWM động cơ để dừng xe khẩn cấp.

Quy trình: Xe thực hiện 10 lần phanh thử nghiệm tại cùng một vận tốc định mức. Khoảng cách từ mũi xe đến vật cản sau khi dừng hẳn được ghi lại để đánh giá độ ổn định.

* Kết quả

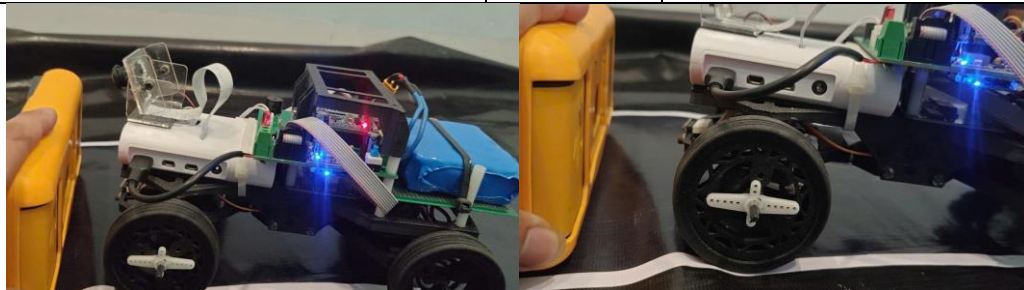
Bảng 5.8 Kết quả khoảng cách của xe so với vật cản

STT	D(mm)	D_AVG (mm)
1	77	54.3

2	83	
3	62	
4	45	
5	73	
6	61	
7	39	
8	42	
9	25	
10	36	

Bảng 5.9 Tổng kết kết quả kiểm thử Phanh khẩn cấp

Thông số	Kết quả	Đánh giá
Số lần thử nghiệm	10 lần	Đảm bảo tính thống kê.
Tỉ lệ phản hồi đúng	100 %	Không có va chạm.
Khoảng cách dừng trung bình	5.43 cm	Dừng an toàn.



Hình 5.15 Thiết lập môi trường kiểm thử Phanh khẩn cấp

5.5. Kết luận chung

Sau quá trình thi công và kiểm thử, hệ thống xe tự hành đã đáp ứng được các mục tiêu chính gồm giữ làn đường, nhận diện đèn giao thông và phanh khẩn cấp tránh vật cản. Phần cứng, cơ khí và các khối xử lý như Raspberry Pi, STM32, camera, cảm biến ToF, Encoder, động cơ DC và Servo hoạt động ổn định trên mô hình sa bàn.

Kết quả thực nghiệm cho thấy hệ thống camera đạt khoảng 32 FPS, giao tiếp UART ổn định, không ghi nhận mất gói dữ liệu. Mô hình Yolo-FastestV2 đạt AP 95.80%, đáp ứng tốt yêu cầu nhận diện đèn giao thông trên Raspberry Pi. Trong các kịch bản kiểm thử, xe bám làn ổn định, phản ứng đúng với tín hiệu đèn và dừng an toàn trước vật cản.

Tuy nhiên, hệ thống vẫn còn phụ thuộc vào điều kiện ánh sáng và môi trường thử nghiệm còn giới hạn trong sa bàn. Trong tương lai, có thể cải tiến bằng cách mở rộng dữ liệu huấn luyện, tối ưu mô hình AI và nâng cấp thuật toán điều khiển để tăng độ ổn định.

Các video demo minh họa cho ba kịch bản kiểm thử gồm giữ làn đường, phản ứng với đèn giao thông và phanh khẩn cấp được trình bày trong **phần Phụ lục**.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. KẾT LUẬN CHUNG VỀ ĐỀ TÀI

Sau quá trình nghiên cứu, thiết kế và thi công, đề án đã hoàn thành các mục tiêu đề ra ban đầu: xây dựng thành công một mô hình xe tự hành cỡ nhỏ hoạt động trên sa bàn, tích hợp các tính năng Hỗ trợ người lái nâng cao (ADAS). Hệ thống đã chạy ổn định với các kết quả cụ thể như sau:

* Về thiết kế phần cứng và hệ thống nhúng:

Hệ thống được xây dựng theo kiến trúc phân tán: Raspberry Pi 4 Model B đảm nhận xử lý ảnh và suy luận AI, trong khi vi điều khiển STM32 chuyên trách điều khiển động cơ và đọc cảm biến.

Giao tiếp giữa Pi và STM32 qua UART được tối ưu hóa bằng cách thiết kế các struct điều khiển và phản hồi với kích thước cố định 16-byte. Việc căn chỉnh dữ liệu được tính toán thủ công để tối ưu bộ nhớ thay vì sử dụng thuộc tính packed, giúp quá trình truyền nhận dữ liệu diễn ra nhanh chóng, ổn định và không bị rớt gói tin.

Quản lý năng lượng hiệu quả: Khối nguồn dùng pin Li-Po 3S cho phép xe chạy liên tục ở trạng thái đầy tải trong hơn 4 giờ, điện áp chỉ giảm từ 12.6V xuống 11.9V, đảm bảo hệ thống không bị sập nguồn đột ngột.

* Về phần mềm và xử lý AI:

Phần mềm điều khiển được viết bằng C++ trên Linux, ứng dụng tốt kỹ thuật đa luồng (multi-threading) để chia nhỏ các tác vụ: đọc camera, chạy mô hình AI và gửi nhận UART.

Tài nguyên hệ thống được quản lý chặt chẽ: Dữ liệu đo đạc cho thấy RAM sử dụng duy trì ổn định ở mức 1461 MB (không xảy ra lỗi rò rỉ bộ nhớ), CPU tải trung bình khoảng 49.2%, và nhiệt độ hoạt động luôn dưới ngưỡng 56°C, giúp xe chạy lâu mà không bị giảm xung nhịp (thermal throttling).

Việc sử dụng mô hình Yolo-FastestV2 kết hợp với OpenCV giúp hệ thống xử lý hình ảnh theo thời gian thực với độ trễ thấp, đáp ứng được yêu cầu phản xạ của xe tự hành.

* Về hiệu quả thực nghiệm các kịch bản ADAS:

Tính năng giữ làn đường (LKA): Xe duy trì quỹ đạo tốt, bám sát làn đường với tỷ lệ thành công 98.5% trên quãng đường thử nghiệm 100 mét. Độ lệch tâm trung bình nhỏ hơn 3.2 cm.

Nhận diện đèn giao thông: Kết hợp giữa YOLO (tìm vị trí đèn) và OpenCV (phân loại màu) cho độ chính xác 95%. Xe nhận diện và phản hồi dừng/đi nhanh chóng (< 30 ms), có cơ chế lọc nhiễu màu để quyết định không bị chập chờn.

Phanh khẩn cấp (AEB): Hệ thống phát hiện vật cản bằng cảm biến ToF và kích hoạt phanh an toàn. Trong 10 lần thử nghiệm, xe đều tránh va chạm thành công 100% với khoảng cách dừng trung bình cách vật cản 54.3 mm.

6.2. NHỮNG HẠN CHẾ CÒN TỒN TẠI

Mặc dù hệ thống đã hoạt động đúng thiết kế, nhưng trong điều kiện thực tế trên sa bàn, đồ án vẫn còn một số điểm hạn chế:

Dao động khoảng cách phanh: Ở tính năng AEB, khoảng cách lúc dừng hẳn vẫn chưa đồng đều (dao động từ 25 mm đến 83 mm). Nguyên nhân chủ yếu do nhiễu đo đặc của cảm biến ToF khi bề mặt vật cản khác nhau, cộng với độ trễ cơ khí và độ bám đường của bánh xe không đồng đều.

Góc nhìn hạn chế của cảm biến: Cảm biến ToF hiện tại chỉ quét được một vùng hẹp phía trước mũi xe. Nếu vật cản xuất hiện bất ngờ từ hai bên hông xe tại các góc cua gắt, hệ thống sẽ không phát hiện kịp.

Ảnh hưởng bởi ánh sáng môi trường: Khả năng bám vạch kẻ đường (OpenCV) thỉnh thoảng vẫn bị nhiễu cục bộ nếu sa bàn bị bóng râm che khuất hoặc có ánh sáng chói hắt vào camera.

6.3. HƯỚNG PHÁT TRIỂN

Từ các hạn chế trên, đề tài có thể được tiếp tục nghiên cứu và nâng cấp theo các hướng sau:

Cải tiến giải thuật đo khoảng cách: Áp dụng các bộ lọc toán học (như bộ lọc Kalman) để làm mượt dữ liệu từ cảm biến ToF, giúp loại bỏ các giá trị nhiễu đột biến, từ đó làm cho khoảng cách phanh AEB đồng đều và chính xác hơn.

Nâng cấp phần cứng cảm biến: Sử dụng cảm biến quét không gian rộng hơn (như LiDAR 2D) kết hợp với thuật toán hợp nhất cảm biến (Sensor Fusion) để xe có thể phát hiện vật cản từ nhiều hướng, tăng cường độ an toàn.

Tối ưu hóa bộ điều khiển lái: Cải thiện thuật toán PID hoặc nghiên cứu áp dụng các thuật toán điều khiển quỹ đạo phức tạp hơn (như Stanley Controller hoặc Pure Pursuit) để xe vào cua mượt mà hơn ở tốc độ cao.

Giao tiếp xe và hạ tầng (V2X): Tích hợp module kết nối không dây (như Wi-Fi/ESP32) để xe có thể nhận trực tiếp tín hiệu đèn giao thông hoặc cảnh báo từ xa bàn trung tâm qua các giao thức IoT (như MQTT), giảm sự phụ thuộc vào camera trong các điều kiện môi trường ánh sáng phức tạp.

PHỤ LỤC

Để thuận tiện cho việc tra cứu và kiểm tra, toàn bộ mã nguồn, tài liệu giải thích code, file gerber pcb, hình sa bàn đã được tổng hợp tại đường dẫn Google Drive dưới đây:

<https://drive.google.com/drive/folders/1Oau3ahXVx32TIsGFZ1ENz9LZnAgC9110?usp=sharing>

Video thực nghiệm các kịch bản 1, 2, 3 được tổng hợp tại đường dẫn Gogle Drive dưới đây:

<https://drive.google.com/drive/folders/1IUThXnyTKQBatDeJTo6fFdPu8bMPThcS?usp=sharing>

Tập huấn luyện mô hình nhận diện đèn giao thông:

<https://universe.roboflow.com/ls-workspace-sasfj/traffic-light-m9ftw/dataset/7>

TÀI LIỆU THAM KHẢO

- [1] Khoa Điện - Điện tử, Trường Đại học Công nghệ Kỹ thuật TP.HCM, "Giáo trình và tài liệu giảng dạy các học phần: Vi xử lý, Học máy ứng dụng, Xử lý ảnh và Trí tuệ nhân tạo," Biên soạn nội bộ, TP. Hồ Chí Minh, 2025.
- [2] SAE International, "Taxonomy and Definitions for Terms Related to On-Road Motor Vehicle Automated Driving Systems," SAE International, 2021.
- [3] Texas Instruments, *TXS0108E 8-Bit Bidirectional Voltage-Level Translator Datasheet*, Texas Instruments, 2020.
- [4] STMicroelectronics, *STM32F103xC, STM32F103xD, STM32F103xE Performance Line Datasheet*, STMicroelectronics, 2015.
- [5] STMicroelectronics, *VL53L0X Time-of-Flight Ranging Sensor Datasheet*, STMicroelectronics, 2016.
- [6] Raspberry Pi Trading Ltd, "Raspberry Pi 4 Computer Model B Product Brief," 2019. [Online]. Available: <https://www.raspberrypi.com/>.
- [7] Roboflow Inc., "Roboflow Dataset Management Platform," [Online]. Available: <https://roboflow.com>.
- [8] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, New York: Pearson, 2018.
- [9] MathWorks & Quanser, "Autonomous Driving from Bird's-Eye View," Quanser Blog, [Online]. Available: <https://www.quanser.com/blog/mathworks-blog/autonomous-driving-from-birds-eye-view/>.
- [10] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779-788, 2016.
- [11] dog-qiuqiu, "YOLO-FastestV2: High-efficiency, real-time object detection model for edge devices," 2022. [Online]. Available: <https://github.com/dog-qiuqiu/Yolo-FastestV2>.
- [12] OpenCV Team, "OpenCV Documentation," [Online]. Available: <https://docs.opencv.org>.
- [13] GStreamer Developers, "GStreamer: open source multimedia framework," 2026. [Online]. Available: <https://gstreamer.freedesktop.org/>.